



BITS Pilani
Pilani Campus

S2-23_AIMLCZG515: Distributed Machine Learning

Dr. Chandra Sekhar V

Chandrasekhar.v@pilani.bits-pilani.ac.in



Session 1
Date – 25th May 2024
Time – 1:40 PM to 3:40 PM

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.

Session Content



- Objective of course
- Evaluation Plan
- Course Overview



Objectives of the course

Introduce you to the foundational concepts of

1. Foundations of Parallel and Distributed Computing
2. Data Parallelism vs. Model Parallelism
3. Algorithm Scaling Examples
4. Distributed Deep Learning Algorithms
5. Federated Learning

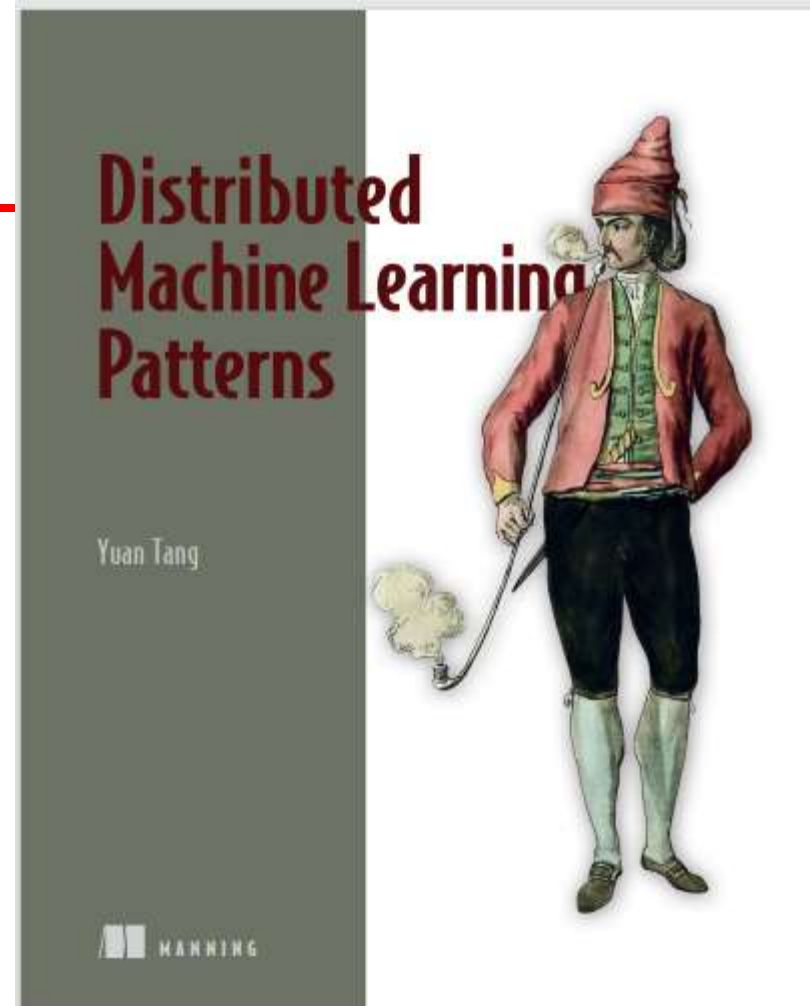


Welcome to AIMLCZG515

- This course gives an overview of Distributed Machine Learning,
- In particular, we will cover distributed machine learning algorithms.
- You will develop fundamental and practical skills in developing scaling machine learning algorithms.

Text Book

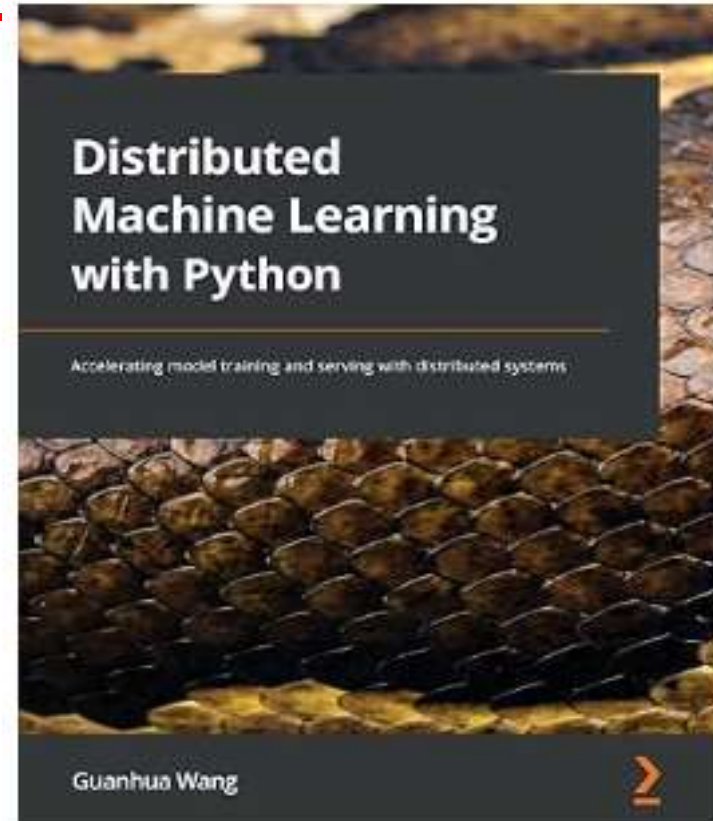
- Yuan Tang, Distributed Machine Learning patterns, Manning publications.
- In addition, we will extensively use online materials (video lectures, blog posts, surveys, papers, etc.)



Text Book



- Guanhua Wang, Distributed Machine Learning with python, Packt publications.
- In addition, we will extensively use online materials (video lectures, blog posts, surveys, papers, etc.)



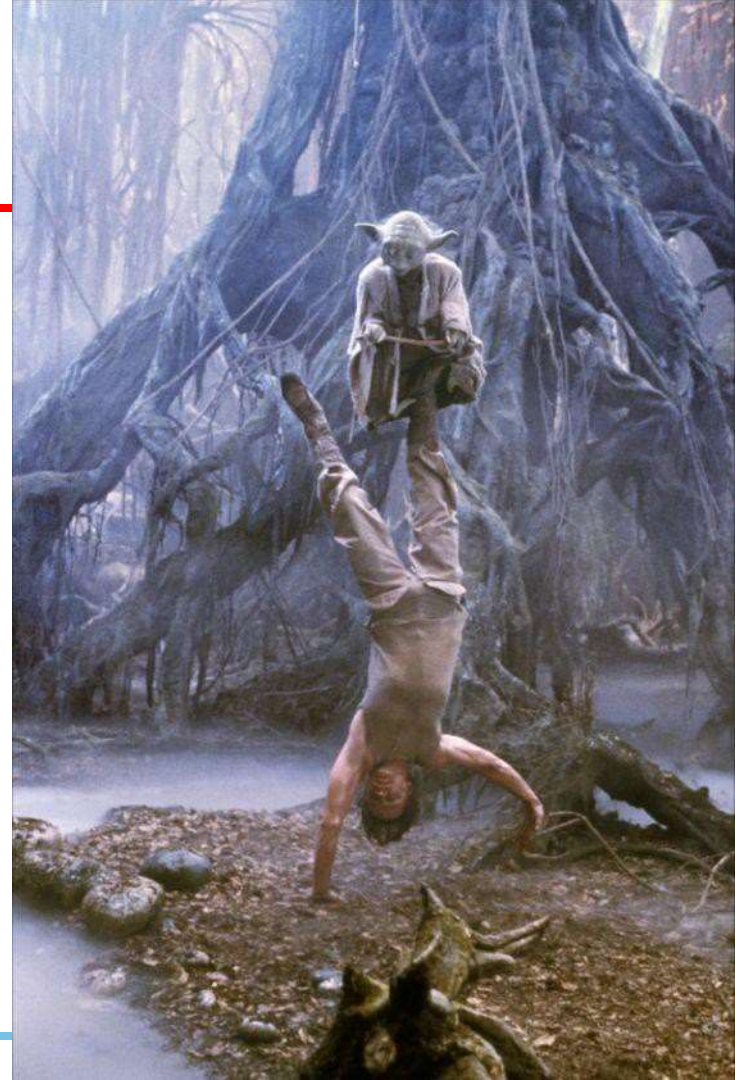
Evaluation Plan

Name	Type	Weight
2 Quiz, total of 2 scores	Online	10%
Assignment-I	Take Home	15%
Assignment-II	Take Home	15%
Mid-Semester Test	Closed Book	30%
Comprehensive Exam	Open Book	30%

Please note there will be no change in submission dates for quiz and assignment

Pedagogy

- Slides will be used to teach during the regular lectures
 - Lab demo and/or problem solving during the webinars
 - Students are responsible for studying and keeping up with the course material outside of class time.
 - Reading certain book chapters, papers or blogs, or
 - Watching some video lectures.
-



Lab Plan

Lab No.	Lab Objective
1	Foundations of Parallel and Distributed Computing
2	Data Parallelism vs. Model Parallelism
3	Algorithm Scaling Examples
4	Distributed Deep Learning Algorithms
5	Federated Learning

- *Labs not graded*
- *Webinars may be conducted for lab sessions*
- *Labs will be conducted in Python*



Course Topics

Course Topics

1. Foundations of Parallel and Distributed Computing
2. Data Parallelism vs. Model Parallelism
3. Algorithm Scaling Examples
4. Distributed Deep Learning Algorithms
5. Federated Learning

Distributed Machine Learning



- Distributed machine learning is an advanced approach to machine learning (ML) that involves spreading computational tasks across multiple processing units, typically across a network of computers.
- This method is particularly useful for handling large datasets, complex model architectures, and intensive computation tasks that are too challenging for a single computer to process efficiently.
- Here's a detailed explanation of distributed machine learning, touching on key aspects such as its need, architecture, techniques, applications¹³ and inherent challenges.

Why Distributed Machine Learning?



1. The need for distributed machine learning arises from several modern-day requirements:

Large Datasets: With the explosion of data in fields like social media, healthcare, and finance, datasets have grown to a size (often terabytes or petabytes) that cannot be processed effectively on a single machine.

Complex Models: Advanced ML models, especially deep learning networks, have millions of parameters and require substantial computational resources that exceed the capacity of individual machines.

14

Speed and Efficiency: Distributed systems can significantly reduce the time needed for training and inference by parallelizing tasks and utilizing many machines simultaneously.

Why Distributed Machine Learning?



1. Large Datasets

The vast amount of data generated daily from various sources, including social media platforms, medical records, financial transactions, and IoT devices, has led to datasets that are enormous in size and complexity. Here's why handling such large datasets necessitates distributed machine learning:

- **Storage Constraints:** Single machines, even with high-capacity storage, can struggle to store terabytes or petabytes of data, making data processing and retrieval inefficient and slow.
- **Processing Power:** Large datasets require substantial processing power for tasks like cleaning, normalization, and analysis, which can overwhelm the capabilities of a single processor or a local cluster of processors.
- **Parallel Processing:** Distributing these large datasets across multiple machines allows for parallel processing, which can significantly speed up data analysis tasks by handling different portions of the data simultaneously.

Generate Images



2. Complex Models

Advanced machine learning models, such as deep neural networks, are known for their deep layers and millions of parameters. These models bring about specific needs that distributed machine learning is well-suited to meet:

- **Memory Requirements:** The size of these models often exceeds the RAM and cache memory available on single machines, necessitating a distributed approach where different parts of the model can reside and compute on different machines.
- **Computational Demands:** Training complex models requires a lot of computational power, often necessitating the parallel use of GPUs or TPUs across multiple machines to handle the computations within a reasonable time frame.
- **Gradient Descent and Optimization:** Distributed systems can perform gradient descent (for optimizing model parameters) more effectively. They parallelize the computation of gradients and updates across many machines, reducing the time taken for each training iteration.

3. Speed and Efficiency

Speed and efficiency are critical in machine learning, especially in commercial applications where time and resource utilization translate directly to costs and revenues.

- **Reduced Training Time:** By distributing tasks across multiple processors, machines, or nodes in a cluster, each part of the task can be executed concurrently, reducing overall model training time.
- **Scalability:** Distributed systems are inherently scalable; additional nodes can be added to the system to improve performance and manage larger loads, a flexibility that single-machine setups cannot offer.
- **Resource Utilization:** Effective distribution of tasks ensures that no single machine is overwhelmed or becomes a bottleneck, which optimizes resource usage across the network.



Large-scale machine learning

The scale of machine learning applications has become unprecedentedly large. Users are demanding faster responses to meet real-life requirements, and machine learning pipelines and model architectures are getting more complex.

Data Partitioning

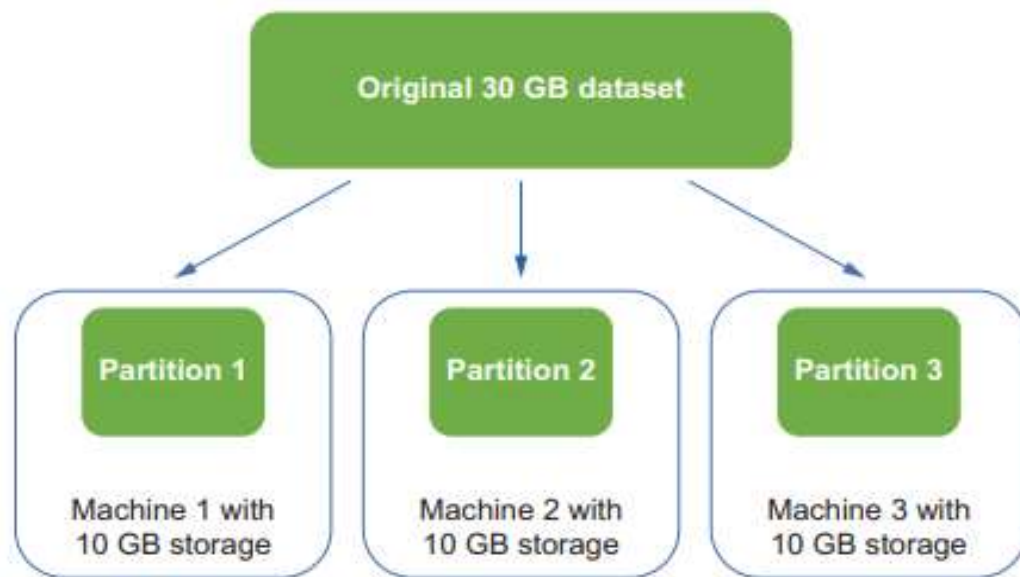


Figure 1.1 An example of dividing a large dataset into three partitions on three separate machines that have sufficient disk storage

Generate Images



Theoretical Explanation

Data Partitioning: The diagram depicts a scenario where an original dataset of 30 GB is divided into three equal parts, each being 10 GB. This partitioning strategy is crucial for distributed data processing, allowing for the workload to be shared across multiple machines rather than overloading a single system.

Distributed Storage: Each partition of the dataset is stored on a separate machine, each equipped with sufficient storage capacity (10 GB in this case) to handle its respective partition. This distribution not only balances the storage requirements but also prepares the system for parallel computational tasks, which can significantly speed up data processing operations like training machine learning models.

Generate Images



Scalability and Flexibility: This setup exemplifies how distributed systems can scale horizontally by adding more machines to handle larger datasets or increase processing speed. It also offers flexibility, as each machine can operate independently or in synchrony with others, depending on the computational requirements.

Generate Images



Mathematical Explanation

Partitioning Formula: Given an original dataset D with size S_D , if the dataset is divided into n partitions where each partition has size S_p , the relationship can be described by the equation:

$$S_D = n \times S_p$$

In the context of the provided diagram:

$$30 \text{ GB} = 3 \times 10 \text{ GB}$$

Load Balancing: The ideal goal of such a partitioning strategy is to ensure that each machine receives an equal amount of data to process, which is mathematically represented as:

$$|D_i| = \frac{|D|}{n}$$

Where $|D_i|$ represents the size of each partition and n is the total number of partitions. This ensures that the computational load is evenly distributed, maximizing the utilization of each machine's resources.

Generate Images



Parallel Processing Efficiency: If each machine i processes its partition D_i independently, the total time T taken to process the entire dataset in parallel can be theoretically reduced, depending on the efficiency of the parallelization strategy and the individual processing capabilities of the machines:

$$T = \max(T_1, T_2, T_3)$$

where T_i is the time taken by machine i to process partition D_i . Ideally, T is significantly less than processing the entire dataset on a single machine.

This approach not only helps in managing large datasets more effectively but also in enhancing the performance of machine learning tasks by leveraging the collective power of multiple computing units. Such distributed processing is fundamental in handling the vast amounts of data typically seen in big data and machine learning applications today.

Generate Images

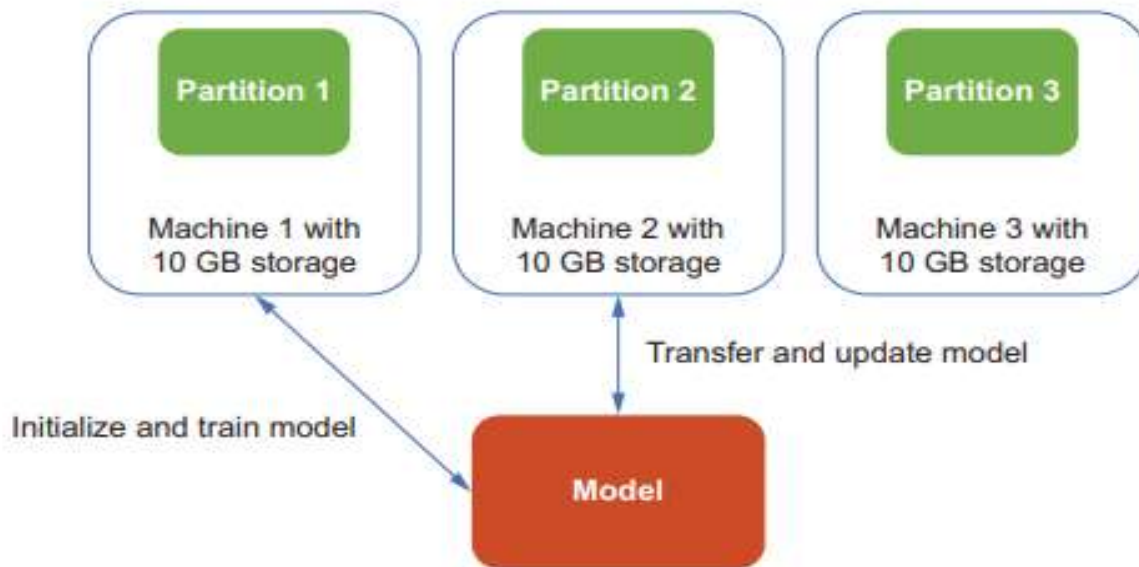


Figure 1.2 An example of training the model sequentially on each data partition

Generate Images



Theoretical Explanation

Sequential Training Process: The diagram shows a machine learning model being trained sequentially across three different machines, each holding a distinct partition of the original dataset. The process starts with Machine 1, which initializes and trains the model using its data partition (Partition 1). Once training on this partition is complete, the model, along with its updated parameters, is transferred to Machine 2, which continues training with its own data partition (Partition 2). This sequential training is repeated with Machine 3 using Partition 3.

Generate Images



Model Updating: After each machine completes training on its data partition, the model parameters are updated based on the learning from that specific subset of data. This updating process involves transferring the state of the model from one machine to the next, ensuring that learning accumulates across the dataset's entirety, despite the data being physically separated.

Efficiency and Practicality: This method can be particularly effective when dealing with large datasets that are too big to fit on a single machine. It allows each machine to fully utilize its computational and memory resources on a manageable portion of the data. However, the effectiveness of the sequential training can depend heavily on the nature of the data and the model. There might be a delay due to the time taken to transfer the model between machines.

Generate Images



Mathematical Explanation

Model Training Dynamics: Let's denote the model parameters after training on partition i as θ_i . Assuming the model parameters are initially θ_0 , the update after training on each machine can be described by the following:

$$\theta_{i+1} = F(\theta_i, D_i)$$

where F represents the training function (such as an epoch of gradient descent), θ_i are the model parameters after training on the i -th partition, and D_i is the data on the i -th machine.

Sequential Training Formula: The sequential update process can be expressed as:

$$\theta_1 = F(\theta_0, D_1)$$

$$\theta_2 = F(\theta_1, D_2)$$

$$\theta_3 = F(\theta_2, D_3)$$

where θ_3 would be the final model parameters after having sequentially processed all data partitions.

Generate Images



Convergence and Accuracy: In theoretical terms, the convergence and accuracy of this sequential training method can depend on the homogeneity of the data in each partition and the adaptability of the model to incremental learning. If data partitions differ significantly or if the model is sensitive to the order of data it encounters, sequential training could introduce biases or convergence issues.

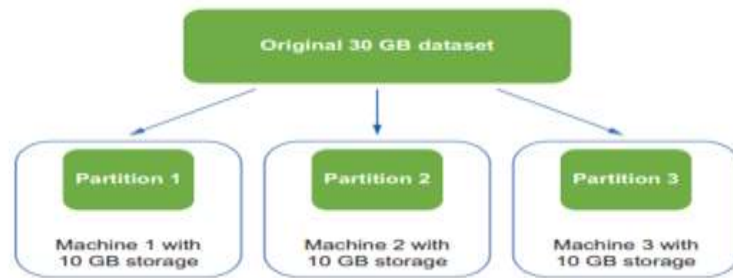
Generate Images



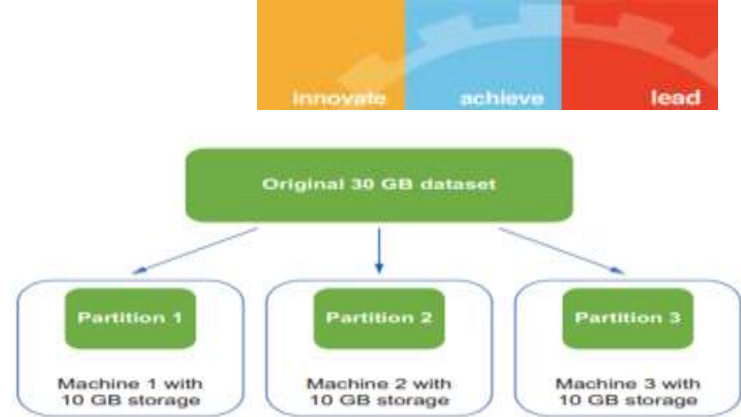
1. Data Partitioning (First Diagram)

The first diagram illustrates a simple data partitioning strategy:

- **Data Division:** The original 30 GB dataset is divided evenly into three 10 GB partitions.
- **Distributed Storage:** Each partition is stored on a separate machine, each equipped with 10 GB of storage.
- **Parallel Processing Capability:** This setup suggests potential for parallel processing, where each machine could independently and simultaneously process its data partition.



Generate Images



1. **Single Model, Distributed Data:** All machines contribute to training or running a single machine learning model. Each machine processes its portion of the data (partition), and the results are aggregated to update or utilize a central model. This approach is typical in scenarios where maintaining a unified model is crucial for consistency and accuracy, such as in large-scale machine learning tasks (e.g., training on massive datasets that cannot fit into the memory of a single machine).
2. **Separate Models per Machine:** Each machine trains or runs its own independent model on the data partition it holds. This approach might be used in scenarios where models are specialized or tailored to specific subsets of data, or when model independence is necessary due to data locality or privacy concerns. After training, these models can either be used independently or their predictions/results might be combined in a certain way (e.g., ensemble methods) to improve overall performance or accuracy.

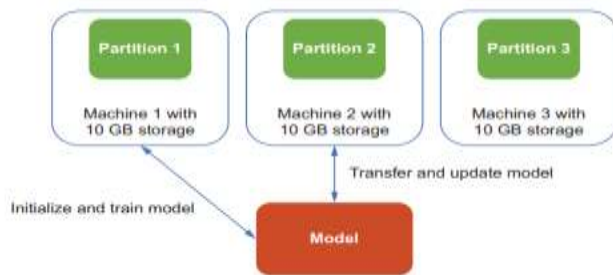
Generate Images



2. Sequential Training (Second Diagram)

The second diagram depicts a sequential approach to training a machine learning model using the partitioned data:

- **Sequential Execution:** The model training starts on Machine 1 with Partition 1. Once training is complete, the model parameters are transferred to Machine 2, where training continues with Partition 2, and so forth to Machine 3.
- **Model Transfer:** The trained model parameters are transferred from one machine to another, indicating a dependency chain in the training process.
- **Cumulative Learning:** Learning accumulates sequentially, with each machine building upon the model parameters refined by the previous machine.



Generate Images



Comparison

Parallel vs. Sequential Processing:

- **Parallelism** (First Diagram): Each machine operates independently, potentially allowing for parallel computations that can significantly reduce the total time required for tasks such as model inference or preliminary data analysis.
- **Sequentialism** (Second Diagram): The training process is inherently serial; each step must wait for the previous one to complete, which can introduce delays especially due to model transfer times between machines.

Generate Images



Resource Utilization:

- **First Diagram:** Implies equal workload distribution but doesn't specify computational tasks, leaving open the possibility for either synchronized tasks or completely independent operations.
- **Second Diagram:** Each machine's computational resources are utilized sequentially for model training, which might lead to underutilization of other machines during each phase of the model training.

Generate Images



Model Consistency and Convergence:

- **First Diagram:** Does not inherently involve model training; thus, issues related to model consistency or convergence are not directly addressed. It focuses more on data management.
- **Second Diagram:** Raises potential concerns about model consistency and convergence, as the model is updated sequentially across different datasets. The final model's performance depends heavily on the order of the datasets and their individual characteristics.

Generate Images



Use Cases:

- **First Diagram** is suited for scenarios where simple tasks need to be performed on different data partitions independently, such as initial data cleaning or individual statistical analyses.
- **Second Diagram** is more applicable to training complex models where the dataset is too large to be handled by a single machine or when model training requires large amounts of memory and processing power sequentially distributed.

Efficiency and Speed:

- **First Diagram** could be more efficient if the tasks are suitable for true parallel processing.
- **Second Diagram** may be less efficient due to the sequential nature of the training and the overhead associated with transferring model parameters between machines.

Batch Processing

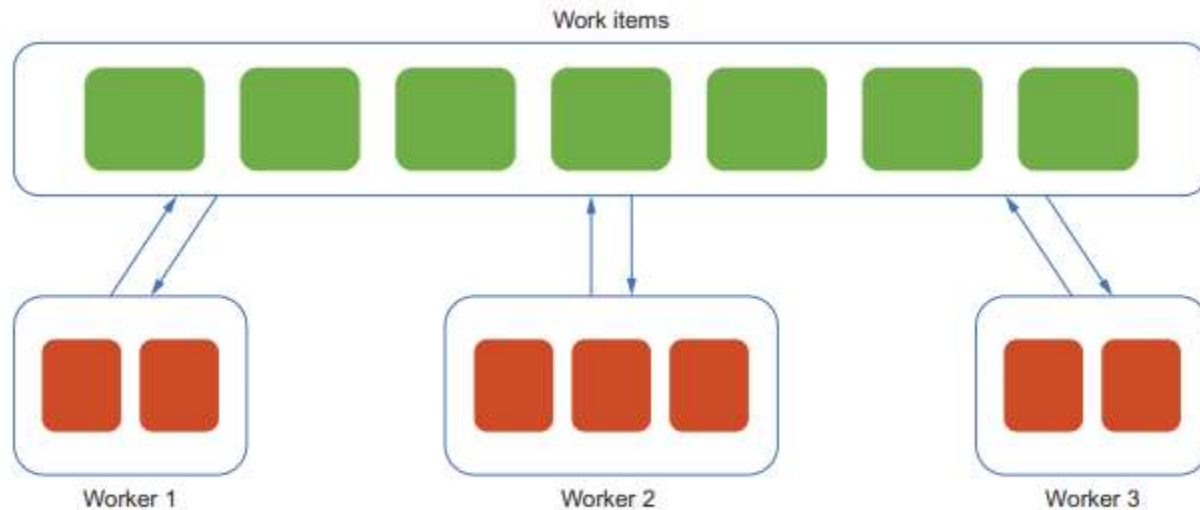


Figure 1.4 An example of a batch processing system using the work-queue pattern to modify images to grayscale

Batch Processing



Analysis

- **Work Items:** Represented by the green boxes, these are the tasks to be processed (images to be converted to grayscale).
- **Workers:** Represented by Worker 1, Worker 2, and Worker 3, these are the processes or threads that handle the tasks.

The work-queue pattern helps distribute tasks among multiple workers, allowing for parallel processing. Each worker picks up tasks from the queue, processes them, and then returns for more work until all tasks are completed.

Multiprocessing Theory



Multiprocessing is a method that allows the execution of multiple processes simultaneously, leveraging multiple CPU cores. This can significantly speed up computations, especially for CPU-bound tasks, by dividing the work among several processors.

Key Concepts

Process:

A process is an independent unit of execution that has its own memory space. Each process runs separately and does not share its memory with other processes.

Batch Processing



Thread:

A thread is the smallest unit of execution within a process. Multiple threads within the same process share the same memory space.

Concurrency vs. Parallelism:

Concurrency involves multiple tasks making progress, often by switching between them (context switching).

Parallelism involves multiple tasks running at the same time on different processors or cores.

Batch Processing

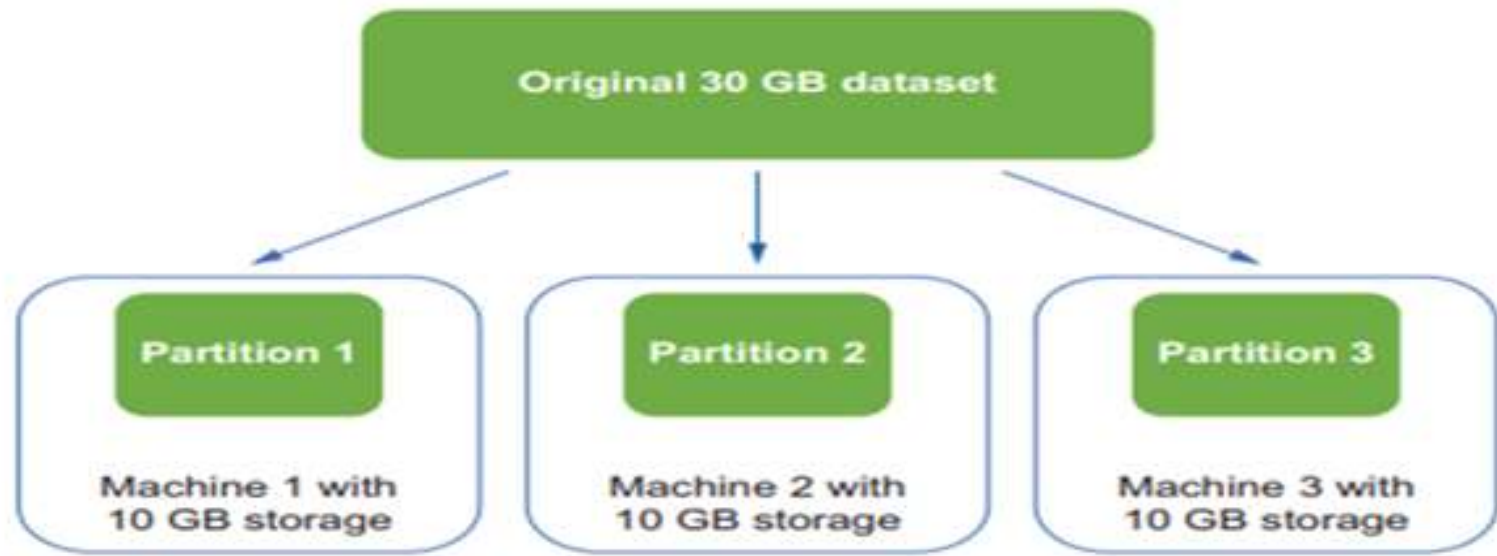


The work-queue pattern helps distribute tasks among multiple workers, allowing for parallel processing. Each worker picks up tasks from the queue, processes them, and then returns for more work until all tasks are completed.

Linear Regression

Dr. V. Chandra Sekhar

Parallelism



Paralellism

In distributed computing systems like the one depicted in your diagram, there are generally two common approaches regarding how machine learning models are handled across different machines:

1. **Single Model, Distributed Data:** All machines contribute to training or running a single machine learning model. Each machine processes its portion of the data (partition), and the results are aggregated to update or utilize a central model. This approach is typical in scenarios where maintaining a unified model is crucial for consistency and accuracy, such as in large-scale machine learning tasks (e.g., training on massive datasets that cannot fit into the memory of a single machine).
2. **Separate Models per Machine:** Each machine trains or runs its own independent model on the data partition it holds. This approach might be used in scenarios where models are specialized or tailored to specific subsets of data, or when model independence is necessary due to data locality or privacy concerns. After training, these models can either be used independently or their predictions/results might be combined in a certain way (e.g., ensemble methods) to improve overall performance or accuracy.

Paralellism

Partitioning a machine learning model across multiple machines, a technique often utilized in distributed computing environments, involves several strategies depending on the model's architecture and the specific computational tasks.

1. Data Parallelism

Data parallelism is one of the most common approaches to distributed machine learning. In this strategy, the complete model is replicated across multiple machines or processors. Each machine works with a different subset (partition) of the data but uses the same model. As each machine processes its data, it computes gradients or updates that are then aggregated (typically on a parameter server or through all-to-all communication) to update the global model. This method is especially popular in training deep neural networks where the same model parameters are updated simultaneously using different batches of data.

Paralellism

Partitioning a machine learning model across multiple machines, a technique often utilized in distributed computing environments, involves several strategies depending on the model's architecture and the specific computational tasks.

1. Data Parallelism

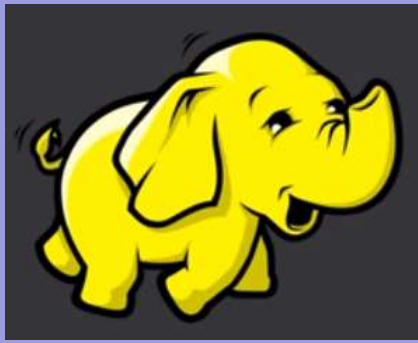
Data parallelism is one of the most common approaches to distributed machine learning. In this strategy, the complete model is replicated across multiple machines or processors. Each machine works with a different subset (partition) of the data but uses the same model. As each machine processes its data, it computes gradients or updates that are then aggregated (typically on a parameter server or through all-to-all communication) to update the global model. This method is especially popular in training deep neural networks where the same model parameters are updated simultaneously using different batches of data.

Paralellism

Model slicing, or model parallelism, involves dividing the model itself across multiple processing units or machines. This method is particularly useful when a model is too large to fit into the memory of a single machine or when different components of a model can be processed in parallel to speed up computation.

1. Layer-wise Partitioning

In deep learning architectures, such as neural networks, each layer (or a group of layers) can be allocated to different machines. For example, the first few layers of a Convolutional Neural Network (CNN) might reside on one machine, and subsequent layers on others. Each machine processes its layers' forward and backward passes, and the outputs are passed between machines as required by the network architecture.



PySpark

Dr.Chandra Sekhar Vorugunti

Introduction to Apache Spark



Agenda

- ✓ Before Spark
- ✓ Need for Spark
- ✓ What is Apache Spark ?
- ✓ Goals
- ✓ Why Spark ?
- ✓ RDD & its Operations
- ✓ Features Of Spark



Before Spark



Batch
Processing

Stream
Processing

Interactive
Processing

Graph
Processing

Machine
Learning

Features of Spark



What is Apache Spark?

- Apache Spark is an open-source distributed **cluster-computing** framework introduced by Apache Software Foundation.
- It is a general engine for **big data analysis**, **processing** and **computation**. It is built for **high speed**, ease of use, offers simplicity, stream analysis and run virtually anywhere.
- It can analyze data in **real-time**. It provides fast computation over the big data.
- The fast computation means that it's faster than previous approaches to work with Big Data such as MapReduce.
- The main feature of Apache Spark is its **in-memory cluster computing** that enhances the processing speed of an application.
- It can be used for multiple things like running distributed SQL, creating data pipelines, ingesting data into a database, running Machine Learning algorithms, working with graphs or data streams, and many more.

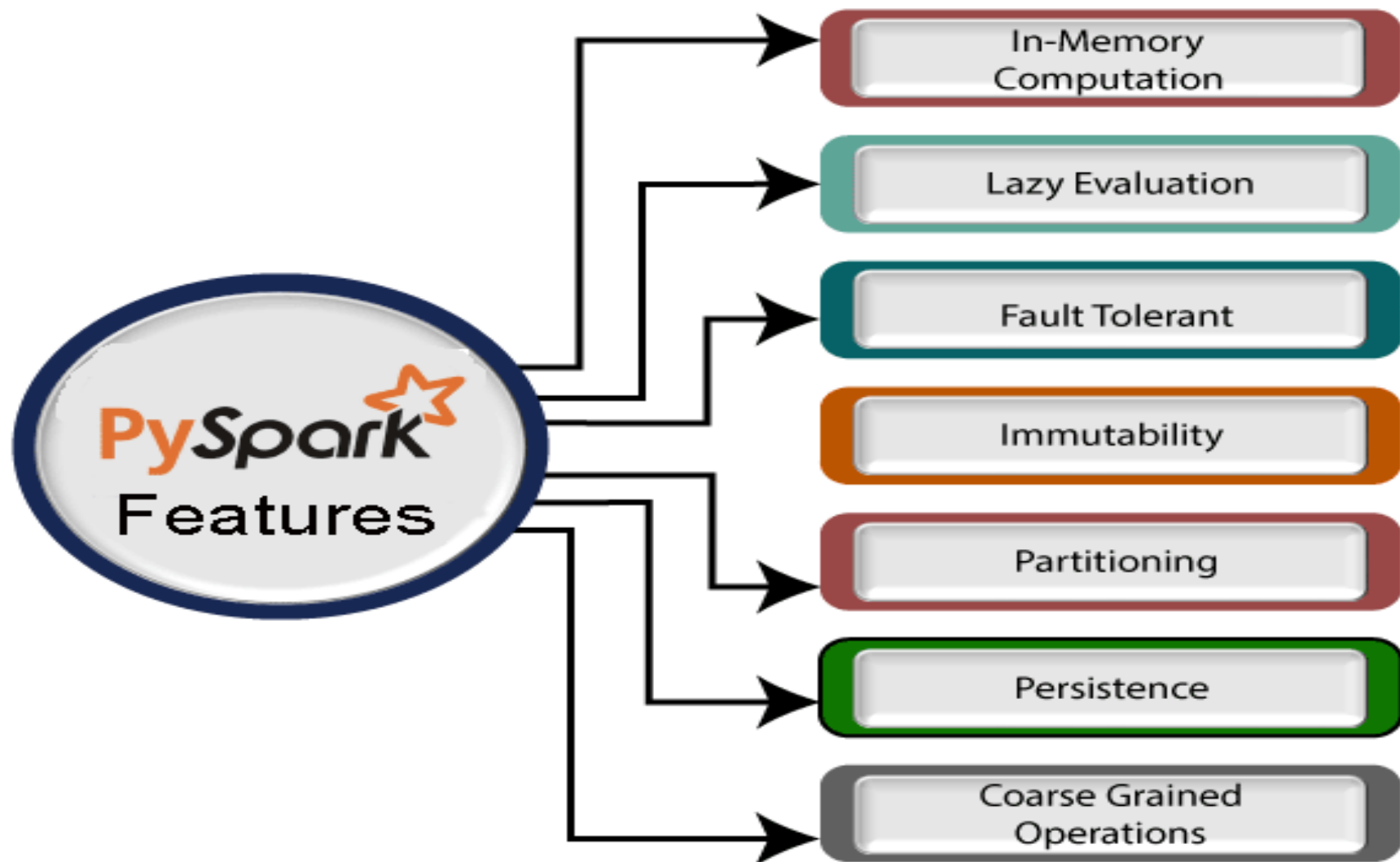
What is PySpark

Spark is an open-source, cluster computing system which is used for big data solution. It is lightning fast technology that is designed for fast computation.

PySpark is the Python API to use Spark.

PySpark is a Python API to support Python with Apache Spark. PySpark provides Py4j library, with the help of this library, Python can be easily integrated with Apache Spark.

PySpark plays an essential role when it needs to work with a vast dataset or analyze them. This feature of PySpark makes it a very demanding tool among data engineers.



Real-time Computation:

PySpark provides real-time computation on a large amount of data because it focuses on in-memory processing. It shows the low latency.

Support Multiple Language:

PySpark framework is suited with various programming languages like Scala, Java, Python, and R. Its compatibility makes it the preferable frameworks for processing huge datasets.

Caching and disk constancy:

PySpark framework provides powerful caching and good disk constancy.

Swift Processing:

PySpark allows us to achieve a high data processing speed, which is about 100 times faster in memory and 10 times faster on the disk.

Works well with RDD:

Python programming language is dynamically typed, which helps when working with RDD. We will learn more about RDD using Python in the further tutorial.

Need For Spark

- **Need for a powerful engine that can process the data in Real-Time (streaming) as well as in Batch mode**
- **Need for a powerful engine that can respond in Sub-second and perform In-memory analytics**
- **Need for a powerful engine that can handle diverse workloads:**
 - Batch
 - Streaming
 - Interactive
 - Graph
 - Machine Learning

What is Apache Spark?

Apache Spark is a powerful open source engine which can handle:

- Batch processing
- Real-time (stream)
- Interactive
- Graph
- Machine Learning (Iterative)
- In-memory

Introduction to Apache Spark

- **Lightening fast cluster computing tool**
- **General purpose distributed system**
- **Provides APIs in Scala, Java, Python, and R**

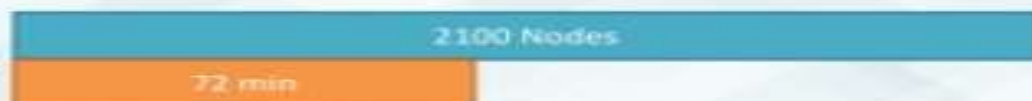


History

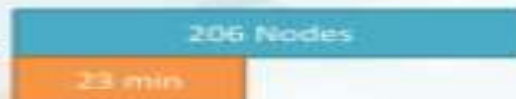


Sort Record

Hadoop-MapReduce



Spark



	Hadoop MapReduce	Spark
➡ Data Size	102.5 TB	100 TB
➡ Time Taken	72 min	23 min
➡ No of nodes	2100	206
➡ No of cores	50400 physical	6592 virtualized
➡ Cluster disk throughput	3150 GBPS	618 GBPS
➡ Network	Dedicated 10 Gbps	Virtualized 10 Gbps

Goals

- *Easy* to combine *batch*, *streaming*, and *interactive* computations



Why Spark ?

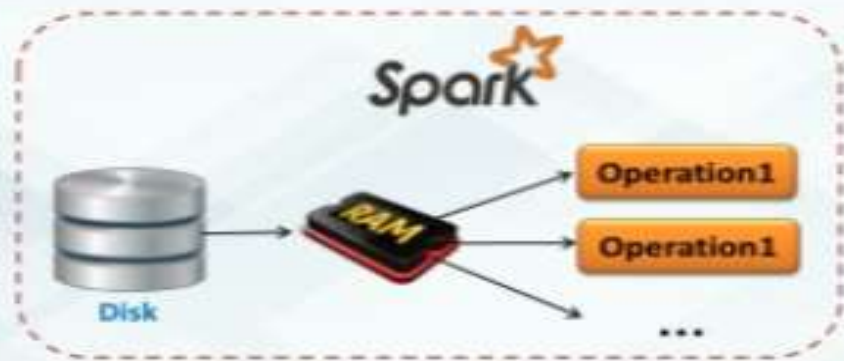
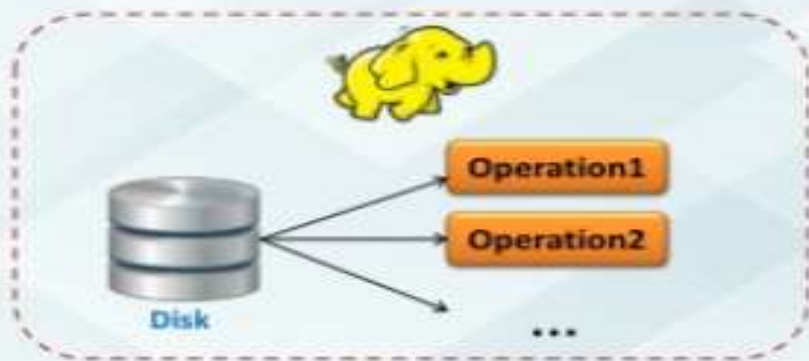
- 100x faster than Hadoop.

Spark



Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.



Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.



Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.
- Language support like Scala, Java, Python and R.

 **Scala**

 **Java**

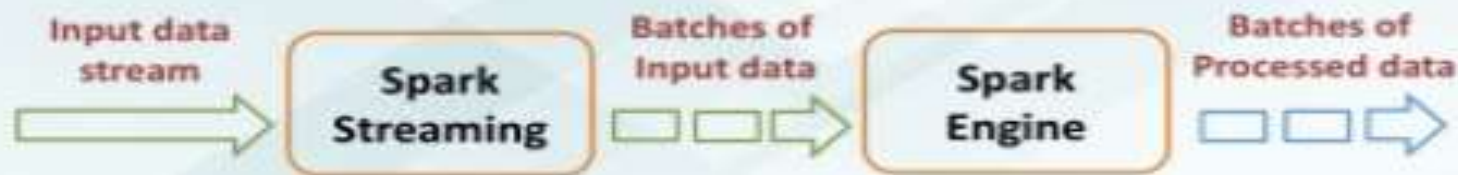


 **Spark**

 **python**

Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.
- Language support like Scala, Java, Python and R.
- Support Real time and Batch Processing.



Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.
- Language support like Scala, Java, Python and R.
- Support Real time and Batch Processing.
- Lazy Operations – optimize the job before execution.

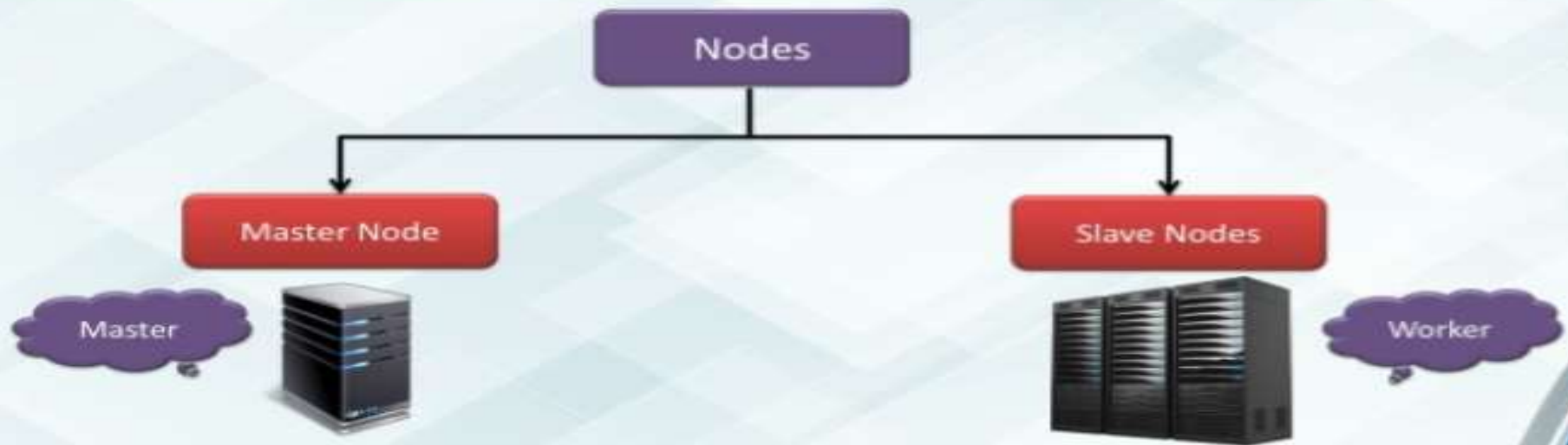


Why Spark ?

- 100x faster than Hadoop.
- In-memory computation.
- Language support like Scala, Java, Python and R.
- Support Real time and Batch Processing.
- Lazy Operations – optimize the job before execution.
- Support for multiple transformations and actions.



Spark Nodes



Resilient Distributed Dataset (RDD)

- RDD is a simple and immutable collection of objects.



RDD

Basic Spark Architecture



Resilient Distributed Dataset (RDD)

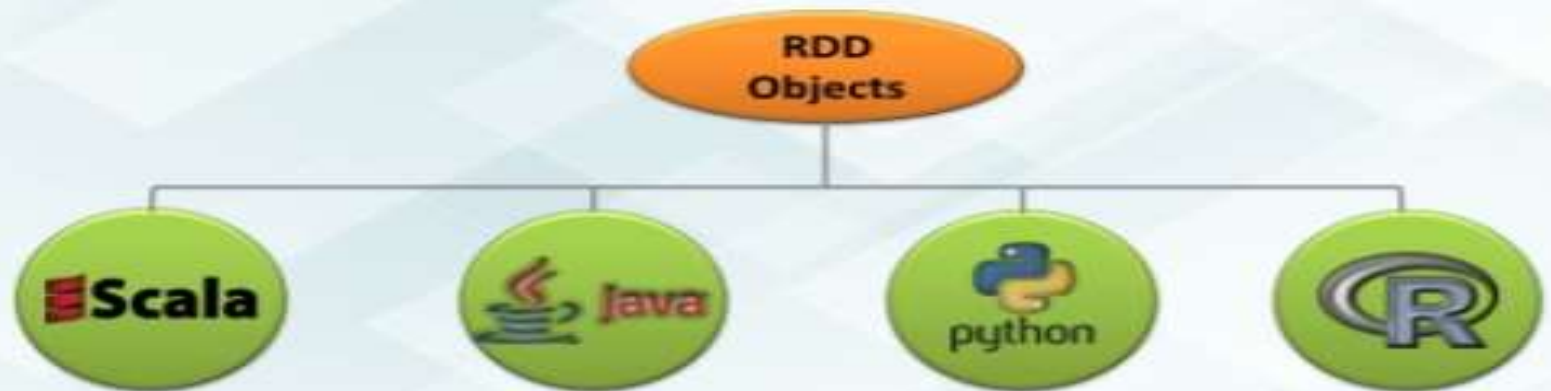
- RDD is a simple and immutable collection of objects.



RDD

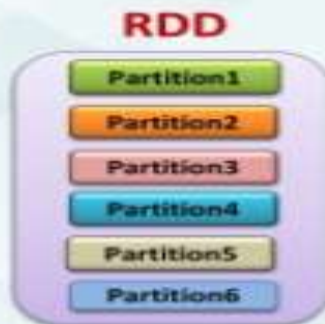
Resilient Distributed Dataset (RDD)

- RDD is a simple and immutable collection of objects.
- RDD can contain any type of (scala, java, python and R) objects.

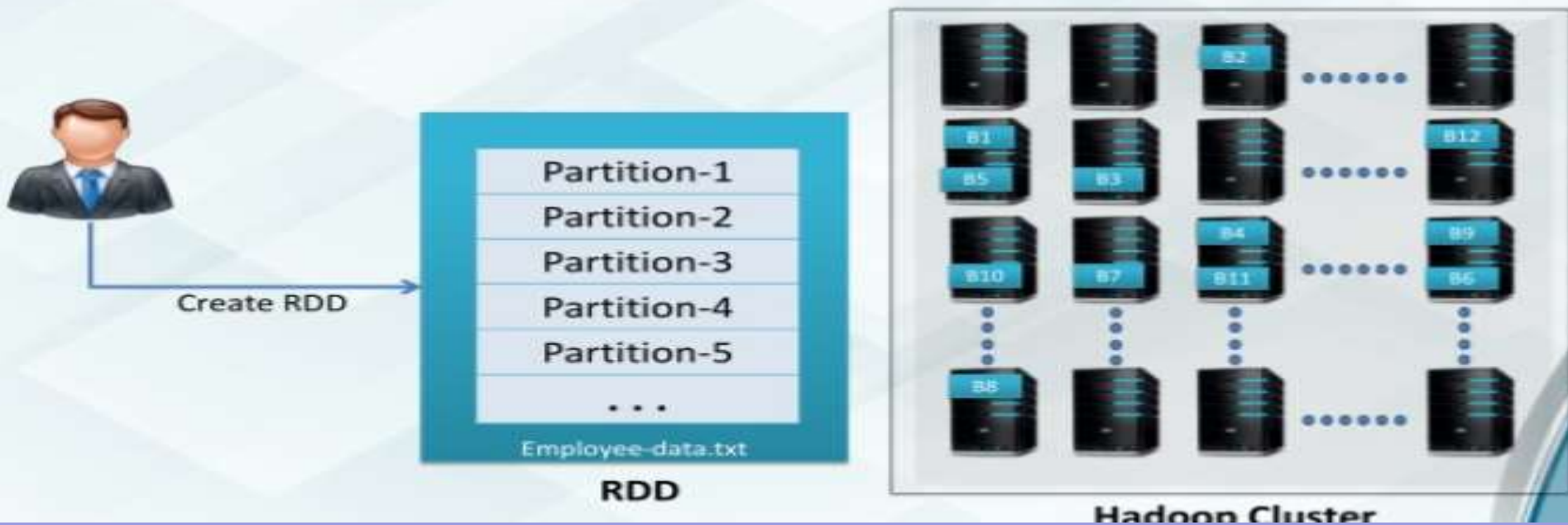


Resilient Distributed Dataset (RDD)

- RDD is a simple and immutable collection of objects.
- RDD can contain any type of (scala, java, python and R) objects.
- Each RDD is split-up into different partitions, which may be computed on different nodes of clusters.



Resilient Distributed Dataset (RDD)



RDD Operations



RDD Operations – Transformation

Transformation:

- Set of operations that define how RDD should be transformed
- Creates a new RDD from the existing one to process the data
- Lazy evaluation: Computation doesn't start until an action associated
- E.g. Map, FlatMap, Filter, Union, GroupBy, etc.

RDD Operations – Action



Action:

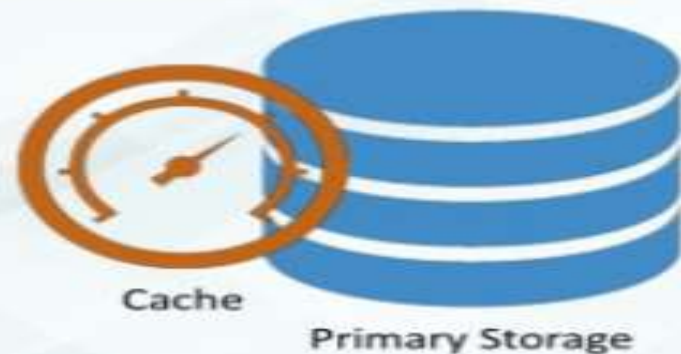
- Triggers job execution.
- Returns the result or write it to the storage.
- E.g. Count, Collect, Reduce, Take, etc.



RDD Operations – Persistence

Persistence:

- Spark allows caching/Persisting entire dataset in memory
- Caches the RDD in the memory for future operations



RDD Operations



Federated Learning

- The standard setting in Machine Learning (ML) considers a **centralized dataset** processed in a **tightly integrated system**
- But in the real world **data is often decentralized across many parties**



Data is born at the edge

Billions of phones & IoT devices constantly generate data

Data enables better products and smarter models



Centralized Machine Learning Setting:

- The image states that the standard setting in Machine Learning considers a centralized dataset.
- In a centralized setting, all data is collected and stored in a **single** location, such as a data center.
- The data is processed within a **tightly** integrated system, meaning all the computational resources, algorithms, and data are **co-located**.
- This centralized approach allows for efficient data management and processing since all the data is **accessible** from **one** place.
- The training of ML models in this scenario is straightforward since the models have **direct** access to all available data.

Decentralized Data in the Real World:

- The second point contrasts this idealized setting with the real world, where data is often decentralized across many parties. In reality, data isn't always stored in one central location.
- Instead, it is distributed across different organizations, departments, devices, and locations.
- Each of these entities may have its own data storage systems, formats, and privacy or security policies, which complicates data access and integration.
- For example, in the healthcare industry, patient data could be stored across different hospitals, clinics, and health devices.
- In a smart city, data from various sectors like transportation, public safety, and utilities are spread across multiple systems.

Challenges of Decentralized Data:

Data Privacy and Security: Decentralized data settings often involve sensitive information (e.g., healthcare records, financial data) that is subject to strict privacy regulations. Sharing data between parties can lead to privacy concerns and the risk of data breaches.

Data Heterogeneity: Different data sources may have different data formats, standards, and quality. Integrating such heterogeneous data for ML can be challenging.

Communication Overhead: In a decentralized system, transferring data to a central location for processing could be impractical or inefficient due to bandwidth limitations or high costs.

Federated Learning: To address the challenges of decentralized data, approaches like Federated Learning have been developed.

Federated Learning allows for training machine learning models across **decentralized** data without the need to move the data to a central server. It leverages **local** data processing at each source and **aggregates** the learned models, **preserving data privacy**.

WHY CAN'T WE JUST CENTRALIZE THE DATA?

1. Sending the data may be **too costly**

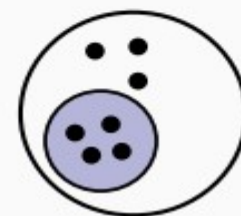
- Self-driving cars are expected to generate several TBs of data a day 
- Some wireless devices have limited bandwidth/power 

2. Data may be considered **too sensitive**

- We see a growing public awareness and regulations on data privacy 
- Keeping control of data can give a competitive advantage in business and research 

HOW ABOUT EACH PARTY LEARNING ON ITS OWN?

1. The local dataset may be **too small**
 - Sub-par predictive performance (e.g., due to overfitting)
 - Non-statistically significant results (e.g., medical studies)
2. The local dataset may be **biased**
 - Not representative of the target distribution



A BROAD DEFINITION OF FEDERATED LEARNING

- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized



Federated Learning (FL): FL is specifically designed to train a machine learning model across multiple **decentralized** data sources while keeping the data localized at each node.

The goal is to **collaboratively** train a **global** model without the need to **centralize** data, which helps to **preserve privacy and reduce the risk of data breaches**.

Use Case: FL is often used in scenarios where data privacy is paramount, such as healthcare, finance, and mobile applications.

For example, in a healthcare setting, patient data remains within each hospital, and only the model updates (gradients) are shared with a central server.

Distributed Computing:

- **Objective:** Distributed Computing is a broader concept that involves dividing a **large** computational **task** into smaller **sub-tasks**, which are then processed **concurrently** across multiple nodes or machines.
- The goal is to speed up computation, improve efficiency, or handle larger tasks that a single machine cannot process alone.
- **Use Case:** Distributed Computing is used in various fields, including large-scale simulations, big data processing (e.g., Hadoop, Spark), and complex mathematical computations.
- It is generally applied to tasks that require high computational power and where the **primary** focus is on **performance** rather than data privacy.

Data Location and Privacy

Federated Learning (FL):

Data Location: Data remains on the local devices or nodes where it was originally generated. Only model updates (e.g., gradients or weights) are sent to a central server for aggregation.

Privacy: FL is designed with privacy in mind. Since the raw data never leaves the local device, FL minimizes the risk of data breaches and complies better with data privacy regulations.

Distributed Computing:

Data Location: In traditional distributed computing, **data** is often **distributed** across multiple nodes. However, the data might still be centralized or shared between nodes as needed. **Data movement** between nodes is common depending on the task requirements.

Privacy: **Privacy** is not inherently a focus of distributed computing. Data can be moved, replicated, or processed on different nodes without specific mechanisms to protect data privacy.

Architecture and Communication

Federated Learning (FL):

Architecture: FL typically follows a client-server architecture where multiple client devices train a local model and send only the updates (e.g., gradients) to a central server.

The server aggregates these updates to create a global model, which is then shared back with the clients.

Communication: Communication in FL is primarily about exchanging model updates. The frequency of communication is often **minimized** to reduce bandwidth usage and **preserve privacy**.

Distributed Computing:

Architecture: Distributed computing can follow various architectures, including **client-server**, **peer-to-peer**, and **hybrid** models. It generally involves **dividing the computational load** and distributing it across multiple nodes.

Communication: Communication in distributed computing is more **extensive** and can involve the exchange of both data and computational tasks. Nodes often need to **synchronize** data and share intermediate results, which can involve significant data transfer.

Computation Focus

Federated Learning (FL):

Computation: In FL, the computation is focused on training machine learning models **locally** on each node using its data. The central server performs **aggregation** of **model** updates but does not handle the raw data.

Model Training: The focus is on collaborative model training with an emphasis on **privacy**.

Distributed Computing:

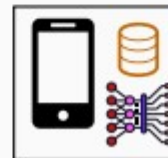
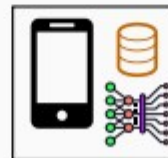
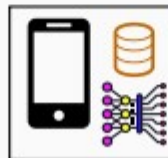
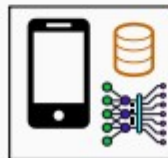
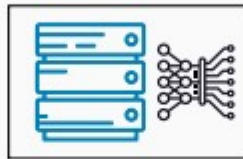
Computation: Distributed computing can involve any computational task, including data processing, simulations, and running large-scale algorithms. The task can be split into independent units that are processed concurrently.

Task Variety: Distributed computing is more general-purpose and can be used for a wide range of computational tasks beyond just model training.

A BROAD DEFINITION OF FEDERATED LEARNING

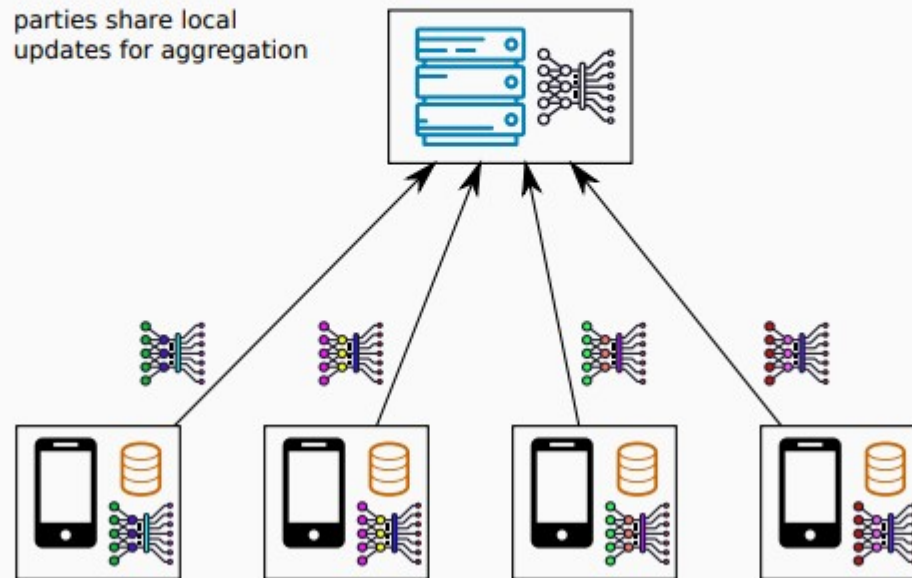
- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized

each party makes an update
using its local dataset



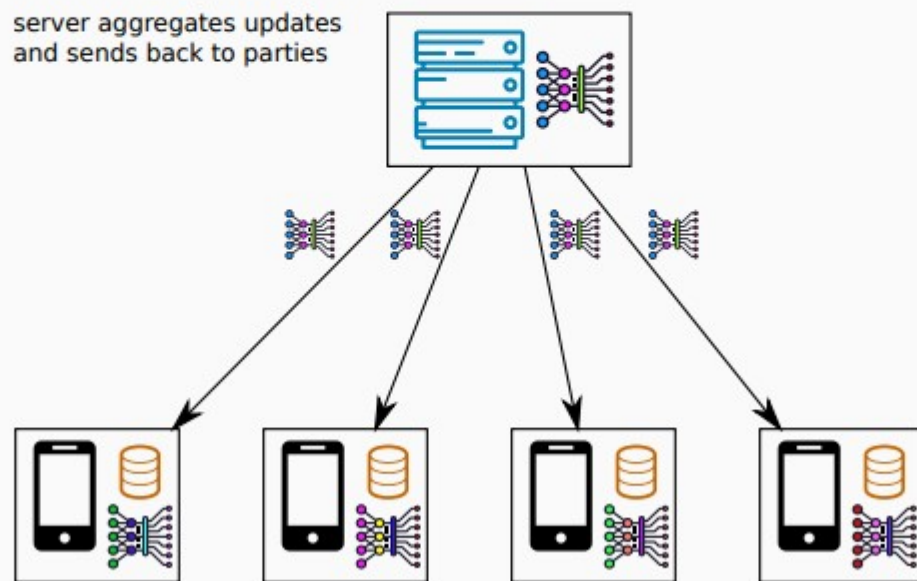
A BROAD DEFINITION OF FEDERATED LEARNING

- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized



A BROAD DEFINITION OF FEDERATED LEARNING

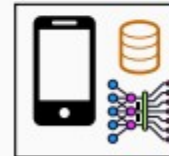
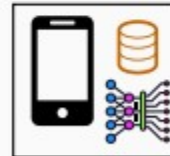
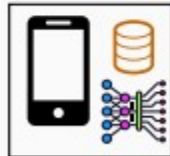
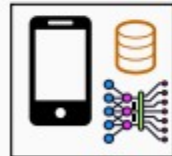
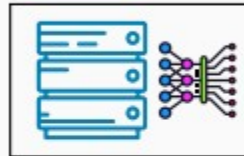
- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized



A BROAD DEFINITION OF FEDERATED LEARNING

- Federated Learning (FL) aims to collaboratively train a ML model while keeping the data decentralized

parties update their copy
of the model and iterate



- We would like the final model to be as good as the centralized solution (ideally), or at least better than what each party can learn on its own

KEY DIFFERENCES WITH DISTRIBUTED LEARNING

Data distribution

- In distributed learning, **data is centrally stored** (e.g., in a data center)
 - The main goal is just to **train faster**
 - We control how data is distributed across workers: usually, it is **distributed uniformly at random** across workers
- In FL, **data is naturally distributed and generated locally**
 - Data is **not** independent and identically distributed (**non-i.i.d.**), and it is **imbalanced**

Additional challenges that arise in FL

- Enforcing **privacy constraints**
- Dealing with the possibly **limited reliability/availability** of participants
- Achieving robustness against **malicious parties**
- ...

FEDERATED LEARNING IS A BOOMING TOPIC

- 2016: the term FL is first coined by Google researchers; 2020: more than 1,000 papers on FL in the first half of the year (compared to just 180 in 2018)¹
- We have already seen some real-world deployments by companies and researchers
- Several open-source libraries are under development: PySyft, TensorFlow Federated, FATE, Flower, Substra...
- FL is highly multidisciplinary: it involves machine learning, numerical optimization, privacy & security, networks, systems, hardware...

This is all a bit hard to keep up with!

¹<https://www.forbes.com/sites/robtoews/2020/10/12/the-next-generation-of-artificial-intelligence/>

Federated learning with encryption

- For federated learning with encryption, we can leverage Homomorphic Encryption (HE) for computation on encrypted data.
- Federated Averaging (FedAvg) to distribute the model training across multiple clients without sharing raw data.

Scenario 1: Training with Encrypted Input and Output

- **Client Side:** Encrypt input data and train the neural network model on encrypted data.
- **Server Side:** The server aggregates the encrypted gradients or updates and sends them back to the client for decryption and further updates.

Explanation:

Encryption: The Paillier encryption scheme is used to encrypt the dataset at the client level.

Federated Averaging: The `federated_training` function simulates federated training by aggregating updates from multiple clients.

Neural Network: A simple neural network is defined with input shape (2,) for two features. In this approach, the training data is encrypted, and the output can also be decrypted using `private_key` after each training round.

Federated learning with encryption

Scenario 2: Federated Model with Decryption Before and After Processing:

In this case, encrypted data is sent to the model, which decrypts it before processing. After the model processes the data, it encrypts the output before sending it back.

Client Side:

Encrypt Input: Data is encrypted using homomorphic encryption and sent to the model.

Decrypt Before Processing: The model decrypts the input data using the private key before processing.

Encrypt Output: The model processes the data, and the result is encrypted before sending back.

Server Side:

Aggregates encrypted updates from multiple clients and sends them back for decryption and further model improvement.

Federated learning with encryption

Scenario 1: Fully Encrypted Input & Output (Homomorphic Encryption):

Data Encryption: In this scenario, data remains encrypted throughout the entire training process. The model operates on encrypted data, and only encrypted results are transmitted back to the client. However, operations on fully encrypted data, especially in neural networks, are extremely limited. Therefore, in practice, this scenario is often simulated by decrypting data just before training and encrypting the outputs again.

Advantages:

Maximum data privacy: The server never sees the raw data in its plain form.

Security: Since computations happen directly on encrypted data, the privacy of the data is preserved.

Drawbacks:

Limited operations: Homomorphic encryption can handle basic operations (e.g., addition and multiplication) but is not fully capable of handling complex neural network operations like backpropagation.

Efficiency: Encryption and decryption are computationally expensive, and real-time training on encrypted data is often infeasible for deep learning.

Federated learning with encryption

Scenario 2: Decryption Before Processing (Decrypt-Train-Encrypt)

Data Encryption: In this scenario, data is encrypted before being transmitted, but once it reaches the model for training, it is decrypted, processed, and re-encrypted after training. The model works on decrypted data during the training process.

Advantages:

Supports complex neural network operations: Since data is decrypted before training, the model can perform backpropagation and gradient updates without issues.

Flexibility: This scenario is more practical for complex models, especially in deep learning, where encryption may hinder proper model training.

Drawbacks:

Less security: There is a temporary exposure of the raw data during training, which poses a privacy risk.

Still computationally intensive: Encrypting and decrypting the data before and after training still incurs overhead, although less than Scenario 1.

Federated learning with encryption

Aspect	Scenario 1 (Fully Encrypted)	Scenario 2 (Decrypt Before Processing)
Data Privacy	Full encryption, data never decrypted	Data is decrypted during processing
Neural Network Operations	Limited due to homomorphic encryption capabilities	Full neural network operations (e.g., backprop)
Security	Stronger security (data never decrypted)	Reduced security (data decrypted during training)
Efficiency	Less efficient due to complex encryption/decryption operations	More efficient, but still computationally costly
Usability	Limited usability for deep learning models	More practical for real-world federated learning

Federated learning with encryption

Scenario1: usecases: Highly Sensitive Data (Healthcare, Finance, Defense):

Medical Imaging & Diagnosis: When training models on sensitive healthcare data (e.g., MRI scans or genetic information), full encryption ensures that no raw data is ever exposed to the central server or even other collaborating entities.

Financial Fraud Detection: Banks and financial institutions can collaboratively train models to detect fraud without revealing individual customer data to each other. This is critical for GDPR compliance and protecting sensitive financial information.

Military or Defense AI Systems: Military organizations might use homomorphically encrypted data to ensure that no raw data (such as intelligence images, strategies, or simulations) is ever exposed during AI training.

Collaborative AI Between Competing Institutions:

For industries where competitors may want to collaborate to improve their models (e.g., training recommendation engines), fully encrypted training ensures that one company's customer data never becomes visible to the others.

Data Aggregation Without Access to Raw Data:

Federated learning with encryption

Scenario2: usecases: Edge Computing & IoT Devices:

Smart Devices (IoT): In edge computing, devices like smartwatches, phones, or IoT sensors can train models on private data collected from users. Data is encrypted during transmission (for security) but decrypted on the device before training. The results are encrypted again before being sent back to a central server.

Smart City Infrastructure: City sensors collecting traffic data, environmental data, or public safety information can encrypt the data during transmission for security but train models locally using decrypted data. This ensures that the devices can handle model training in real-time while securing data transmission.

Healthcare Diagnostics & Wearables:

Wearable Health Devices: Devices like fitness trackers, heart monitors, or glucose monitors collect sensitive user data. They encrypt this data while transmitting to the user's phone or cloud but decrypt it locally for processing and training a model to provide health insights. The results are encrypted again when sending back to central servers for aggregating multiple users' data.

Local Diagnostics Systems: In telemedicine, healthcare systems may locally process sensitive patient data to train models, such as predicting patient risk for specific diseases, before securely sending aggregated updates back to the central hospital server.

Private AI on Mobile Devices: Personalized AI Models:

Federated learning with encryption

Limitations:

Security vs. Performance Trade-off: While this scenario is more efficient than fully encrypted training, it is less secure because the data is decrypted locally before training. The model is not trained on encrypted data, which makes it potentially vulnerable to attacks at the client side.

Local Storage and Processing Requirements: The device or node must have the ability to store and process decrypted data, which may pose risks if the local system is compromised.

Aspect	Scenario 1 (Fully Encrypted)	Scenario 2 (Decrypt Before Processing)
Data Handling	Data is encrypted throughout the process	Data is encrypted during transmission but decrypted before training
Security	Highest level of security (data never decrypted)	Lower security (data is decrypted for training)
Computational Overhead	Very high due to homomorphic encryption	Lower overhead, since data is decrypted before training
Use Cases	Healthcare, Finance, Defense, Highly sensitive data	Edge devices, IoT, personalized AI, federated learning
Privacy Risks	Minimal (data stays encrypted)	Higher risk since data is decrypted before training
Model Training Efficiency	Very slow, limited deep learning operations	More efficient, allows full neural network operations

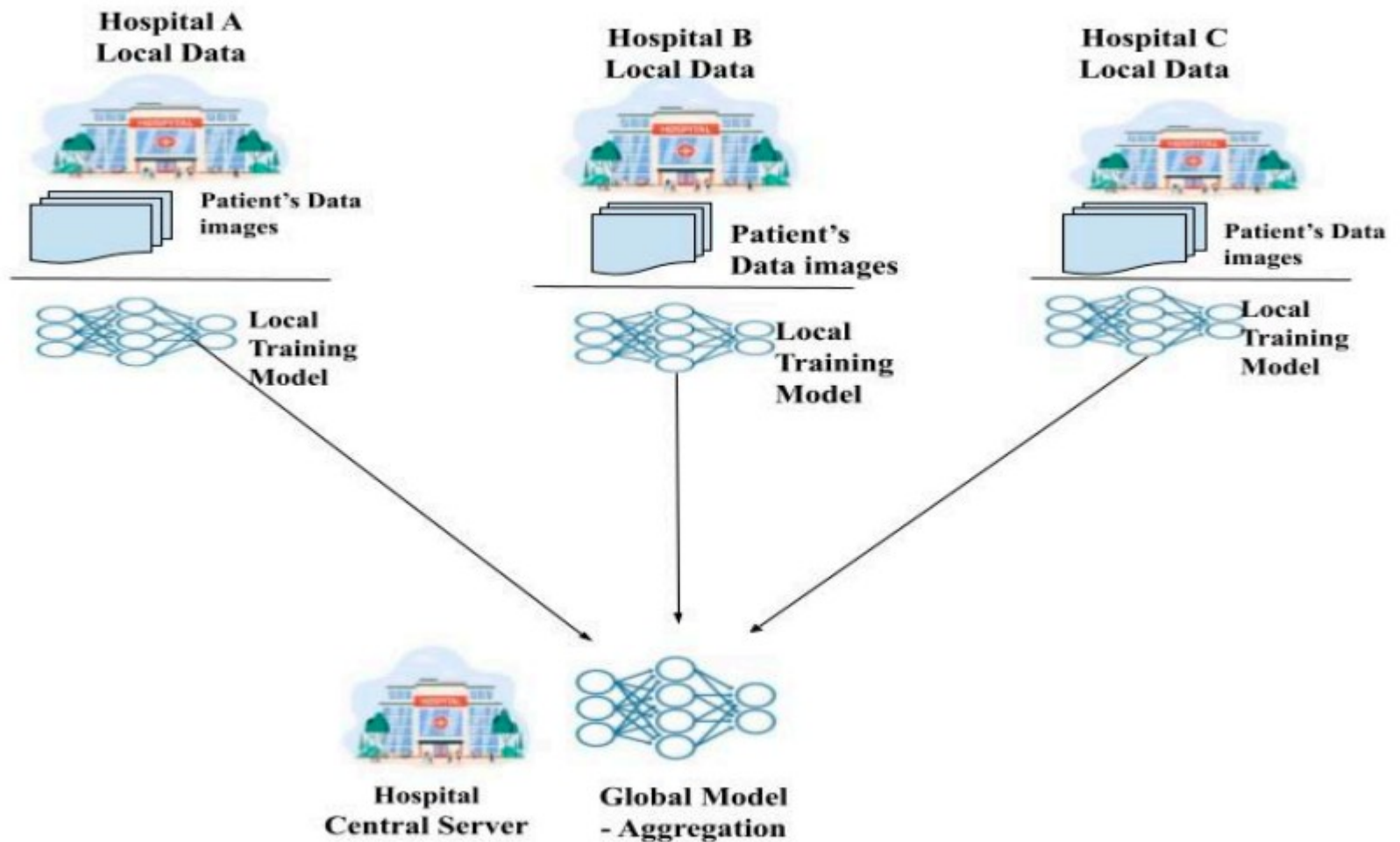


Figure 1. Federated learning framework.

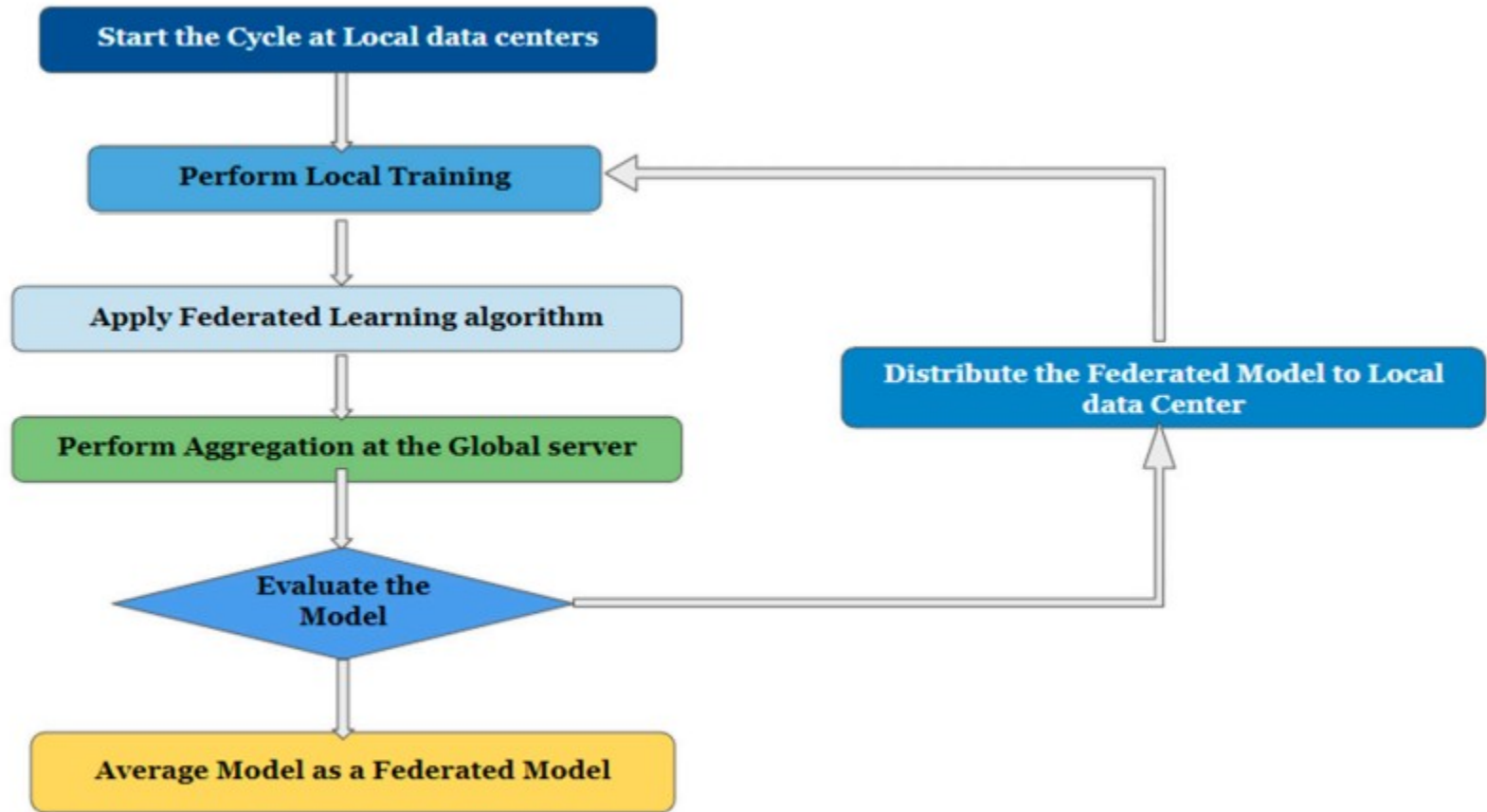


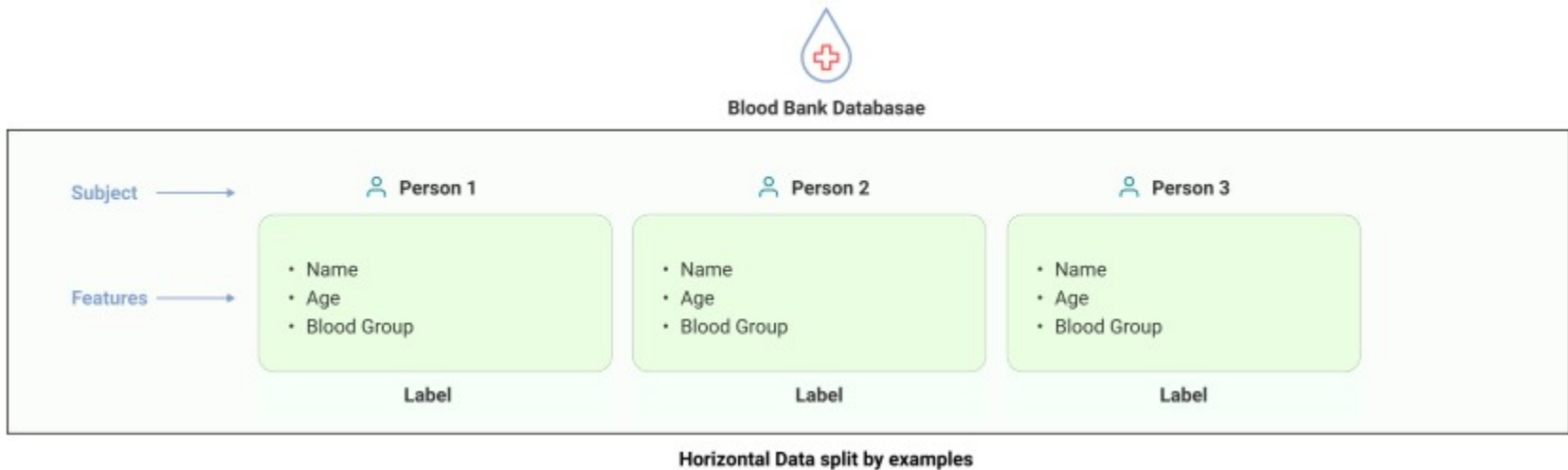
Figure 2. Flowchart of the federated learning framework.

Secure Aggregation Based Privacy Preservation Algorithms for Federated Learning

- Federated Averaging (FedAvg)
- Federated Match Averaging (FedMA)
- Federated Personalization (FedPer)
- FedNAS Algorithm
- FedGKT Algorithm
- Secured Weighted Aggregation
- Secure Federated matrix factorization
- Secret Sharing
- Homomorphic Encryption

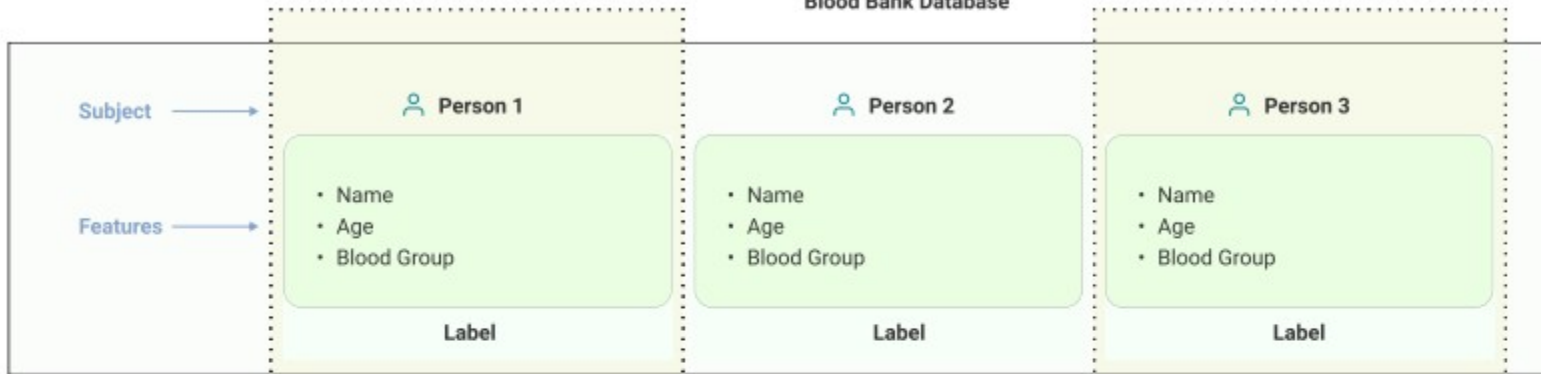
Data Partitioning.

- Horizontal Federated Learning
- Vertical Federated Learning
- Hybrid/Transfer Federated Learning





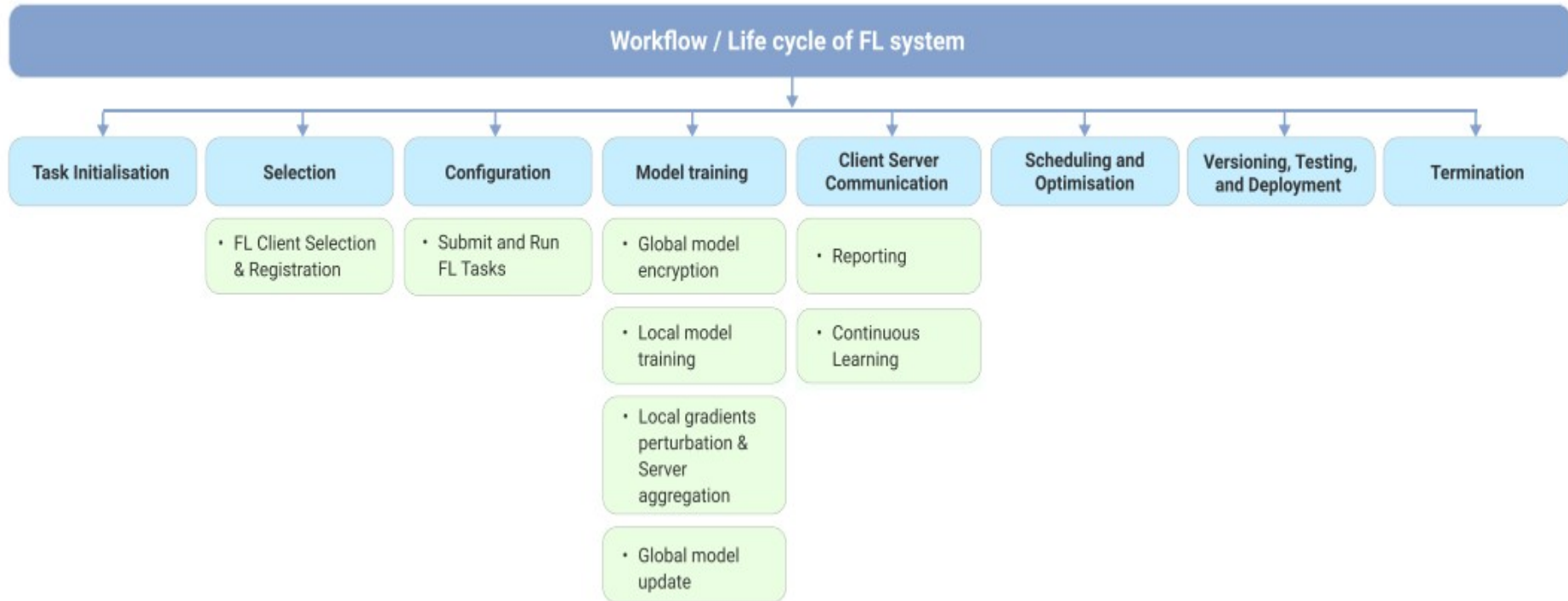
Blood Bank Database



Hospital Database



Machine Learning pipeline.



Challenges of FL in Healthcare



Privacy and Attacks

- Adversarial attacks
- Inference and poisoning attacks
- Genomic data privacy



Data Challenges

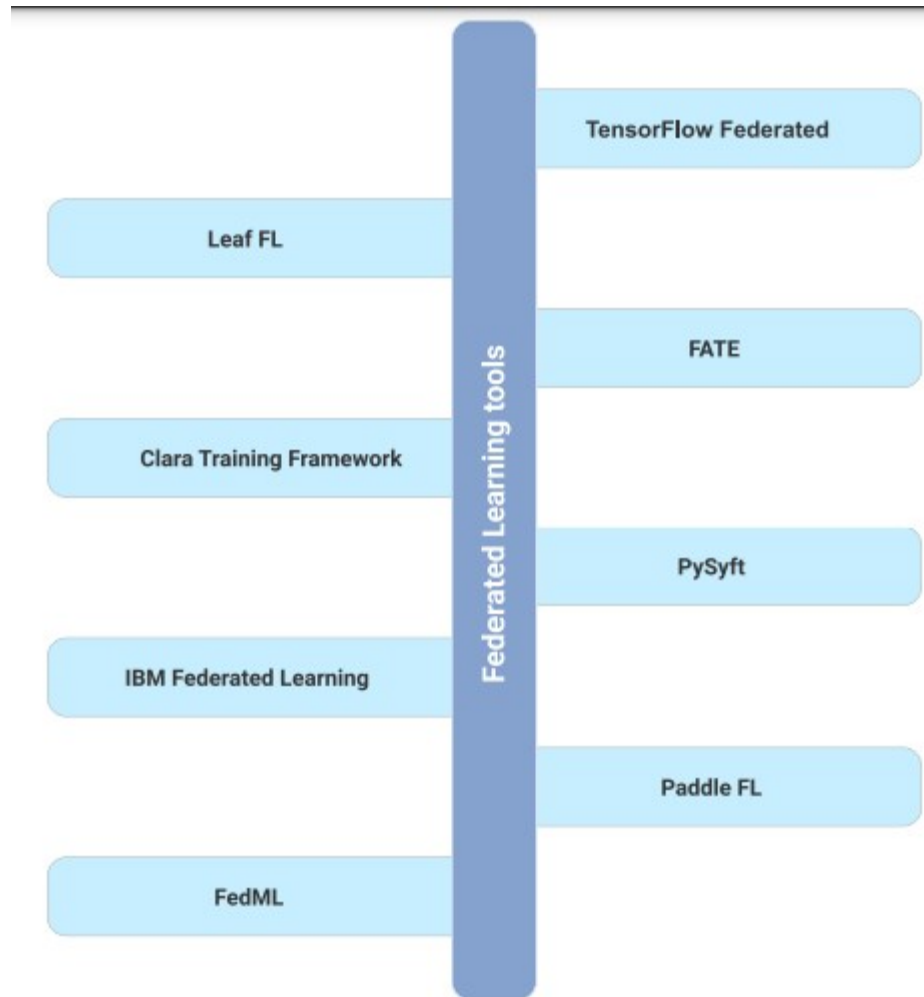
- Data quality
- Data biases
- Data noises



Communication Challenges

- Failure and dropouts
- Energy Consumption
- Computational and Communication overhead

Tools of federated learning



federated learning

Federated Averaging (FedAvg) Algorithm:

- Federated Averaging (FedAvg) is one of the most widely used algorithms in federated learning.
- The key idea is to train models locally on decentralized data held by different clients and then average the model weights to update the global model.
- This approach allows for **privacy-preserving training** since the data does not leave the clients, only model updates are shared with the server.

federated learning

FedAvg Algorithm Steps:

- Initialize a global model on the central server.
- Distribute the global model to a set of clients.
- Each client trains the model locally on its private data for a fixed number of epochs (or iterations).
- After local training, each client sends the updated model weights (or gradients) back to the server.
- The server aggregates the model weights by taking a weighted average (based on the size of local datasets) to update the global model.
- The process repeats for a predefined number of rounds until convergence.

federated learning

Mathematical Representation of FedAvg:

Let there be K clients, each with a dataset D_k of size n_k . The global dataset size is $N = \sum_{k=1}^K n_k$. Each client k updates its local model w_k using Stochastic Gradient Descent (SGD) or another optimizer.

- **Local Update:** Each client performs:

$$w_k^{t+1} = w_k^t - \eta \nabla L_k(w_k^t)$$

where $\nabla L_k(w_k^t)$ is the gradient of the local loss for client k , and η is the learning rate.

- **Global Aggregation:** The server aggregates the weights from all clients by weighted averaging:

$$w^{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_k^{t+1}$$

where w^{t+1} is the updated global model.

What is Federated Matched Averaging (FedMA)?

- FedMA is an extension of the FedAvg algorithm. In FedAvg, all clients' model weights are averaged directly, which works well when the clients' data distributions are homogeneous.
- However, in practice, clients may have heterogeneous data (non-i.i.d. data), leading to varying importance or roles for **different neurons across clients' models**.
- FedMA addresses this issue by **matching neurons across clients** before averaging them, ensuring that neurons with similar roles are aligned.

Key Features of FedMA:

Neuron Matching: FedMA introduces a neuron matching procedure, where neurons are matched between clients based on their importance and role in the model (e.g., based on activation similarity). This ensures that only neurons with similar functionalities across clients are averaged together.

Layer-Wise Matching: Neuron matching happens layer by layer, starting from the input layer to the output layer. Each layer's neurons are matched across clients before the aggregation of weights happens.

Weighted Averaging: Once neurons are matched, the matched neurons' weights are aggregated using weighted averaging (similar to FedAvg but only for matched neurons).

federated learning

FedMA Algorithm:

Step-by-Step Process:

1. **Initialize Global Model:** The central server initializes the global model and sends it to the clients.
2. **Local Model Training:** Each client trains its local model on its private data for a specified number of epochs, updating the weights of the model based on its data.
3. **Neuron Matching:**
 - After local training, the client models are sent back to the server.
 - The server performs **neuron matching** by comparing the neurons (at each layer) from the clients' models. Neurons are matched based on their **activation similarities** or other metrics, such as neuron importance. This ensures that neurons with similar roles are aligned before aggregation.

federated learning

4. Layer-Wise Weight Aggregation:

- Once the neurons are matched, the corresponding weights are **aggregated** using a weighted average similar to FedAvg.
- This process is performed **layer by layer** across the network.

5. Global Model Update: The server updates the global model with the newly aggregated weights from the matched neurons.

6. Repeat: This process repeats for several rounds, similar to FedAvg, until convergence or a stopping criterion is reached.

Federated Personalization (FedPer)

Overview:

FedPer is a federated learning algorithm that focuses on personalizing models for each client. While the global model is trained on all clients' data, certain layers of the neural network (usually the last layers) are left client-specific, allowing each client to personalize these layers based on its local data.

Shared Layers: Lower layers of the neural network, which learn generalized features, are shared across clients.

Personalized Layers: The higher layers, which are more specific to each client's data, are kept private and updated locally.

Key Idea:

- The lower layers of the network are updated collaboratively across clients using traditional federated averaging, but each client updates its own personalized layers based on its local data.
- This leads to a personalized model for each client while still benefiting from the shared knowledge of the global model.

Algorithm Steps:

- Train the shared layers (lower layers) collaboratively using FedAvg.
- Train the personalized layers (upper layers) on each client's private data.
- Keep the personalized layers on each client and avoid sharing them with the server.

Advantages:

Personalized Models: Each client gets a model tailored to its data.

Efficient: Personalization happens only in a few layers, so it's computationally efficient.

Better Performance: Models perform better when trained on non-i.i.d. data since each client adapts its model to its local data.

2. Federated Neural Architecture Search (FedNAS)

Overview:

- FedNAS allows each client to search for the best local neural architecture that works for its data.
- Unlike FedAvg, where all clients share the same architecture, FedNAS dynamically discovers the optimal model architecture for each client, and the server aggregates both architectures and weights.

NAS: Neural Architecture Search explores different architectures, typically using reinforcement learning or evolutionary strategies.

Personalized Architectures: Each client can have a different architecture optimized for its own data.

Federated Neural Architecture Search (FedNAS)

Algorithm Steps:

- Each client performs a Neural Architecture Search (NAS) locally to find the optimal architecture for its data.
- The server aggregates the discovered architectures and model weights across clients.
- The server sends the best architecture to clients for global training.

Steps for Implementation:

Define the Search Space: Create a space of possible CNN architectures.

Local NAS Search: Each client searches for the best architecture by evaluating different models on its own data.

Federated Training: After each round, the clients' best architectures are sent to the server, and we aggregate the architectures and weights to update the global model.

1. Client's Best Architecture:

- Each client performs a local search for the best architecture.
- This means that each client has a search space of potential network architectures, and it evaluates these architectures based on its own data (local training data).
- After evaluating the architectures, the client selects the one that performed the best (i.e., the one that gave the highest accuracy on its local data).

2. Sending Best Architectures to the Server:

Once each client selects its best architecture, it sends the following information to the server:

Architecture Description: This includes information about the structure of the network (e.g., number of convolutional layers, number of neurons, pooling layers, etc.).

Optional:

Weights: In some cases, the client may also send the trained weights associated with the best architecture.

This information helps the server know what the best architecture is, and if weights are included, the server can use these trained weights to improve the global model more quickly.

3. Aggregation at the Server:

The server receives the best architectures from all the clients. There are two possible things that can happen here:

Architecture Aggregation: The server could either select the single best architecture out of all the clients' suggestions or combine multiple architectures.

Aggregating architectures means taking the structural information from clients and creating a combined architecture that could perform well across all clients.

Weight Aggregation: If clients have sent weights, the server can average the weights (similar to the FedAvg algorithm) to create a single set of weights for the global model.

4. Global Model Update:

After the server aggregates the architectures and weights, it creates a new global model.

This global model could:

- Use the best architecture or a combined architecture from all clients.
- Use averaged weights or a combined set of weights.
- This global model is then sent back to the clients for the next round of training.

Why Aggregate Architectures and Weights?

Architecture Aggregation: The goal is to learn a neural network structure that works well for all clients. Clients might have different data distributions, so aggregating architectures can help create a more generalized model.

Weight Aggregation: This is similar to traditional federated learning (FedAvg), where model weights are averaged across clients to update the global model.

In the case of FedNAS, the weights from the best architectures are averaged to form the final global model.

Benefits of Aggregating Architectures and Weights:

Generalization: By aggregating both architectures and weights from different clients, the global model can learn to generalize across diverse data distributions that the clients have.

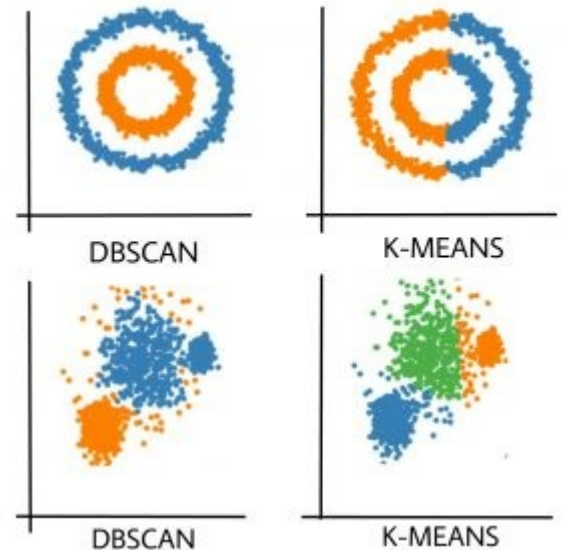
Adaptation: Different clients may have different data patterns. FedNAS allows each client to adapt its architecture locally, while still contributing to the global model.

Efficiency: The aggregation of weights helps in speeding up convergence because clients don't start training from scratch after each round. Instead, they use the updated global weights.

Distributed DBSCAN and Logistic Regression

Normal DBSCAN Algorithm:

1. Initialize all points as unvisited.
2. For each unvisited point P:
 - a. Mark P as visited.
 - b. Retrieve the density-reachable points from P (neighbors).
 - c. If P has enough neighbors (MinPts):
 - i. Start a new cluster.
 - ii. Recursively add all density-reachable points to the cluster.
 - d. If P does not have enough neighbors, mark P as noise.
3. Continue until all points have been visited.



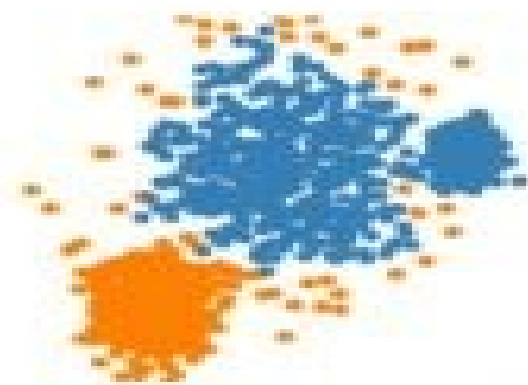
Normal DBSCAN Algorithm:



DBSCAN



K-MEANS



DBSCAN



K-MEANS

Distributed DBSCAN Algorithm:

In a distributed environment, DBSCAN is adapted to handle large datasets by partitioning the data across multiple nodes and then combining the results.

Challenges:

Synchronization: Ensuring that clusters identified on different nodes are correctly merged.

Convergence: Making sure that all parts of the dataset have been properly clustered.

Approach:

Partitioning the Data: The dataset is split across multiple nodes.

Local DBSCAN: Each node runs DBSCAN on its local partition.

Border Point Exchange: Nodes exchange information about points near the boundaries of their partitions.

Cluster Merging: Clusters that span multiple partitions are merged based on shared border points.

In a distributed environment, the DBSCAN algorithm is adapted to handle large-scale datasets by dividing the workload across multiple nodes. This process involves several key steps to ensure that the clusters identified on different nodes are correctly merged into a consistent global result.

1. Partition the Dataset into Overlapping Regions Across Nodes:

Purpose: The dataset is divided into smaller subsets (partitions) that can be processed independently on different nodes. Overlapping regions are crucial because clusters might span across the boundaries of these partitions.

Details:

Each node is responsible for a specific region of the dataset. However, to correctly identify clusters that cross partition boundaries, each node is also given some data points from neighboring regions.

The size of the overlap is usually determined by the ϵ parameter of DBSCAN, ensuring that if a cluster extends beyond a partition's boundary, the node can still recognize it.

Example: If $\epsilon=3$ and a dataset is divided into two partitions, the overlap between these partitions should be at least 3 units wide. This way, points close to the boundary in one partition will include their potential neighbors from the adjacent partition.

2. Each Node Runs Local DBSCAN on Its Partition:

Purpose: Once the data is partitioned, each node runs the DBSCAN algorithm independently on its subset of data. This step identifies local clusters within each partition.

Details:

- Local DBSCAN processes each partition as if it were the entire dataset, identifying core, border, and noise points.
- Each node produces a set of clusters based on the local data and the specified `eps` and `min_samples` parameters.

Example: In a dataset divided into three partitions, Node 1 might identify clusters in its partition, Node 2 in its partition, and so on, without initially considering how these clusters might connect across partitions.

3. Identify Border Points That Could Belong to Clusters Spanning Multiple Nodes:

Purpose: After running local DBSCAN, each node needs to identify points near the boundaries of its partition that might belong to clusters extending into neighboring partitions. These are the border points.

Details:

Border points are **critical** because they determine whether clusters identified in different partitions are actually part of the same larger cluster.

Each node checks which of its clusters include points that are within the overlap region shared with neighboring partitions. These points are potential candidates for merging clusters.

Example: Suppose a point in Node 1's partition is within ϵ distance of a point in Node 2's partition. If both points are core or border points in their respective partitions, they might belong to the same cluster.

4. Exchange Border Points with Neighboring Nodes:

Purpose: To determine if clusters should be merged across partitions, nodes exchange information about their border points with neighboring nodes.

Details:

Nodes send the coordinates and cluster assignments of their border points to adjacent nodes. Neighboring nodes then check if these border points match any of their own clusters, i.e., if the points in both nodes are within ϵ distance of each other and belong to a dense region.

Example: Node 1 sends the information of its border points to Node 2. Node 2 checks if these points connect to any of its own clusters. If they do, Node 1 and Node 2 will consider merging these clusters.

5. Merge Clusters Across Nodes If They Share Border Points:

Purpose: If clusters identified on different nodes share border points, these clusters are merged to form a single, larger cluster that spans multiple partitions.

Details:

After exchanging border points, nodes update their cluster assignments based on the results of the comparisons. Clusters that are found to be connected via border points are merged.

The merging process ensures that clusters are not split artificially just because they happen to lie on partition boundaries.

Example: If a cluster in Node 1 shares border points with a cluster in Node 2, these clusters are merged, and both nodes update their cluster labels to reflect this.

6. Repeat the Process Until Convergence (No Further Cluster Merging):

Purpose: This process is repeated until no further merging of clusters across partitions is possible, ensuring that the final clusters are consistent and complete.

Details:

Convergence is reached when all nodes agree on the cluster assignments, and no new connections are found that would require additional merging.

This step may involve multiple rounds of communication and merging, especially in complex datasets where clusters extend across multiple partitions.

Example: After the first round of merging, the nodes check again to ensure that no new clusters have formed that need to be merged. Once no further merging is required, the algorithm converges, and the final cluster labels are assigned.

Distributed DBSCAN Algorithm:

In a distributed environment, DBSCAN is adapted to handle large datasets by partitioning the data across multiple nodes and then combining the results.

Challenges:

Synchronization: Ensuring that clusters identified on different nodes are correctly merged.

Convergence: Making sure that all parts of the dataset have been properly clustered.

1. Introduce Overlapping Partitions

One way to handle clusters that span across partition boundaries is to introduce overlapping regions between the partitions. This overlap will allow DBSCAN to detect and merge clusters that would otherwise be split across partitions.

2. Adjust DBSCAN Parameters

Fine-tuning the **eps** and **min_samples** parameters can lead to better clustering. For instance, a slightly larger eps value might help in connecting clusters that are close but on the border.

3. Merge Clusters Across Partitions

After running DBSCAN locally on each partition, you should identify border points and merge clusters across partitions based on proximity.

4. Automate Partitioning and Merging

Instead of manual partitioning, you could automate the partitioning process and implement logic to merge clusters after each local DBSCAN run.

SGD in Neural Network Training:

Overview of MultiWorkerMirroredStrategy:

- MultiWorkerMirroredStrategy is a synchronous distributed training strategy. It allows you to distribute your model's training across multiple workers (each potentially running on different machines).
- Each worker runs the same code independently, but TensorFlow manages synchronization to ensure that the model parameters are consistently updated across all workers.

SGD in Neural Network Training:

1. Cluster Configuration (TF_CONFIG)

- **TF_CONFIG Environment Variable:** Before training starts, you define the cluster configuration using the `TF_CONFIG` environment variable. This configuration tells each worker what role it plays (e.g., worker, chief) and how to communicate with other workers.
- **Worker Roles:**
 - **Worker 0 (Chief Worker):** This is usually the first worker (`index: 0`). It's responsible for coordinating the training, saving checkpoints, and managing other global tasks like writing TensorBoard logs.
 - **Worker 1, 2, ...:** These are additional workers that participate in training. They perform the same training steps but do not handle tasks like checkpointing.

SGD in Neural Network Training:

2. Model Definition and Strategy Scope:

Strategy Scope: The model and optimizer are defined within the `strategy.scope()`. This ensures that TensorFlow distributes the model and data correctly across all workers.

Synchronization: Gradient Averaging: While each worker computes gradients based on its own data subset, the gradients are averaged across all workers before updating the model parameters. This helps in efficiently utilizing the distributed resources while maintaining the integrity of the training process.

- Each worker initializes the model parameters independently, but before training starts, TensorFlow synchronizes these parameters to ensure consistency.
- Any updates to the model parameters are averaged across all workers.

Distributed K-means and SVM

Dr. V. Chandra Sekhar

Distributed K-Nearest Neighbors (K-NN) Algorithm

Objective: Classify a new data point using distributed data across multiple nodes.

1. Input:

- A distributed dataset across multiple nodes.
- A new data point to classify.
- The number of neighbors, k , to consider.

2. Steps:

1. Local Distance Calculation:

- Each node computes the distance between the new data point and all the data points it holds.

2. Select Local Neighbors:

- Each node selects its top k nearest neighbors based on the computed distances.

3. Global Neighbor Selection:

- Each node sends its k nearest neighbors and their distances to a central node or coordinator.
- The central node collects the nearest neighbors from all nodes and selects the global top k nearest neighbors.

4. Majority Voting:

- The central node performs majority voting among the k -nearest neighbors to determine the class label for the new data point.

5. Output:

- The class label for the new data point.

3. End.

Distributed K-Means Clustering Algorithm

Objective: Partition a distributed dataset into k clusters, ensuring coordination across multiple nodes.

1. Input:

- A distributed dataset across multiple nodes.
- The number of clusters, k .

2. Steps:

1. Initialize Global Cluster Centers:

- Initialize k cluster centers (centroids) either randomly or by aggregating initial points from all nodes.
- Broadcast the initial centroids to all nodes.

2. Local Assignment:

- Each node assigns its local data points to the nearest centroid (based on the initial centroids).

3. Local Centroid Update:

- Each node calculates the centroids of the clusters formed by its local data points.
- Send the local centroids and their associated weights (number of points) to the central node.

4. Global Centroid Update:

- The central node aggregates the local centroids from all nodes to compute the new global centroids (weighted average).
- Broadcast the new centroids to all nodes.

5. Repeat:

- Repeat steps 2 to 4 until the centroids converge (i.e., no significant change) or a maximum number of iterations is reached.

6. Output:

- The final cluster centroids and the assignment of data points to clusters.

3. End.

Distributed Support Vector Machine (SVM) Algorithm

Objective: Train an SVM classifier on a distributed dataset by coordinating across multiple nodes.

1. Input:

- A distributed dataset across multiple nodes.
- The type of SVM (e.g., linear, kernel-based).
- The regularization parameter C .

2. Steps:

1. Local Gradient Computation:

- Each node independently computes the local gradient of the objective function based on its local data.

2. Global Gradient Aggregation:

- Each node sends its computed gradient to a central node or coordinator.
- The central node aggregates the gradients from all nodes to form a global gradient.

3. Update Model Parameters:

- The central node updates the SVM model parameters (e.g., the weights and bias for linear SVM) using the global gradient.
- The updated model parameters are broadcast to all nodes.

4. Local Update and Repeat:

- Each node updates its local view of the SVM model with the new parameters.
- Steps 1 to 3 are repeated until convergence (i.e., when the model parameters stabilize) or a maximum number of iterations is reached.

5. Handle Non-Linear SVM (if using kernel-based SVM):

- If the dataset is not linearly separable, apply the kernel trick to transform the input space into a higher-dimensional space.
- Perform the above steps in the transformed space.

6. Output:

- The final SVM model parameters (e.g., support vectors, weights, bias) that classify the data.

3. End.

Distributed K-Nearest Neighbors (K-NN) Algorithm

Objective: Classify a new data point by aggregating nearest neighbors from multiple distributed nodes.

1. Input:

- A distributed dataset across N nodes.
- A new data point x_{new} to classify.
- The number of neighbors k to consider.

2. Steps:

1. Local Distance Calculation (at each node):

- For each node i , calculate the distance between x_{new} and all data points X_i stored in that node.
- Store the distances and corresponding class labels locally.

2. Select Local Neighbors (at each node):

- Each node i selects its top k nearest neighbors based on the computed distances.

3. Global Neighbor Collection (at the central node):

- Each node i sends its k nearest neighbors and their distances to a central node or aggregator.
- The central node collects the neighbors from all N nodes.

4. Global Neighbor Selection (at the central node):

- The central node sorts the collected neighbors by distance.
- Select the global top k nearest neighbors from the combined list.

5. Majority Voting (at the central node):

- The central node performs majority voting among the k -nearest neighbors to determine the class label for x_{new} .

6. Output:

- The class label for the new data point x_{new} .

3. End.

Distributed K-Means Clustering Algorithm

Objective: Partition a distributed dataset into k clusters by synchronizing centroids across multiple nodes.

1. Input:

- A distributed dataset across N nodes.
- The number of clusters k .

2. Steps:

1. Initialize Global Centroids:

- Randomly select k initial cluster centroids.
- Broadcast these initial centroids to all nodes.

2. Local Assignment (at each node):

- Each node i assigns its local data points X_i to the nearest centroid.
- Store the local assignments and compute the local centroids (means) of the clusters.

3. Local Centroid Computation (at each node):

- Each node i calculates the mean of the data points assigned to each centroid to form local centroids.

4. Global Centroid Aggregation (at the central node):

- Each node i sends its local centroids and the number of points assigned to each centroid to the central node.
- The central node aggregates these local centroids (weighted by the number of points) to compute new global centroids.

5. Update Global Centroids:

- The central node broadcasts the new global centroids to all nodes.

6. Repeat:

- Repeat steps 2 to 5 until the centroids converge (i.e., when the centroids no longer

Objective: Train an SVM classifier on a distributed dataset by iteratively updating the model parameters across multiple nodes.

1. Input:

- A distributed dataset across N nodes.
- The type of SVM (e.g., linear, kernel-based).
- The regularization parameter C .

2. Steps:

1. Initialize Model Parameters:

- Initialize the SVM model parameters (e.g., weights w and bias b for linear SVM).
- Broadcast the initial parameters to all nodes.

2. Local Gradient Computation (at each node):

- Each node i computes the gradient of the SVM objective function based on its local data X_i .

3. Global Gradient Aggregation (at the central node):

- Each node i sends its computed gradient to the central node.
- The central node aggregates the gradients from all nodes to form the global gradient.

4. Update Model Parameters (at the central node):

- The central node updates the SVM model parameters (e.g., w and b for linear SVM) using the global gradient.
- The updated parameters are broadcast to all nodes.

Distributed Synchronous SGD: Concepts and Challenges

- Distributed Synchronous SGD: Concepts and Challenges
- Decentralized SGD: Methodology and Applications
- Asynchronous SGD: Approaches, Benefits, and Drawbacks
- Overlap and Quantized SGD: Innovations and Impacts

1. Distributed Synchronous SGD:

- **Concept:** In this approach, all workers compute gradients based on their subset of the data and then synchronize those gradients at the end of each iteration. The averaged gradients are then used to update the model parameters. This synchronization ensures that all workers have identical model parameters after each update.
- **Challenges:** The primary challenge is the synchronization overhead, especially in heterogeneous environments where some workers (stragglers) might be slower. This can lead to idle time for faster workers and limits scalability. High communication costs due to frequent synchronization can also become a bottleneck.

2. Decentralized SGD:

- **Methodology:** Instead of synchronizing with a central server, each worker maintains its model and periodically exchanges parameters with its neighbors. This approach reduces reliance on a central parameter server and minimizes communication overhead.
- **Applications:** Decentralized SGD is suitable for settings where communication costs are high, such as in edge computing or federated learning, and where central coordination is impractical. It can tolerate failures better since updates are local to each worker or its neighbors.

3. Asynchronous SGD:

- **Approaches:** In Asynchronous SGD, each worker independently computes gradients and updates the global model parameters without waiting for other workers. This approach can reduce idle times and is effective in environments with heterogeneous hardware or varying network speeds.
- **Benefits and Drawbacks:** The main benefit is the reduction in idle times and improved utilization of computational resources. However, the lack of synchronization can lead to stale gradients, potentially slowing convergence or impacting model accuracy.

4. Overlap and Quantized SGD:

- **Innovations and Impacts:** This approach combines two techniques: overlapping communication with computation and using quantized gradients. Overlapping reduces idle times as communication and computation occur simultaneously. Quantized gradients reduce the precision of gradients to minimize communication overhead.
- **Impact on Performance:** While quantization can reduce communication costs significantly, it introduces rounding errors and noise into the gradient updates, which can impact convergence and accuracy. The overlap strategy helps mitigate the communication bottleneck by using the time during gradient computation for communication.

Concepts:

- In distributed synchronous SGD, multiple workers (machines or GPUs) participate in training a shared model.
- Each worker computes the gradients on its subset of the data, and the gradients are averaged across all workers to update the model parameters synchronously.
- The training is synchronized at the end of each iteration, meaning all workers must complete their gradient computation before the model parameters are updated.

Challenges:

Communication Overhead: Synchronizing gradients across all workers can be communication-intensive, especially as the number of workers increases.

Stragglers: The overall training speed is limited by the slowest worker (straggler). If one worker is slower due to hardware variability, network latency, or any other reason, all other workers must wait, causing inefficiency.

Scalability: As the number of workers grows, the time spent on communication and synchronization increases, which can offset the benefits of parallelism.

Asynchronous SGD: Approaches, Benefits, and Drawbacks

Approaches:

- Asynchronous SGD (ASGD) allows each worker to compute gradients and update the model parameters independently, without waiting for other workers to finish their computation.
- Each worker communicates with a central parameter server or uses a peer-to-peer strategy to update the global model.

SGD in Neural Network Training:

Benefits:

- **Reduced Waiting Time:** No need for synchronization; workers can proceed independently, which can reduce the waiting time associated with synchronous SGD.
- **Tolerant to Stragglers:** Faster workers do not wait for slower ones, making the training process more robust to heterogeneous environments.

ASGD in Neural Network Training:

Drawbacks:

Stale Gradients: Since updates are performed asynchronously, gradients computed by a worker might be stale, i.e., computed based on outdated parameters. This can affect convergence rates and stability.

Convergence Challenges: ASGD often requires careful tuning of hyperparameters, such as learning rate, to ensure proper convergence.

Decentralized SGD: Methodology and Applications

Methodology:

- In decentralized SGD, there is no central parameter server. Instead, each worker maintains its own model parameters and periodically exchanges these parameters with its peers.
- This method reduces reliance on a central server and can effectively utilize network resources by minimizing bottlenecks associated with centralized architectures.

Applications:

- Decentralized SGD is particularly useful in environments where network latency is a concern or where a central server could become a bottleneck.
- It's also suitable for federated learning, where data privacy is crucial, and training data cannot leave local devices.

Decentralized SGD in Neural Network Training:

1. Include realistic gradient computation.
2. parameter updates, and
3. parameter exchange between workers.

This decentralized approach ensures that each worker updates its parameters independently and occasionally synchronizes with its neighbors to ensure convergence.

1. Each worker has its own set of parameters and computes gradients based on its subset of data.
2. Workers will periodically communicate with their "neighbors" (other workers) to exchange parameter information, mimicking a decentralized training setup.

Decentralized SGD in Neural Network Training:

- In a decentralized SGD setup, there are no "common parameters" shared across all workers. Instead, each worker maintains its own set of model parameters and updates them independently based on its local data.
- Periodic communication or parameter exchange between workers (based on the defined topology) helps the distributed models to converge towards a consensus, but there is no central or shared parameter storage.

Overlap and Quantized SGD: Innovations and Impacts

Overlap SGD:

- Overlap SGD allows for overlapping communication and computation tasks.
- Workers start computing the next set of gradients while the current gradients are being communicated, thus hiding the communication overhead.
- This method is especially beneficial when communication costs are high compared to computation costs.

Overlap and Quantized SGD: Innovations and Impacts

Quantized SGD:

- Quantized SGD reduces the precision of gradients before transmitting them across workers, thus reducing the communication bandwidth requirement.
- This method can speed up training in distributed environments by reducing the amount of data sent over the network.

Overlap and Quantized SGD: Innovations and Impacts

Overlap SGD: Helps in achieving better utilization of resources by reducing idle times due to communication.

Quantized SGD: Significantly reduces the communication overhead, allowing for faster iterations, particularly in large-scale settings.

Overlap and Quantized SGD: Innovations and Impacts

Formula for Gradient Quantization

Quantization is a process where we reduce the precision of gradient values to a limited number of bits. The quantization process can be summarized as follows:

1. Find the maximum absolute value of the gradients:

$$\text{max_val} = \max(|\text{gradients}|)$$

2. Scale the gradients to fit within the range defined by the number of bits:

$$\text{scale} = \frac{2^{\text{bits}} - 1}{\text{max_val}}$$

3. Quantize the gradients by scaling and rounding:

$$\text{quantized_gradients} = \frac{\text{round}(\text{gradients} \times \text{scale})}{\text{scale}}$$

Overlap and Quantized SGD: Innovations and Impacts

Parameter	Distributed Synchronous SGD	Decentralized SGD	Asynchronous SGD	Overlap and Quantized SGD
Computation Efficiency	High: Each worker computes gradients in parallel. However, all workers must wait for the slowest one to synchronize, leading to potential idle time.	Moderate: Each worker computes gradients and updates independently. Computation efficiency is influenced by network topology and parameter exchange frequency.	High: No waiting for synchronization; each worker computes and updates independently, resulting in efficient use of resources.	High: Computation overlaps with communication, reducing idle time. Quantization reduces the computational load by processing fewer bits.

Overlap and Quantized SGD: Innovations and Impacts

Communication Overhead	High: Requires frequent synchronization of all gradients across workers after each batch. The amount of data communicated is proportional to the number of workers and model size.	Low: Communication is limited to a subset of neighbors or local nodes, reducing total communication load. The overhead depends on the network topology.	Moderate: Frequent but smaller updates reduce the size of communication. Communicates independently with a parameter server or peers, reducing communication volume.	Low: Quantized gradients reduce the size of communication. Overlapping communication with computation hides communication delays.
------------------------	---	--	---	--

Overlap and Quantized SGD: Innovations and Impacts

Synchronization Required	Yes: Workers must synchronize gradients after each step, requiring all workers to wait for the slowest one (straggler). This ensures consistent updates but increases waiting time.	No: There is no global synchronization. Workers only synchronize with their immediate neighbors or based on a predefined topology, allowing for more flexibility and independence.	No: Each worker updates asynchronously without waiting for others, reducing delays from stragglers but potentially using stale gradients.	Depends: Overlapping tasks may require some synchronization, but the impact is reduced by overlapping communication and computation.
---------------------------------	--	---	--	---

Overlap and Quantized SGD: Innovations and Impacts

Scalability	Limited: Scalability is constrained by synchronization overhead and the straggler effect, which becomes more pronounced with an increasing number of workers.	High: Scales well as there is no global synchronization. The decentralized approach allows adding more workers without significantly increasing communication overhead.	High: Scales well since each worker operates independently. The system is less affected by network delays, making it suitable for large-scale deployments.	High: Reduces communication overhead and hides latency, enabling better scalability even in environments with many nodes.
-------------	--	--	---	--

Overlap and Quantized SGD: Innovations and Impacts

Fault Tolerance	Low: The failure of a single worker can halt training or cause significant delays due to synchronization requirements.	High: Failure of individual workers affects only local regions or neighbors, not the entire network. The system continues to function despite some node failures.	Moderate: Tolerates worker failures better as updates are asynchronous, but the parameter server or network issues can still impact training.	Moderate to High: The system can tolerate some failures due to reduced communication needs and overlapping tasks, but quantization errors can accumulate.
Accuracy	High: Precise gradient averaging across all workers ensures consistent and accurate updates, leading to high model accuracy.	Moderate: Accuracy depends on the frequency and strategy of parameter exchanges. Infrequent communication may introduce discrepancies among models.	Potentially Lower: Stale gradients and inconsistent updates can reduce accuracy, requiring more epochs or lower learning rates to compensate.	Lower: Quantization introduces noise and rounding errors, potentially affecting accuracy. The impact on accuracy can be controlled by adjusting quantization levels.

Overlap and Quantized SGD: Innovations and Impacts

Comparison and Analysis of Computational Complexity

1. Distributed Synchronous SGD:

- **Computation:** Each worker computes gradients with a complexity of $O(G)$, where G represents the gradient computation time.
- **Communication:** After each batch, all workers must synchronize, requiring communication of all gradients. The complexity of this communication is $O(C)$, where C depends on the network bandwidth and latency.
- **Overall Complexity:** $O(W \cdot G + C)$. The synchronization overhead (communication cost C) is significant, especially as the number of workers W increases. This approach can be bottlenecked by the slowest worker (straggler), leading to inefficiencies in computation and communication.

Overlap and Quantized SGD: Innovations and Impacts

Computational Complexity of Each SGD Approach

SGD Approach	Computational Complexity	Explanation
Distributed Synchronous SGD	$O(W \cdot G + C)$	Each worker computes gradients in $O(G)$ and then participates in a synchronization step, which incurs a communication complexity of $O(C)$. The total complexity depends on the number of workers W , gradient size G , and communication cost C .
Decentralized SGD	$O(W \cdot G + T \cdot N \cdot C)$	Each worker computes gradients independently in $O(G)$. The communication complexity is lower, $O(T \cdot N \cdot C)$, where T is the topology factor (number of neighbors) and N is the number of parameter exchanges.
Asynchronous SGD	$O(W \cdot G + A \cdot C)$	Each worker computes gradients in $O(G)$. The communication complexity depends on the frequency of updates A to the parameter server and the cost per communication C . The complexity is lower than synchronous SGD due to the absence of global synchronization.
Overlap and Quantized SGD	$O(W \cdot G/Q + O(C/Q))$	The computation is overlapped with communication, and gradients are quantized, reducing the computation to $O(G/Q)$ and communication cost to $O(C/Q)$, where Q is the quantization level (bit reduction factor). This significantly reduces the complexity compared to other methods.

Overlap and Quantized SGD: Innovations and Impacts

- **Distributed Synchronous SGD:** Complexity Level: High (due to synchronization and communication overhead).
- **Decentralized SGD:** Complexity Level: Moderate (lower communication cost but still dependent on the network topology and frequency of parameter exchanges).
- **Asynchronous SGD:** Complexity Level: Moderate to Low (no global synchronization, but still requires frequent updates, which can be costly depending on network conditions).
- **Overlap and Quantized SGD:** Complexity Level: Low (efficient overlap of communication and computation, and reduced data size due to quantization).

Overlap and Quantized SGD: Innovations and Impacts

Which One is Best?

The "best" SGD technique depends on several factors, including network conditions, hardware capabilities, model size, and training environment. Here's a quick guide to help choose the most suitable approach:

For High Accuracy and Reliable Environments:

- Distributed Synchronous SGD is the best choice when you have a high-speed, reliable network and homogeneous hardware.
- It ensures consistent and accurate model updates but can suffer from high communication overhead and scalability issues.

For Scalability and Fault Tolerance:

- Decentralized SGD is ideal for environments where network reliability is an issue, or central coordination is impractical.
- It scales well and is fault-tolerant but may require more iterations to converge due to reduced synchronization.

For Heterogeneous and Large-Scale Environments:

- Asynchronous SGD is suitable for large-scale distributed training setups with mixed hardware and network conditions.
- It minimizes idle times and makes efficient use of resources but may result in stale gradients, requiring careful tuning of learning rates.

For Communication-Constrained Environments:

- Overlap and Quantized SGD is the best approach when communication costs are a significant concern.
- It reduces data transfer size and overlaps computation with communication, making it highly efficient in bandwidth-limited scenarios.

Each SGD technique has its strengths and is best suited for specific use cases:

- Choose Distributed Synchronous SGD for consistency and accuracy in controlled, homogeneous environments.
- Choose Decentralized SGD for edge computing, federated learning, or environments with unreliable networks.
- Choose Asynchronous SGD for large-scale, heterogeneous, or cloud environments where minimizing idle time is crucial.
- Choose Overlap and Quantized SGD for bandwidth-constrained environments or where network efficiency is critical.

SVM and Distributed SVM

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm primarily used for classification tasks, but it can also be adapted for regression problems. The fundamental idea behind SVM is to find the optimal hyperplane that separates data points of different classes with the maximum margin.

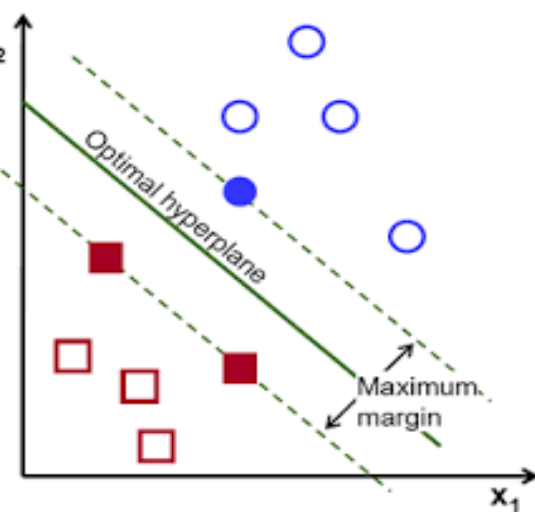
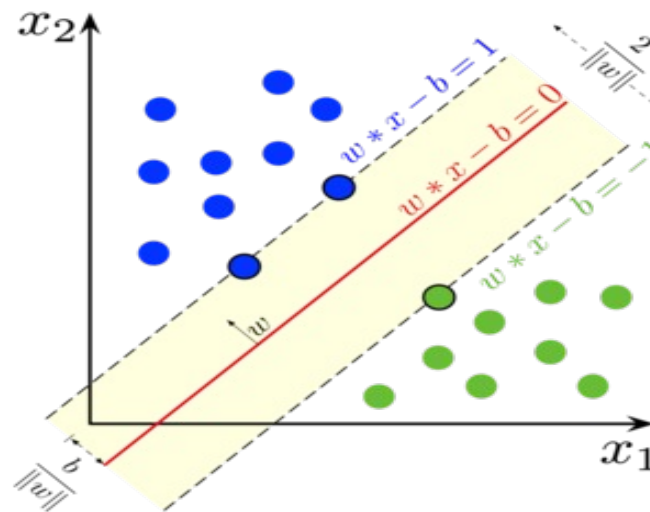
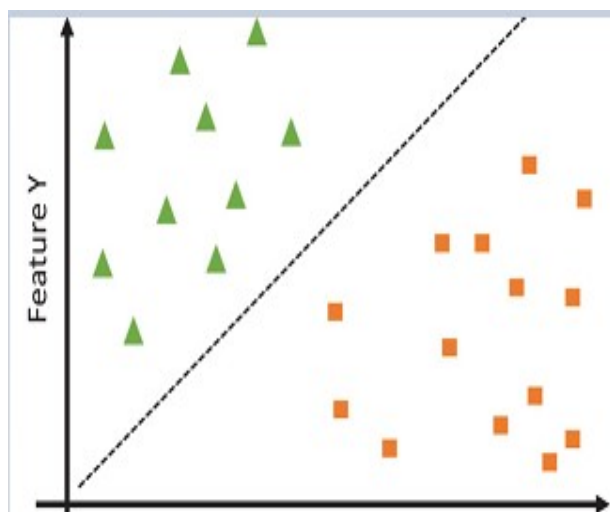
Key Concepts:

1. Hyperplane:

- A hyperplane in an n -dimensional space is a flat affine subspace of one dimension less than the space itself. For a 2D space, the hyperplane is a line, and for a 3D space, it is a plane.
- SVM aims to find the hyperplane that best separates the data into different classes.

2. Margin:

- The margin is the distance between the hyperplane and the nearest data point from either class. SVM tries to maximize this margin to ensure better generalization to unseen data.



3. Support Vectors:

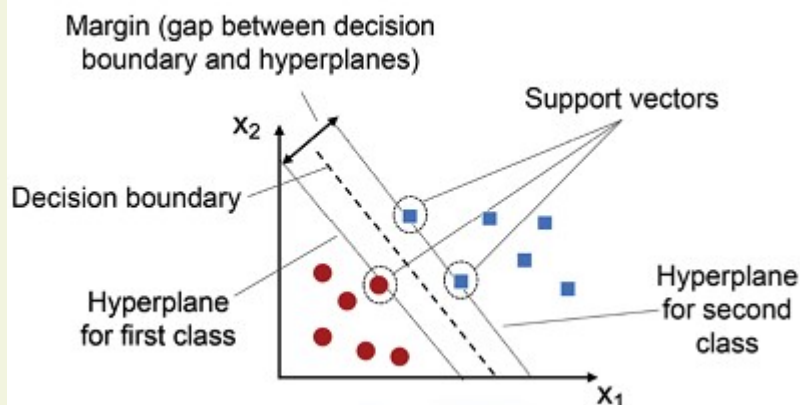
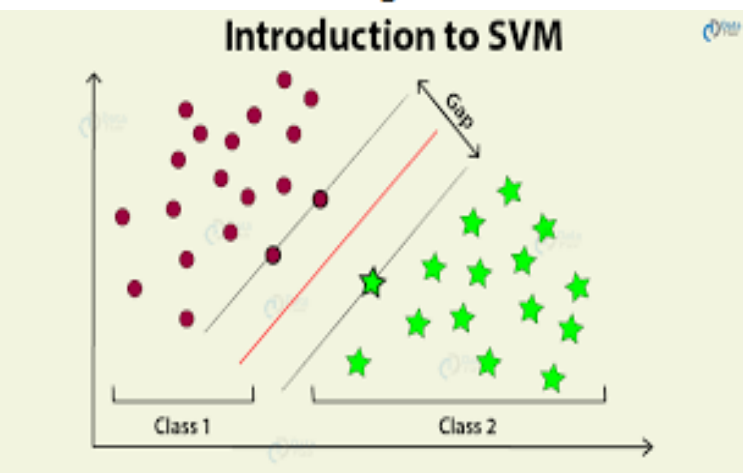
- These are the data points that are closest to the hyperplane. The position of these points is critical in determining the position and orientation of the hyperplane. They "support" the optimal hyperplane.

4. Kernel Trick:

- In cases where data is not linearly separable, SVM uses a technique called the kernel trick to map data into a higher-dimensional space where a hyperplane can separate the classes. Common kernels include linear, polynomial, radial basis function (RBF), and sigmoid.

5. Cost Function:

- SVM optimizes a cost function that includes a regularization term (to control overfitting) and a hinge loss function, which penalizes misclassified points.



- **Without Kernel Trick (Linear Kernel):** Only works well when the data is linearly separable. It fails for non-linear data.
- **With Kernel Trick (Polynomial Kernel):** Transforms the data into a higher-dimensional space, allowing a linear decision boundary in that space to correspond to a non-linear boundary in the original space.
- **With Kernel Trick (RBF Kernel):** Maps data into an infinite-dimensional space, producing a non-linear decision boundary in the original space.

1. Linear Kernel

Without Kernel Trick

For a linear SVM, the decision boundary is a hyperplane defined by:

$$f(x) = w \cdot x + b = 0$$

Where:

- w is the weight vector.
- x is the input vector.
- b is the bias term.

The decision function that determines the class is:

$$\text{class} = \text{sign}(f(x)) = \text{sign}(w \cdot x + b)$$

Example:

Suppose we have 2D data points:

- $x_1 = (1, 2), y_1 = +1$
- $x_2 = (2, 3), y_2 = +1$
- $x_3 = (3, 3), y_3 = -1$
- $x_4 = (4, 5), y_4 = -1$

1. Formulate the Problem as a Quadratic Programming Problem

The SVM optimization problem can be expressed in the following form:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

subject to: $y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \text{for all } i$

Where:

- w is the weight vector,
- b is the bias term,
- ξ_i are slack variables (for non-separable cases, but we'll ignore them for the hard margin case),
- C is the regularization parameter (set to a very large value for a hard margin).

The Role of ξ_i :

- $\xi_i = 0$: The data point is either on the margin or correctly classified outside the margin, satisfying $y_i(w \cdot x_i + b) \geq 1$.
- $0 < \xi_i < 1$: The data point is inside the margin but still correctly classified, meaning $0 \leq y_i(w \cdot x_i + b) < 1$.
- $\xi_i > 1$: The data point is misclassified, meaning $y_i(w \cdot x_i + b) < 0$.

2. Express the Constraints

Given the hard-margin SVM, the constraints simplify to:

$$y_i(w \cdot x_i + b) \geq 1 \quad \text{for all } i$$

Let's express this explicitly for each data point:

For $x_1 = (1, 2)$, $y_1 = +1$:

$$w_1 \cdot 1 + w_2 \cdot 2 + b \geq 1 \quad \text{or} \quad w_1 + 2w_2 + b \geq 1$$

For $x_2 = (2, 3)$, $y_2 = +1$:

$$2w_1 + 3w_2 + b \geq 1$$

For $x_3 = (3, 3)$, $y_3 = -1$:

$$-3w_1 - 3w_2 - b \geq 1 \quad \text{or} \quad 3w_1 + 3w_2 + b \leq -1$$

For $x_4 = (4, 5)$, $y_4 = -1$:

$$-4w_1 - 5w_2 - b \geq 1 \quad \text{or} \quad 4w_1 + 5w_2 + b \leq -1$$

3. Set Up the Lagrangian

The Lagrangian function for the primal form is:

$$L(w, b, \lambda) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \lambda_i [y_i(w \cdot x_i + b) - 1]$$

Where:

- λ_i are the Lagrange multipliers corresponding to each constraint.

The Lagrangian function for the SVM primal problem is given by:

$$L(w, b, \lambda) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \lambda_i [y_i(w \cdot x_i + b) - 1]$$

Breaking Down the Lagrangian Function

1. Objective Term:

$$\frac{1}{2} \|w\|^2$$

- This is the original objective function of the SVM, which seeks to minimize the magnitude (or norm) of the weight vector w . The term $\frac{1}{2}$ is used for mathematical convenience (it simplifies the derivative). Minimizing $\|w\|^2$ helps to maximize the margin between the two classes.

2. Constraint Term:

$$- \sum_{i=1}^m \lambda_i [y_i(w \cdot x_i + b) - 1]$$

- This term incorporates the constraints into the optimization problem. Here:
 - λ_i is the Lagrange multiplier corresponding to the i -th constraint.
 - $y_i(w \cdot x_i + b) - 1$ represents the constraint for each data point. The condition $y_i(w \cdot x_i + b) \geq 1$ ensures that the data point is correctly classified and lies outside the margin.
- If a constraint is not satisfied (i.e., $y_i(w \cdot x_i + b) < 1$), the corresponding λ_i will be positive, thereby increasing the value of the Lagrangian function and penalizing the solution. If the constraint is satisfied, λ_i can be zero or any positive value, contributing nothing to the penalty.

Lagrange Multipliers λ_i

- $\lambda_i \geq 0$ for all i , as they represent the "strength" of each constraint.
- The value of λ_i determines the influence of the corresponding constraint in the optimization process. If a constraint is active (i.e., the corresponding data point lies on the margin), then λ_i will be greater than zero. If the constraint is not active (i.e., the data point is correctly classified with room to spare), λ_i will be zero.

4. Derive the Dual Form

To solve this, we need to:

- Take the derivative of L with respect to w and b and set them to zero.
- This will give us expressions for w and b in terms of λ .

Taking derivatives:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^m \lambda_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_{i=1}^m \lambda_i y_i x_i$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^m \lambda_i y_i = 0 \quad \Rightarrow \quad \sum_{i=1}^m \lambda_i y_i = 0$$

Substituting w into the Lagrangian and simplifying leads to the dual optimization problem:

$$\begin{aligned} \max_{\lambda} \quad & \sum_{i=1}^m \lambda_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \lambda_i \lambda_j y_i y_j (x_i \cdot x_j) \\ \text{subject to:} \quad & \sum_{i=1}^m \lambda_i y_i = 0, \quad \lambda_i \geq 0 \quad \text{for all } i \end{aligned}$$

Gradient with respect to w :

$$\frac{\partial J(w, b)}{\partial w} = w - C \sum_{i=1}^m y_i x_i \text{ if } y_i(w \cdot x_i + b) < 1$$

Gradient with respect to b :

$$\frac{\partial J(w, b)}{\partial b} = -C \sum_{i=1}^m y_i \text{ if } y_i(w \cdot x_i + b) < 1$$

Update the Parameters:

- Update the weight vector w using:

$$w := w - \eta \frac{\partial J(w, b)}{\partial w}$$

- Update the bias b using:

$$b := b - \eta \frac{\partial J(w, b)}{\partial b}$$

Where η is the learning rate, controlling the step size.

The Lagrangian function for the SVM problem is given by:

$$L(w, b, \lambda) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \lambda_i [y_i(w \cdot x_i + b) - 1]$$

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^m \lambda_i y_i x_i = 0 \quad w = \sum_{i=1}^m \lambda_i y_i x_i$$

Step 2: Substitute w into the Lagrangian

Now that we have an expression for w in terms of the Lagrange multipliers λ_i , we can substitute this back into the Lagrangian function:

$$L\left(\sum_{i=1}^m \lambda_i y_i x_i, b, \lambda\right) = \frac{1}{2} \left\| \sum_{i=1}^m \lambda_i y_i x_i \right\|^2 - \sum_{i=1}^m \lambda_i \left[y_i \left(\sum_{j=1}^m \lambda_j y_j x_j \cdot x_i + b \right) - 1 \right]$$

Step 3: Simplify the Lagrangian

We simplify the Lagrangian by expanding and combining like terms:

Term 1: Simplifying $\frac{1}{2} \|w\|^2$

$$\frac{1}{2} \left\| \sum_{i=1}^m \lambda_i y_i x_i \right\|^2 = \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \lambda_i \lambda_j y_i y_j (x_i \cdot x_j)$$

Step 4: The Dual Optimization Problem

After substituting and simplifying, the Lagrangian reduces to the dual form:

$$\text{Maximize: } W(\lambda) = \sum_{i=1}^m \lambda_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \lambda_i \lambda_j y_i y_j (x_i \cdot x_j)$$

Subject to:

$$\sum_{i=1}^m \lambda_i y_i = 0 \quad \text{and} \quad \lambda_i \geq 0 \quad \text{for all } i$$

Step 1: Train Local SVMs on Each Node

Each node independently trains an SVM on its subset of the data. During training, each SVM identifies support vectors, which are the critical points that lie closest to the decision boundary.

Example Data and Decision Boundaries:

- Node 1:
 - Data points: A, B, C, D
 - Support vectors: Assume A and D are identified as support vectors.
- Node 2:
 - Data points: E, F, G, H
 - Support vectors: Assume F and H are identified as support vectors.
- Node 3:
 - Data points: I, J, K, L
 - Support vectors: Assume I and L are identified as support vectors.

Step 2: Aggregation of Support Vectors

Once each node has trained its local SVM, we gather the support vectors from each node.

- Node 1 Support Vectors: A, D
- Node 2 Support Vectors: F, H
- Node 3 Support Vectors: I, L

These support vectors are the most critical data points in defining the decision boundary for the global model.

Step 3: Train the Global SVM

Now, the global SVM is trained using only the support vectors gathered from the local models:

- **Global Support Vectors:** A, D, F, H, I, L

By using these support vectors, the global SVM will define a decision boundary that tries to approximate the decision boundaries that would have been formed if the entire dataset was available on a single machine.

Step 4: Variability in Support Vectors

- **Local vs. Global Differences:**
 - **Local Support Vectors:** Each node's support vectors are determined by the specific data subset it sees. These vectors may differ significantly from the global support vectors if the data is not well-partitioned or if the local datasets are too small.
 - **Global Support Vectors:** The global model aggregates the critical information from all local models. The support vectors of the global model are more representative of the overall data distribution, as they are derived from the most influential points from all nodes.

1. Aggregation of Support Vectors:

- **Support Vector Set Union:** The simplest approach is to take the union of all support vectors identified by the local SVMs. This means combining all the support vectors from different nodes into a single set:

$$\text{Global Support Vector Set} = \bigcup_{i=1}^k \text{Local Support Vectors}_i$$

- This aggregated set becomes the input to the global SVM training process.

2. Weighted Aggregation:

- In some implementations, particularly when using ensemble methods or when data distribution across nodes is uneven, the contribution of each local support vector might be weighted. This could be based on the number of data points in each node, the importance of the node, or other factors.

$$\text{Weighted Global Support Vector Set} = \sum_{i=1}^k \alpha_i \times \text{Local Support Vectors}_i$$

3. Global Optimization Process:

- Once the support vectors from all nodes are aggregated, the global SVM optimization process begins. The standard quadratic programming problem is solved, but with the aggregated support vectors as input:

$$\min_w \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

$$\text{subject to: } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

4. Consensus-Based Methods:

- In some distributed SVM implementations, especially in decentralized systems, a consensus-based approach is used to combine the local models. Each node shares its local model parameters (e.g., weight vector and bias), and these are averaged or combined iteratively until the models converge to a global solution.

$$w_{\text{global}} = \frac{1}{k} \sum_{i=1}^k w_i, \quad b_{\text{global}} = \frac{1}{k} \sum_{i=1}^k b_i$$

- This iterative consensus approach ensures that all nodes contribute equally to the final global model.

The equation $y_i(w \cdot x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$ is a fundamental part of the soft-margin Support Vector Machine (SVM) optimization problem. It represents the constraints that ensure each data point is either correctly classified with a margin or is within a margin violation (for misclassified or borderline points). This equation applies both in a centralized SVM training scenario and in a distributed SVM setting.

Revisiting the Equation

- $y_i(w \cdot x_i + b) \geq 1 - \xi_i$:
 - y_i is the label of the i -th data point ($+1$ or -1).
 - w is the weight vector.
 - x_i is the feature vector of the i -th data point.
 - b is the bias term.
 - ξ_i is the slack variable for the i -th data point, allowing some points to be inside the margin or misclassified.
 - The term $1 - \xi_i$ allows for soft-margin SVM, meaning the model can tolerate some errors or points inside the margin.
- $\xi_i \geq 0$:
 - Ensures that the slack variable is non-negative, meaning that points are either correctly classified with a margin or have a margin violation.

1. Local SVM Training at Each Node

- Each node j has a subset of data D_j and trains an SVM on this subset.
- The node j solves the local optimization problem:

$$\min_{w_j, b_j, \xi_j} \frac{1}{2} \|w_j\|^2 + C \sum_{i \in D_j} \xi_i$$

subject to: $y_i(w_j \cdot x_i + b_j) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i \in D_j$

2. Aggregation of Local Models

- After each node has trained its local SVM and identified local support vectors, the support vectors from all nodes are aggregated.
- The global model then incorporates these support vectors, applying the same constraint equation but on the combined set of support vectors.

$$y_i(w_{\text{global}} \cdot x_i + b_{\text{global}}) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- Here, w_{global} and b_{global} are the global weight vector and bias, respectively.

3. Framing the Global Model

- The global SVM is trained using the aggregated support vectors. The same optimization problem and constraint equations are applied:

$$\min_{w_{\text{global}}, b_{\text{global}}, \xi} \frac{1}{2} \|w_{\text{global}}\|^2 + C \sum_{i \in \text{support vectors}} \xi_i$$

$$\text{subject to: } y_i(w_{\text{global}} \cdot x_i + b_{\text{global}}) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- The global model aims to satisfy the margin constraints for the aggregated set of support vectors, combining the information from all nodes.

- Distributed Gradient Descent (DGD) is a technique used in machine learning and optimization where the gradient descent algorithm is distributed across multiple nodes or processors.
- This approach is commonly used in large-scale machine learning problems to handle large datasets or models that cannot be processed by a single machine.

Overview of Distributed Gradient Descent:

Data Partitioning: The data is divided among different workers (nodes).

Model Replication: Each worker has a copy of the model.

Local Gradient Computation: Each worker computes the gradient on its local data.

Gradient Aggregation: The gradients from all workers are aggregated, typically using a parameter server or a decentralized method.

Model Update: The global model parameters are updated based on the aggregated gradients.

Benefits of Distributed Gradient Descent:

Scalability: Enables the handling of large-scale data by distributing computation across multiple machines.

Speed: Reduces the time required to train models on large datasets by leveraging parallelism.

Benefits Compared to Normal SGD

Scalability:

Normal SGD: Runs on a single machine or device, which may become a bottleneck when dealing with large datasets or complex models.

Distributed SGD: By distributing the data and computations across multiple devices, the overall training time can be reduced significantly. Each device handles a portion of the workload, allowing for larger models and datasets to be processed more efficiently.

Speed:

Normal SGD: The training process can be slow on a single device, especially when the dataset is large or when training requires many epochs.

Distributed SGD: Since multiple devices work in parallel, the time taken to complete one epoch is reduced. This is particularly beneficial when training large-scale models on large datasets.

Resource Utilization:

Normal SGD: Utilizes only one device (CPU/GPU), which might not fully leverage the hardware capabilities of modern multi-GPU systems.

Distributed SGD: Efficiently utilizes all available hardware resources (multiple GPUs/CPU), leading to better resource utilization and potentially faster convergence.

Handling Large Datasets:

Normal SGD: Processing large datasets on a single device can be memory-intensive and may lead to out-of-memory errors.

Distributed SGD: The dataset is split across multiple devices, reducing the memory load on each device and enabling the processing of larger datasets.

Linear Regression

Dr. V. Chandra Sekhar

Linear Regression

- Linear Regression is a simple and powerful model for predicting a numeric response from a set of one or more independent variables.
- And although it may seem like linear regression is overlooked in modern machine learning's ever-increasing world of complex neural network architectures.
- The algorithm is still widely used across a large number of domains because it is **effective**, **easy to interpret**, and **easy to extend**.
- The key ideas in linear regression are recycled everywhere, so understanding the algorithm is a must-have for a strong foundation in machine learning.

Linear Regression

Let's Be More Specific

Linear regression is a supervised algorithm^[i] that learns to model a dependent variable, y , as a function of some independent variables (aka "features"), x_i , by finding a line (or surface) that best "fits" the data. In general, we assume y to be some number and each x_i can be basically anything. For example: predicting the price of a house using the number of rooms in that house (y : price, x_1 : number of rooms) or predicting weight from height and age (y : weight, x_1 : height, x_2 : age).

In general, the equation for linear regression is

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$$

Linear Regression

In general, the equation for linear regression is

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$$

where:

y : the dependent variable; the thing we are trying to predict. [\[i\]](#)

x_i : the independent variables: the features our model uses to model y . [\[i\]](#)

β_i : the coefficients (aka "weights") of our regression model. These are the foundations of our model. They are what our model "learns" during optimization. [\[i\]](#)

ϵ : the irreducible error in our model. A term that collects together all the unmodeled parts of our data.

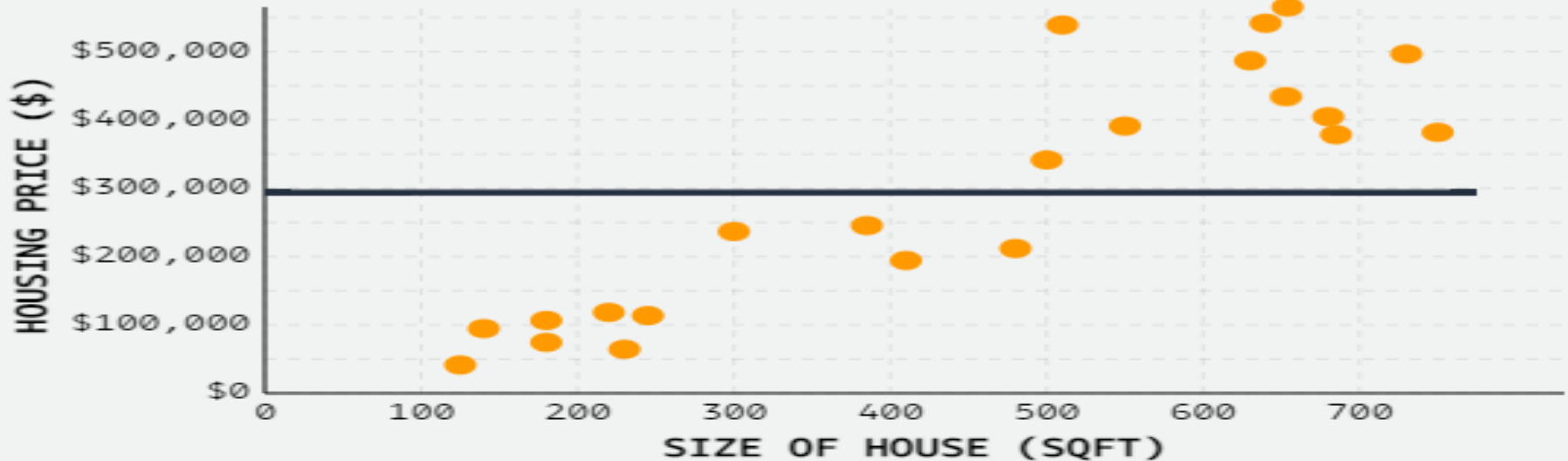
Linear Regression

Fitting a linear regression model is all about finding the set of coefficients that best model y as a function of our features. We may never know the true parameters for our model, but we can estimate them (more on this later). Once we've estimated these coefficients, $\hat{\beta}_i$, we predict future values, $\hat{y}$, as:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_p x_p$$

So predicting future values (often called inference), is as simple as plugging the values of our features x_i into our equation!

Linear Regression



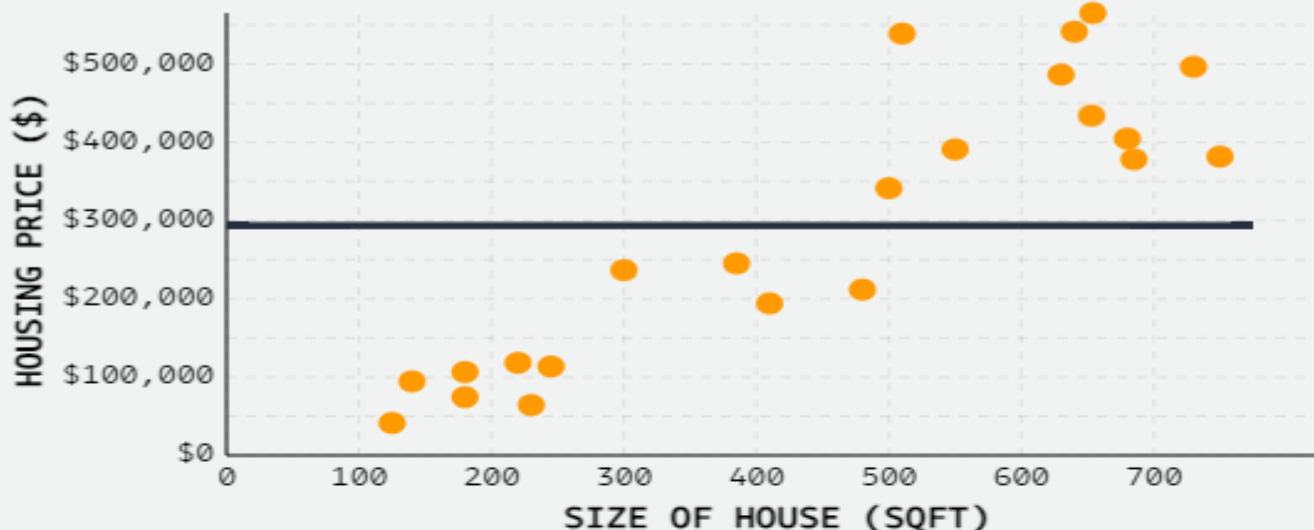
Let's fit a model to predict housing price (\$) in San Diego, USA using the size of the house (in square-footage):

$$\text{house-price} = \hat{\beta}_1 * sqft + \hat{\beta}_0$$

We'll start with a very simple model, predicting the price of each house to be just the average house price in our dataset, ~\$290,000, ignoring the different sizes of each house:

$$\text{house-price} = 0 * sqft + 290000$$

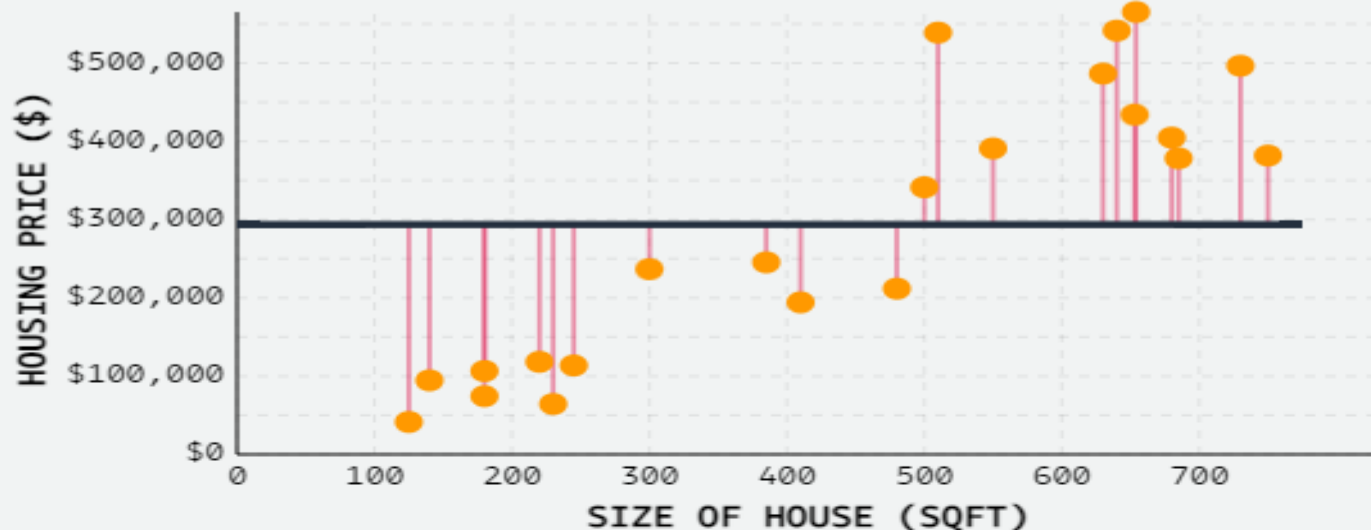
Linear Regression



Of course we know this model is bad - the model doesn't fit the data well at all. But how can we quantify exactly *how* bad?

To evaluate our model's performance quantitatively, we plot the error of each observation directly. These errors, or **residuals**, measure the distance between each observation and the predicted value for that observation. We'll make use of these residuals later when we talk about evaluating regression models, but we can clearly see that our model has a lot of error.

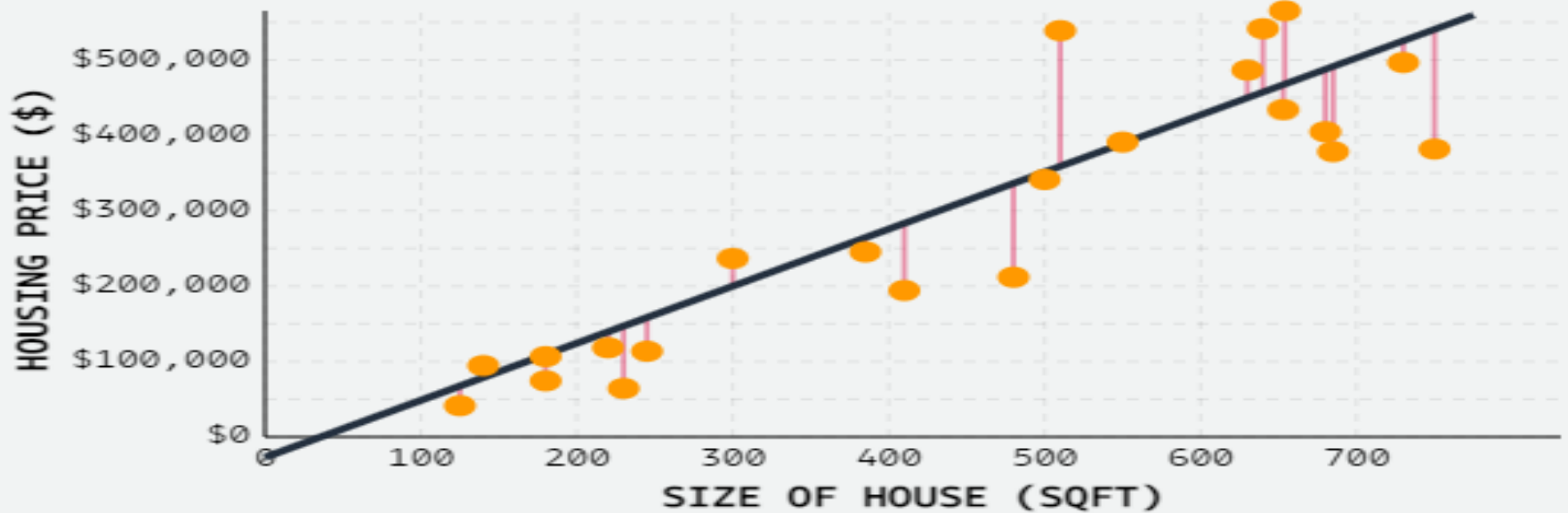
Linear Regression



Of course we know this model is bad - the model doesn't fit the data well at all. But how can we quantify exactly *how* bad?

To evaluate our model's performance quantitatively, we plot the error of each observation directly. These errors, or **residuals**, measure the distance between each observation and the predicted value for that observation. We'll make use of these residuals later when we talk about evaluating regression models, but we can clearly see that our model has a lot of error.

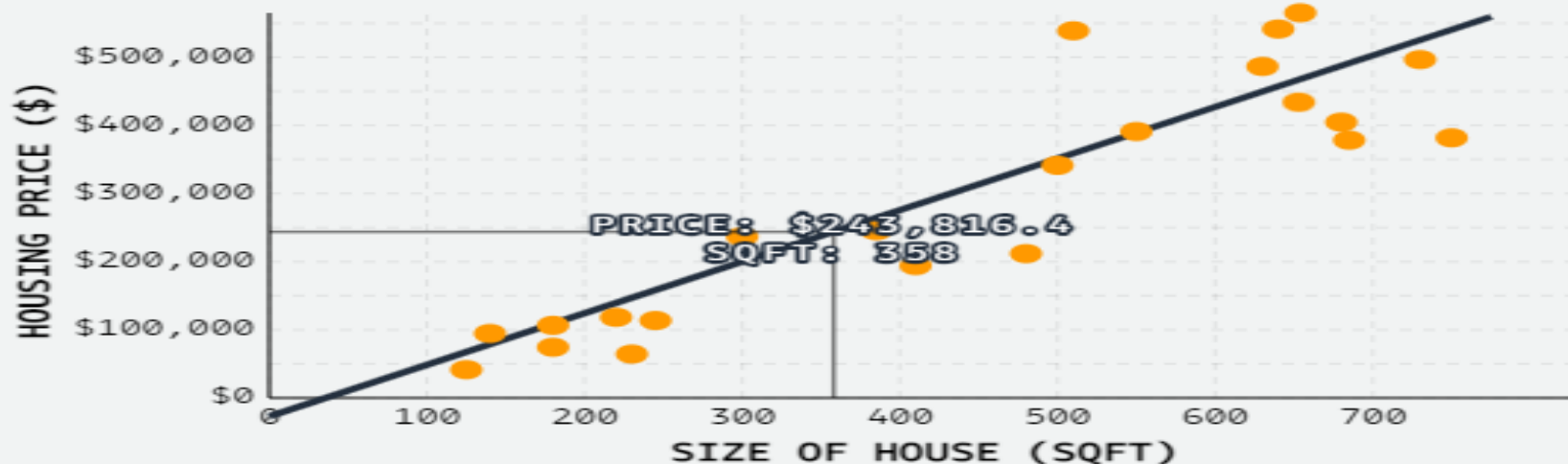
Linear Regression



The goal of linear regression is reducing this error such that we find a line/surface that 'best' fits our data. For our simple regression problem, that involves estimating the y-intercept and slope of our model, $\hat{\beta}_0$ and $\hat{\beta}_1$.

For our specific problem, the best fit line is shown. There's still error, sure, but the general pattern is captured well. As a result, we can be reasonably confident that if we plug in new values of square-footage, our predicted values of price would be reasonably accurate.

Linear Regression



Once we've fit our model, predicting future values is super easy! We just plug in any x_i values into our equation!

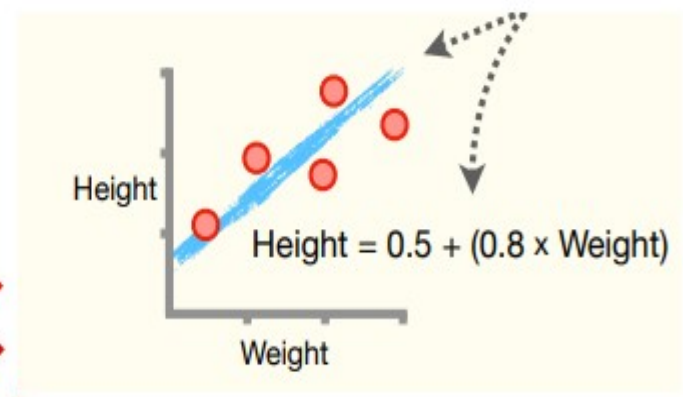
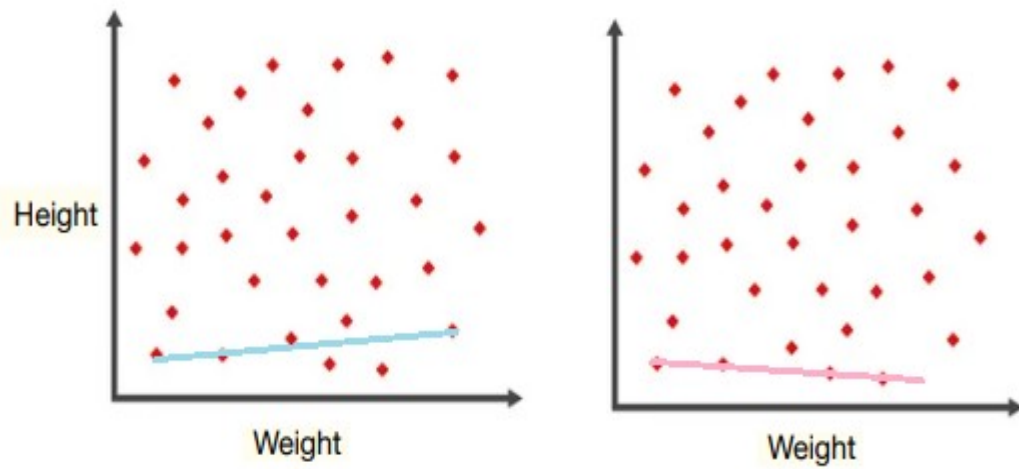
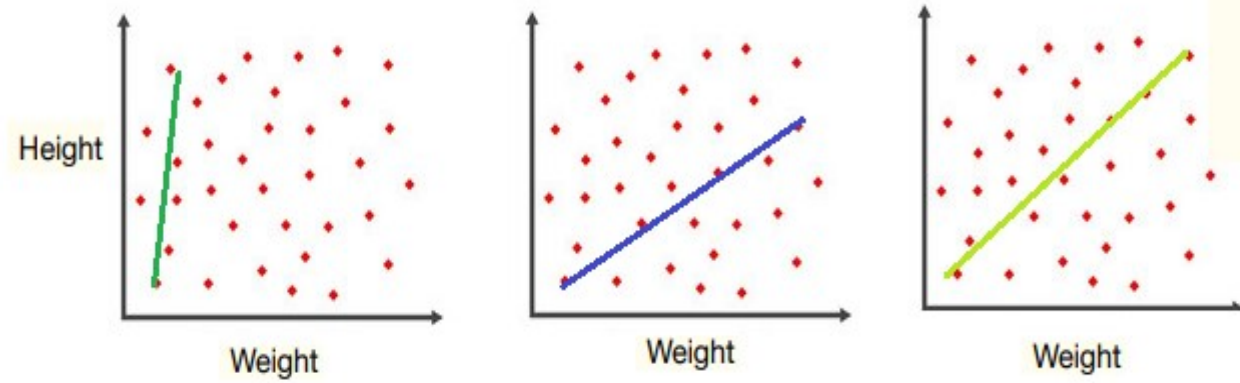
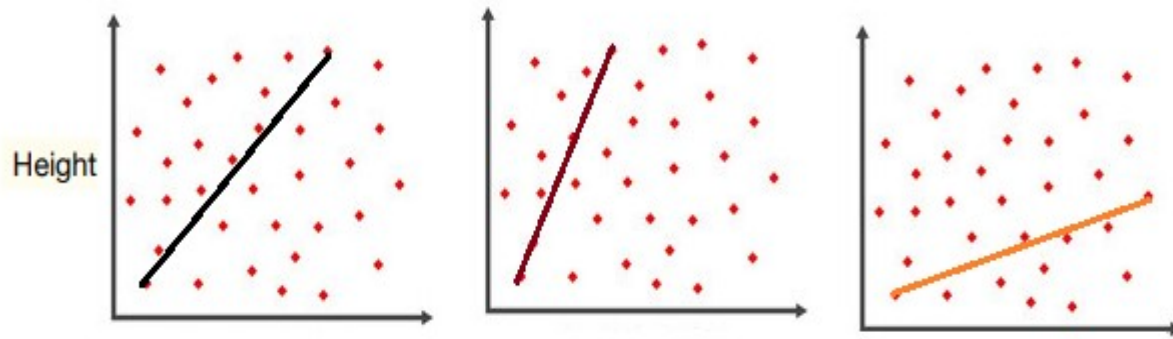
For our simple model, that means plugging in a value for *sqft* into our model:

sqft Value: 358

$$\hat{y} = 756.9 * 358 - 27153.8$$

$$\hat{y} = 243816$$

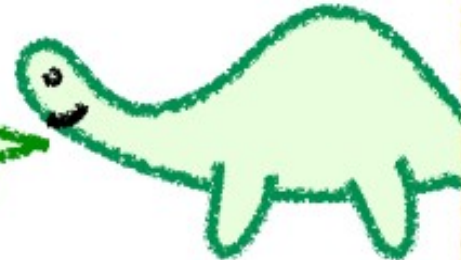
Thus, our model predicts a house that is 358 square-feet will cost \$243,816.



Terminology Alert!!! Probability vs. Likelihood: Part 1



Hey **Norm**, don't the words **Probability** and **Likelihood** mean the same thing?



In casual conversations, yes, we can use **Probability** and **Likelihood** interchangeably. But unfortunately, in the context of statistics, these terms have different uses and, in some cases, completely different meanings.

1 Way back in **Chapter 3**, when we described the **Normal Distribution**, we saw that the y-axis represented **Likelihood**.

In this specific example, the y-axis represents the **Likelihood** of observing any specific Height.

More Likely

Less Likely

The **Normal Distribution's** maximum likelihood value occurs at its mean.

Shorter

Average Height

Taller

For example, it's relatively rare to see someone who is super short...

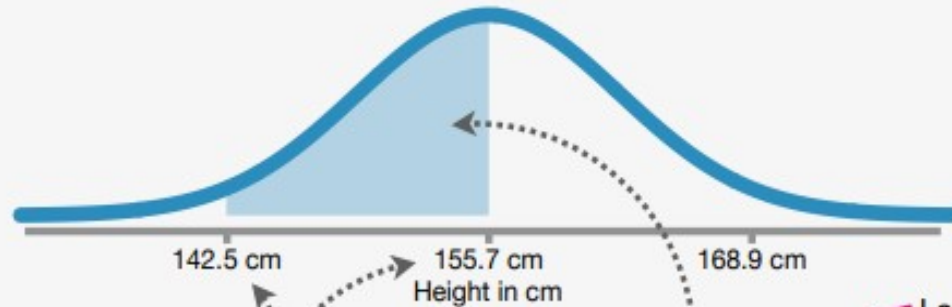
...relatively common to see someone who is close to the average height...

...and relatively rare to see someone who is super tall.

Terminology Alert!!! Probability vs. Likelihood: Part 2

2

In contrast, later in **Chapter 3**, we saw that **Probabilities** are derived from a **Normal Distribution** by calculating the **area under the curve** between two points.



For example, given this **Normal Distribution** with **mean = 155.7** and **standard deviation = 6.6**, the probability of getting a measurement between **142.5** and **155.7** cm...

...is equal to this area under the curve, which, in this example, is **0.48**. So, the probability is **0.48** that we will measure someone in this range.

Lastly, in **Chapter 3** we mentioned that when we use a **Continuous Distribution**, like the **Normal Distribution**, the probability of getting any specific measurement is always **0** because the area of something with no width is **0**.

3

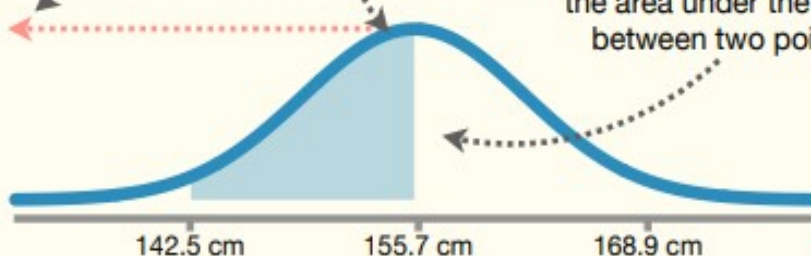
So, in the case of the **Normal Distribution**...

...**Likelihoods** are the y-axis coordinates for specific points on the curve...

...whereas **Probabilities** are the area under the curve between two points.

More Likely

Less Likely



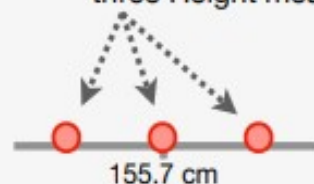
I think I understand **Probabilities**, but what do we do with **Likelihoods**?

We'll talk about that on the next page.

Terminology Alert!!! Probability vs. Likelihood: Part 3

4

Likelihoods are often used to evaluate how well a statistical distribution fits a dataset. For example, imagine we collected these three Height measurements...



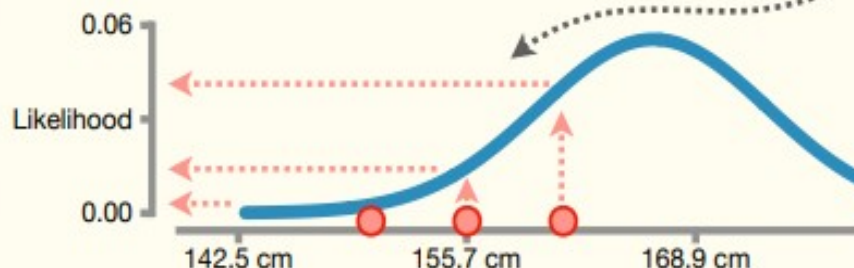
...and we wanted to compare the fit of a **Normal Curve** that has its peak to the right of the data...

...to the fit of a **Normal Curve** that is centered over the data.



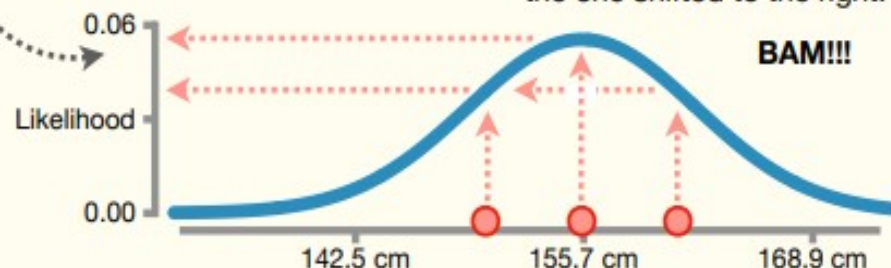
5

First, we determine the **Likelihoods**, the y-axis coordinates on the curves, for each data point...



...and, by eye, we can see that, overall, the **Likelihoods** are larger when we center the curve over the data.

And larger likelihoods suggest that the centered curve fits the data better than the one shifted to the right.



6

NOTE: When we try to fit a **Normal Curve** to data, we can't use **Probabilities** because, as we saw in **Chapter 3**, the probability for a specific point under a **Normal Curve** is always 0.

7

Lastly, as we just saw with the **Normal Distribution**, **Probabilities** and **Likelihoods** can be different. However, as we'll soon see, this is not always the case. In other words, sometimes **Probabilities** and **Likelihoods** are the same.

When **Probabilities** and **Likelihoods** are the same, we could use either **Probabilities** or **Likelihoods** to fit a curve or a squiggle to data. However, to make the terminology consistent, when we're fitting a curve or squiggle to data in a statistical context, we almost always use **Likelihoods**.

Now that we know how to use **Likelihoods** to fit curves, let's talk about the main ideas of fitting a squiggle to data for **Logistic Regression**.



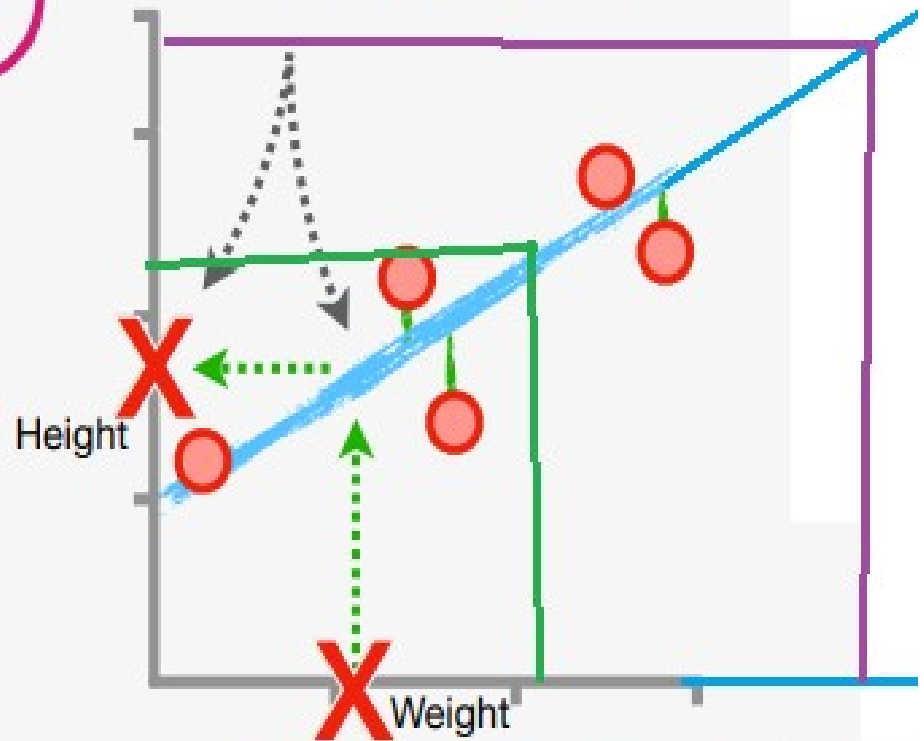
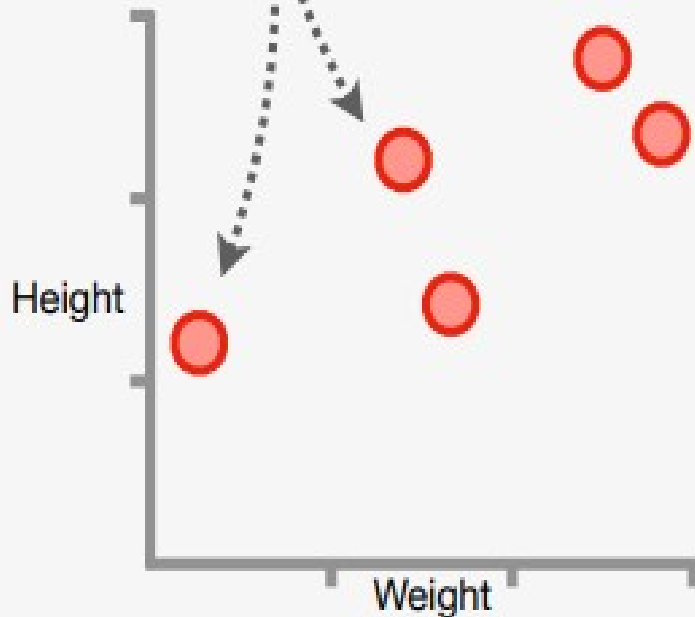
Linear Regression!!!

Linear Regression:

1

The Problem: We've collected Weight and Height measurements from 5 people, and we want to use Weight to predict Height, which is *continuous*...

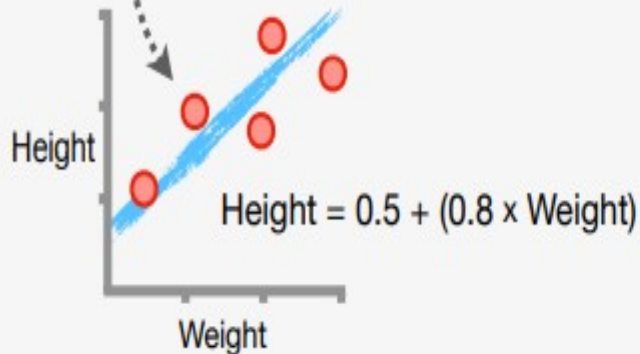
...and in **Chapter 3**, we learned that we could fit a **line** to the data and use it to make predictions.



Models: Main Ideas

3

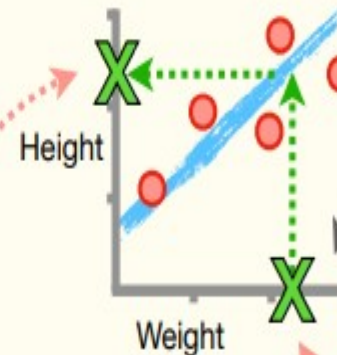
As we saw in **Chapter 1**, models need **Training Data**. Using machine learning lingo, we say that we *build models by training machine learning algorithms*.



4

Models, or equations, can tell us about people we haven't measured yet.

For example, if we want to know how tall someone is who weighs this much...



...we plug the Weight into the equation and solve for Height...

$$\text{Height} = 0.5 + (0.8 \times \text{Weight})$$

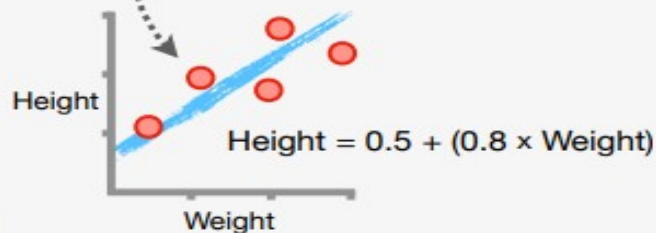
$$\text{Height} = 0.5 + (0.8 \times 2.1)$$

$$\text{Height} = 2.18$$

...and get **2.18**.

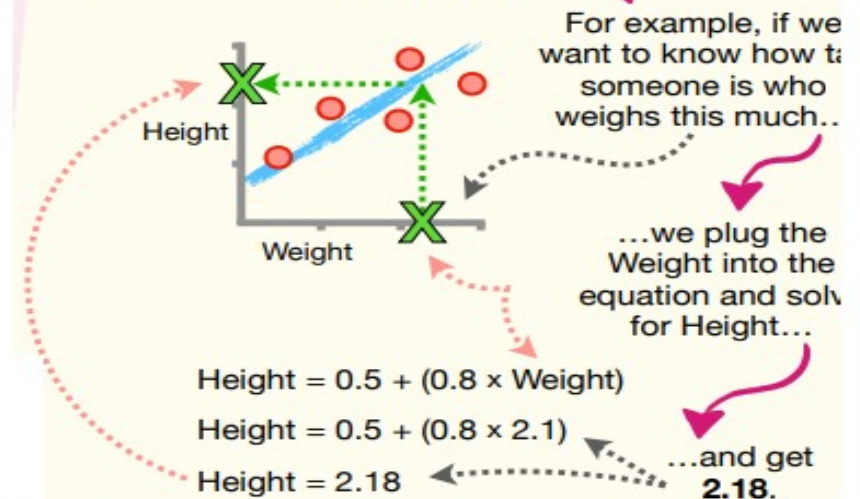
3

As we saw in **Chapter 1**, models need **Training Data**. Using machine learning lingo, we say that we *build models by training machine learning algorithms*.



4

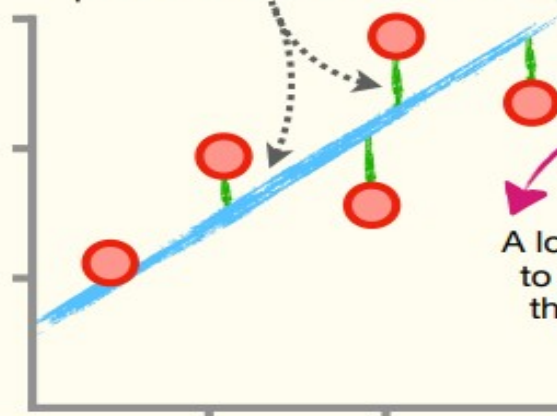
Models, or equations, can tell us about people we haven't measured yet.



5

Because models are only approximations, it's important that we're able to measure the quality of their predictions.

These **green lines** show the distances from the model's predictions to the actual data points.



A lot of statistics is dedicated to quantifying the quality of the predictions made by a model.

6

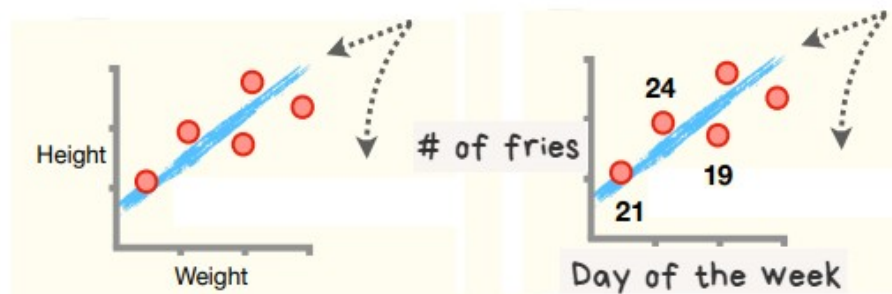
In summary:

- 1) Models approximate reality to let us explore relationships and make predictions.
- 2) In machine learning, we build models by training machine learning algorithms with **Training Data**.
- 3) Statistics can be used to determine if a model is useful or believable.

Bam!

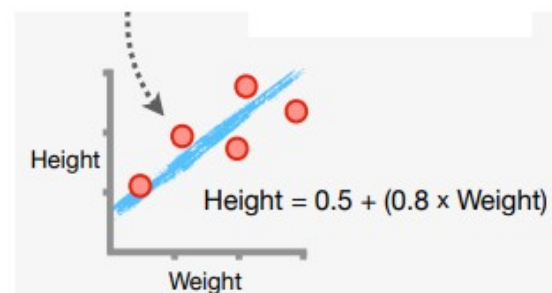
Now let's talk about how statistics can quantify the quality of a model. The first step is to learn about the **Sum of the Squared Residuals**, which is something we'll use throughout this book.

① Train Data (seen data)



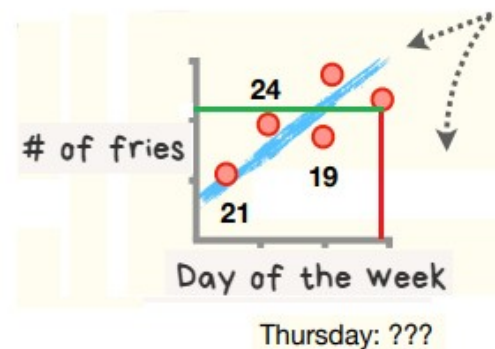
②

Curve Fitting (Model Development) (Function Learning)



④

Testing / Inference (Unseen or Unknown data)



③ Evaluate the model

Now let's talk about how statistics can quantify the quality of a model.



Fry Diary

Monday: 21 fries

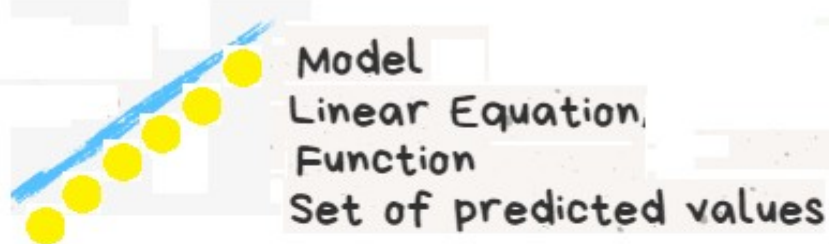
Tuesday: 24 fries

Wednesday: 19 fries

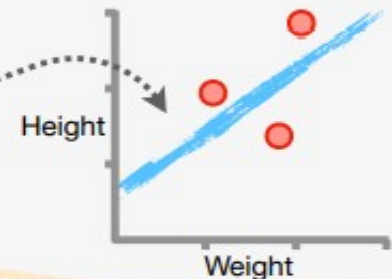
Thursday: ???

The Sum of the Squared Residuals:

● Observed
● Predicted



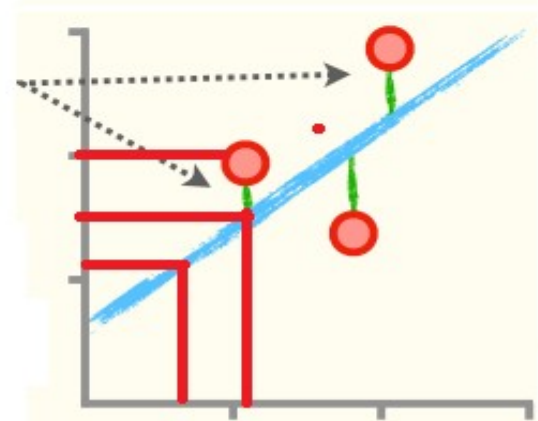
- 1 **The Problem:** We have a model that makes predictions. In this case, we're using Weight to predict Height. However, we need to quantify the quality of the model and its predictions.



- 2 **A Solution:** One way to quantify the quality of a model and its predictions is to calculate the **Sum of the Squared Residuals**.

As the name implies, we start by calculating **Residuals**, the differences between the **Observed** values and the values **Predicted** by the model.

$$\text{Residual} = \text{Observed} - \text{Predicted} \text{ (Error)}$$



The Sum of Squared Residuals (SSR)

is usually defined with fancy **Sigma** notation and the right-hand side reads: "The sum of all observations of the squared difference between the observed and predicted values."

$$SSR = \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$$

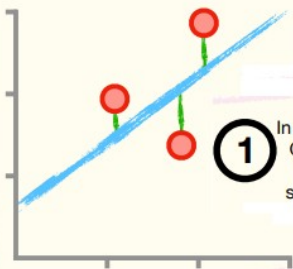
n = the number of **Observations**.

i = the index for each **Observation**. For example, $i = 1$ refers to the first **Observation**.

The **Sigma** symbol, Σ , tells us to do a **summation**.

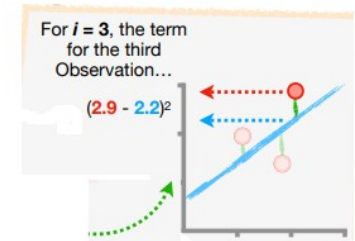
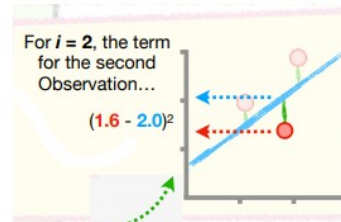
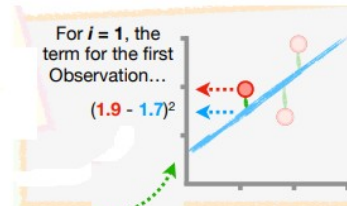
So, instead of calculating the sum of the **Residuals**, we square the **Residuals** first and calculate the **Sum of the Squared Residuals (SSR)**.

SSR: Step-by-Step



1 In this example, we have 3 Observations, so $n = 3$, and we expand the summation into 3 terms.

$$\text{SSR} = (1.9 - 1.7)^2 + (1.6 - 2.0)^2 + (2.9 - 2.2)^2 = 0.69$$

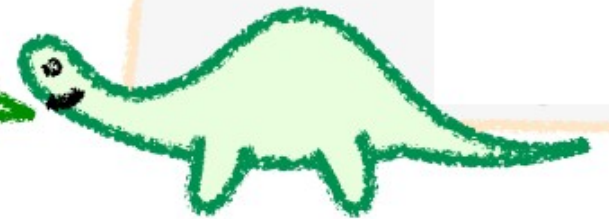


The Sum of Squared Residuals (SSR) = $\sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$

The Sum of Squared Residuals (SSR) $= \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$

BAM!!!

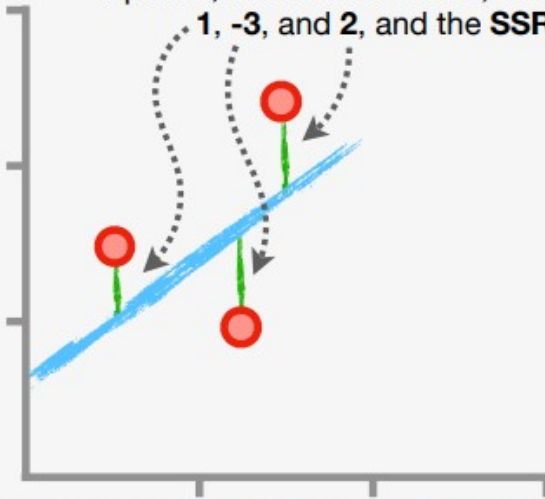
Don't get me wrong, the **SSR** is awesome, but it has a pretty big problem that we'll talk about on the next page.



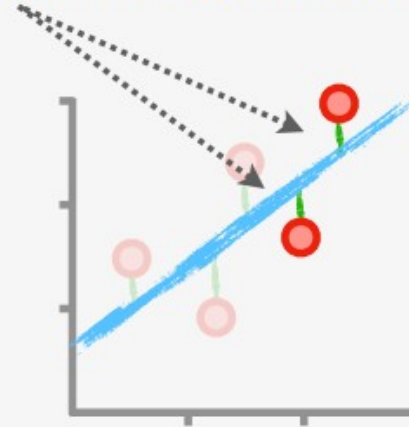
1

The Problem: Sum of the Squared Residuals (SSR), although awesome, is not super easy to interpret because it depends, in part, on how much data you have.

For example, if we start with a simple dataset with 3 points, the **Residuals** are, from left to right, 1, -3, and 2, and the **SSR = 14**.



Now, if we have a second dataset that includes 2 more data points added to the first one, and the **Residuals** are -2 and 2, then the **SSR increases to 22**.



However, the increase in the **SSR** from 14 to 22 does *not* suggest that the second model, fit to the second, larger dataset, is worse than the first. It only tells us that the model with more data has more **Residuals**.

$$\text{Mean Squared Error (MSE)} = \frac{\text{The Sum of Squared Residuals (SSR)}}{\text{Number of Observations, } n} = \sum_{i=1}^n \frac{(\text{Observed}_i - \text{Predicted}_i)^2}{n}$$

Mean Squared Error (MSE):

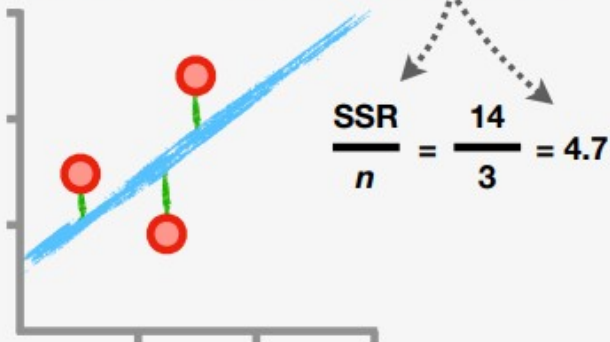
1

Now let's see the **MSE** in action by calculating it for the two datasets!!!

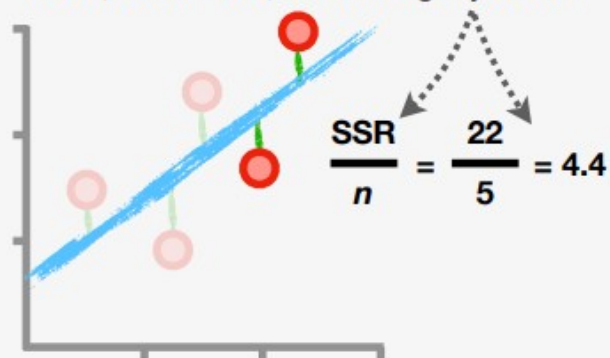
$$\text{Mean Squared Error (MSE)} = \frac{\text{SSR}}{n} = \sum_{i=1}^n \frac{(\text{Observed}_i - \text{Predicted}_i)^2}{n}$$

2

The first dataset has only 3 points and the **SSR** = 14, so the **Mean Squared Error (MSE)** is $14/3 = 4.7$.



The second dataset has 5 points and the **SSR** increases to 22. In contrast, the **MSE**, $22/5 = 4.4$, is now slightly lower.

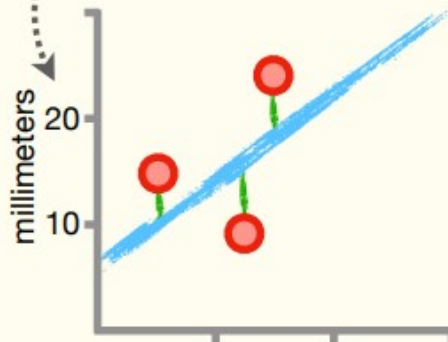


So, unlike the **SSR**, which increases when we add more data to the model, the **MSE** can increase or decrease depending on the average residual, which gives us a better sense of how the model is performing overall.

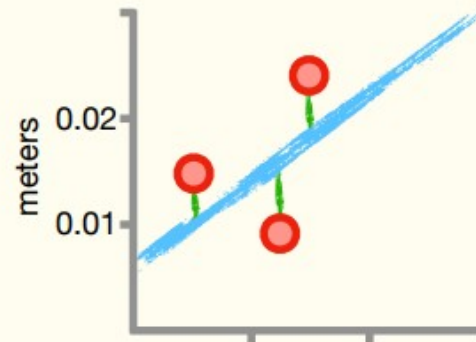
3

Unfortunately, **MSEs** are still difficult to interpret on their own because the maximum values depend on the scale of the data.

For example, if the y-axis is in *millimeters* and the **Residuals** are 1, -3, and 2, then the **MSE** = 4.7.



However, if we change the y-axis to *meters*, then the **Residuals** for the exact same data shrink to 0.001, -0.003, and 0.002, and the **MSE** is now 0.0000047. It's tiny!

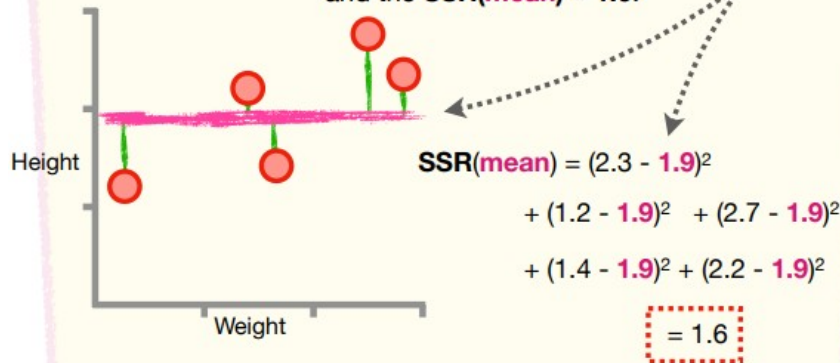


The good news, however, is that both the **SSR** and the **MSE** can be used to calculate something called R^2 , which is independent of both the size of the dataset and the scale, so keep reading!

R²: Details Part 1

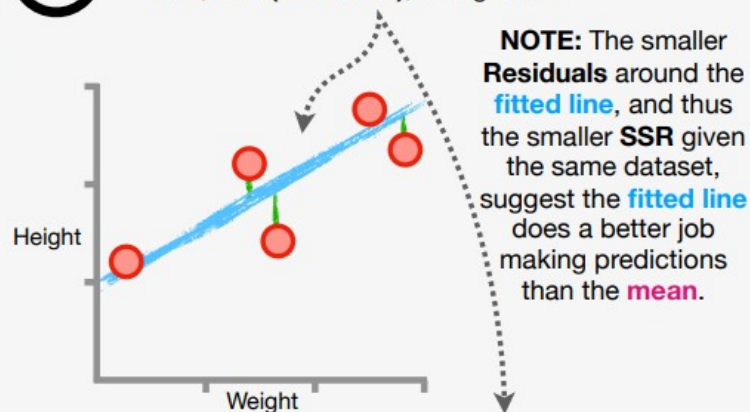
1

First, we calculate the **Sum of the Squared Residuals** for the **mean**. We'll call this **SSR** the **SSR(mean)**. In this example, the mean Height is **1.9** and the **SSR(mean) = 1.6**.



2

Then, we calculate the **SSR** for the **fitted line**, **SSR(fitted line)**, and get **0.5**.



$$SSR(\text{fitted line}) = (1.2 - 1.1)^2 + (2.2 - 1.8)^2 + (1.4 - 1.9)^2$$

$$+ (2.7 - 2.4)^2 + (2.3 - 2.5)^2$$

$$= 0.5$$

3

Now we can calculate the **R²** value using a surprisingly simple formula...

$$R^2 = \frac{SSR(\text{mean}) - SSR(\text{fitted line})}{SSR(\text{mean})}$$

$$= \frac{1.6 - 0.5}{1.6}$$

$$= 0.7$$

...and the result, **0.7**, tells us that there was a **70%** reduction in the size of the **Residuals** between the **mean** and the **fitted line**.

4

In general, because the numerator for **R²**...

$$SSR(\text{mean}) - SSR(\text{fitted line})$$

...is the amount by which the **SSRs** shrank when we fitted the line, **R²** values tell us the percentage the **Residuals** around the **mean** shrank when we used the **fitted line**.

When **SSR(mean) = SSR(fitted line)**, then both models' predictions are equally good (or equally bad), and **R² = 0**

$$\frac{SSR(\text{mean}) - SSR(\text{fitted line})}{SSR(\text{mean})}$$

$$= \frac{0}{SSR(\text{mean})} = 0$$

When **SSR(fitted line) = 0**, meaning that the **fitted line** fits the data perfectly, then **R² = 1**.

$$\frac{SSR(\text{mean}) - 0}{SSR(\text{mean})} = \frac{SSR(\text{mean})}{SSR(\text{mean})} = 1$$

Gentle Reminders:

Residual = Observed - Predicted

SSR = Sum of Squared Residuals

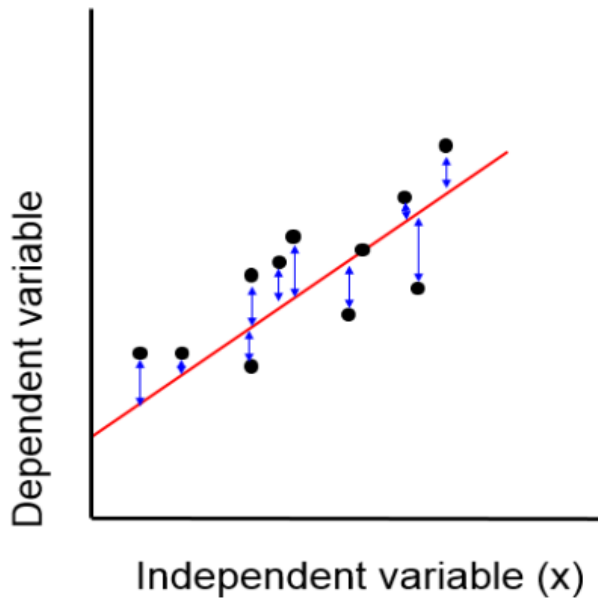
$$\mathbf{SSR} = \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2$$

$$\mathbf{\text{Mean Squared Error (MSE)}} = \frac{\mathbf{SSR}}{\mathbf{n}}$$

...where n is the sample size

$$\mathbf{R^2} = \frac{\mathbf{SSR(\text{mean})} - \mathbf{SSR(\text{fitted line})}}{\mathbf{SSR(\text{mean})}}$$

Linear Regression



SSR (Sum of Squares Regression) $SSR = \sum (\hat{y} - \bar{y})^2$ (measure of explained variation)

- **Definition:** SSR, also known as the sum of squares due to regression or explained sum of squares, measures the amount of variance in the dependent variable that is explained by the independent variables in the model.
- **Formula:** $SSR = \sum (\hat{y} - \bar{y})^2$
- **Components:**
 - $\hat{y}$ represents the predicted values of the dependent variable, obtained from the regression model.
 - $\bar{y}$ is the mean of the observed values of the dependent variable.

SSE (Sum of Squares Error) $SSE = \sum (y - \hat{y})^2$ (measure of unexplained variation)

- **Definition:** SSE, also known as the sum of squares of residuals or the unexplained sum of squares, quantifies the variance in the dependent variable that the model does not explain.
- **Formula:** $SSE = \sum (y - \hat{y})^2$
- **Components:**
 - y is the actual observed values of the dependent variable.
 - $\hat{y}$ again represents the predicted values from the model.

SST (Sum of Squares Total) $SST = SSR + SSE = \sum (y - \bar{y})^2$ (measure of total variation in y)

- **Definition:** SST represents the total variance in the dependent variable. It is the sum of the variances explained and unexplained by the model.
- **Formula:** $SST = SSR + SSE = \sum (y - \bar{y})^2$
- **Components:**
 - y is the observed values of the dependent variable.
 - $\bar{y}$ is the mean of these observed values.

Linear Regression

Components of the Equation

1. R^2 : This is the R-squared statistic, which represents the proportion of the variance in the dependent variable that is predictable from the independent variables.
2. y_i : These are the actual observed values of the dependent variable.
3. $\hat{y}_i$: These are the predicted values of the dependent variable based on the regression model.
4. $\bar{y}$: This is the mean (average) of the observed values of the dependent variable.

Breaking Down the Equation

- **Numerator** $(\sum_{i=1}^n (y_i - \hat{y}_i)^2)$: This part of the formula represents the Sum of Squares of Residuals (SSE), which quantifies the variance in the dependent variable that is not explained by the model. It is the sum of the squared differences between each observed value and its corresponding predicted value.
- **Denominator** $(\sum_{i=1}^n (y_i - \bar{y})^2)$: This part represents the Total Sum of Squares (SST), which quantifies the total variance in the dependent variable. It is the sum of the squared differences between each observed value and the mean of all observed values.

Linear Regression

The formula for R^2 is structured to provide a scale from 0 to 1, where:

- $R^2 = 1$: Indicates a perfect fit. All data points lie exactly on the regression line, meaning all variance in the dependent variable is explained by the model. The SSE is 0 because each y_i equals $\hat{y}_i$.
- $R^2 = 0$: Indicates no explanatory power. The regression model does not explain any of the variance in the dependent variable; the predictions $\hat{y}_i$ are as inaccurate as just using the mean $\bar{y}$ for every prediction. This happens when the SSE equals the SST.
- **Values between 0 and 1** indicate the proportion of the total variance in the dependent variable that is explained by the independent variables in the model.

Learning The Coefficients

- Let's recap what we've learned so far: Linear regression is all about finding a line (or surface) that fits our data well.
- And as we just saw, this involves **selecting the coefficients** for our model that **minimize** our evaluation metric.
- But how can we best **estimate these coefficients**? In practice, they're unknown, and selecting them by hand quickly becomes infeasible for regression models with many features.

Linear Regression

The gradient in the context of mathematics and optimization is a vector that indicates the direction and rate of the steepest increase of a function. It plays a crucial role in various optimization techniques, especially in gradient descent, where it helps determine how parameters should be adjusted to minimize a function such as a loss function in machine learning.

Characteristics of the Gradient

1. **Direction:** The gradient of a function at any given point is directed toward the direction of greatest increase of the function. In terms of topography, you can think of it as pointing uphill.
2. **Magnitude:** The magnitude (or length) of the gradient vector represents the rate of increase of the function. A larger magnitude means the function increases more steeply in that direction.

Implications in Optimization

- **Descent Direction:** In optimization, particularly in the method known as gradient descent, the gradient is used to determine the direction in which the function increases most rapidly. Since the objective in such cases is usually to find the minimum value, the process involves moving in the opposite direction of the gradient—thus the term "descent".
- **Step Size:** The magnitude of the gradient often influences the step size. If the gradient is large, the function's surface is steep, suggesting more significant changes to the parameter values might be necessary.

Linear Regression

Gradient Descent

In gradient descent, the gradient is calculated for a loss function with respect to its parameters, and adjustments are made to the parameters in the direction that most reduces the loss (i.e., moving downhill on the loss surface):

$$\theta := \theta - \alpha \nabla f(\theta)$$

Here:

- θ represents the parameters.
- α is the learning rate, controlling the size of the step.
- $\nabla f(\theta)$ is the gradient of the loss function with respect to θ .

Linear Regression

Components of Linear Regression

In simple linear regression, the model predicts the outcome based on a linear relationship between the input variable x and the output variable y . This relationship is defined by:

$$y = \beta_0 + \beta_1 x$$

where:

- β_0 is the intercept,
- β_1 is the slope coefficient.

Loss Function: Mean Squared Error (MSE)

The MSE is used to measure the quality of a regressor. It is calculated as the average of the squares of the differences between the predicted and actual values:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- y_i are the actual values,
- $\hat{y}_i = \beta_0 + \beta_1 x_i$ are the predicted values,
- n is the number of observations.

Linear Regression

Derivatives of the MSE

To use Gradient Descent, we need to find the partial derivatives of the MSE with respect to each coefficient (β_0 and β_1). These derivatives give us the gradients.

1. Gradient with respect to β_0

$$\frac{\partial}{\partial \beta_0} MSE = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Simplifies to:

$$\frac{\partial}{\partial \beta_0} MSE = \frac{2}{n} \sum_{i=1}^n (\beta_0 + \beta_1 x_i - y_i)$$

2. Gradient with respect to β_1

$$\frac{\partial}{\partial \beta_1} MSE = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

Simplifies to:

$$\frac{\partial}{\partial \beta_1} MSE = \frac{2}{n} \sum_{i=1}^n (\beta_0 + \beta_1 x_i - y_i) x_i$$

Applying Gradient Descent

Using these gradients, Gradient Descent updates the coefficients as follows in each iteration:

$$\begin{aligned}\beta_0 &:= \beta_0 - \alpha \left(\frac{\partial}{\partial \beta_0} MSE \right) \\ \beta_1 &:= \beta_1 - \alpha \left(\frac{\partial}{\partial \beta_1} MSE \right)\end{aligned}$$

where α is the learning rate, a hyperparameter that determines the size of the steps taken towards the minimum.

Linear Regression : Gradient Descent

Gradient descent is an iterative optimization algorithm that estimates some set of coefficients to yield the minimum of a convex function. Put simply: it will find suitable coefficients for our regression model that minimize prediction error (remember, lower MSE equals better model).

Gradient descent works as follows. We assume that we have some convex function representing the error of our machine learning algorithm (in our case, MSE). Gradient descent will iteratively update our model's coefficients in the direction of our error functions minimum **[i]** .

In our case, our model takes the form:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1$$

and our error function takes the form:

$$\begin{aligned} MSE(\hat{\beta}_0, \hat{\beta}_1) &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_1))^2 \end{aligned}$$

Linear Regression

Example:

house price = $-39591 + 742 * sqft$

Interpretation: This model summarizes the average house prices across differently sized houses (●) as measured in square feet.

The coefficient, \$742, represents the average difference in housing price for one-unit difference in the square-footage of the house. In other words, we expect each additional square-foot, on average, to raise the price of a house by \$742.

Logistic Regression

- Logistic regression is a statistical method for analyzing datasets in which there are one or more independent variables that determine an outcome.
- The outcome is a binary variable, meaning it has two possible outcomes.
- The model predicts the probability of a certain class or event, such as pass/fail, win/lose, alive/dead, etc

Logistic Regression

Mathematical Formulation

1. Logistic Function

The core of logistic regression is the logistic function, also known as the sigmoid function, which maps any real-valued number into the range (0, 1):

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

where z is the input to the function.

2. Linear Combination of Inputs

In logistic regression, the input z to the logistic function is a linear combination of the independent variables (features):

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Here,

- β_0 is the intercept term.
- $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients (weights) for the features $x_1, x_2, \dots, x_n$.

Logistic Regression

3. Probability Prediction

The logistic regression model predicts the probability P of the binary outcome as:

$$P(y = 1|x) = \sigma(z) = \frac{1}{1+e^{-(\beta_0+\beta_1x_1+\beta_2x_2+\dots+\beta_nx_n)}}$$

The probability of the other class (i.e., $y = 0$) is:

$$P(y = 0|x) = 1 - P(y = 1|x)$$

Logistic Regression

4. Decision Rule

To make a binary decision, we choose a threshold (commonly 0.5):

$$\begin{cases} 1 & \text{if } P(y = 1|x) \geq 0.5 \\ 0 & \text{if } P(y = 1|x) < 0.5 \end{cases}$$

5. Cost Function

To estimate the coefficients $\beta_0, \beta_1, \dots, \beta_n$, we use maximum likelihood estimation. The cost function (log loss or binary cross-entropy loss) for logistic regression is:

$$\mathcal{L}(\beta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(P(y_i|x_i)) + (1 - y_i) \log(1 - P(y_i|x_i))]$$

where m is the number of training examples.

Logistic Regression

Example

Let's consider a simple example with a single feature x :

Data

Suppose we have a dataset of student scores and whether they passed (1) or failed (0):

Score (x)	Pass (y)
50	0
60	0
70	0
80	1
90	1

Logistic Regression

Step-by-Step Calculation

1. Linear Combination:

Let's assume the weights are $\beta_0 = -5$ and $\beta_1 = 0.1$. For a student with a score of 75:

$$z = \beta_0 + \beta_1 x = -5 + 0.1 \times 75 = 2.5$$

2. Logistic Function:

The probability that the student passes:

$$P(y = 1|x = 75) = \sigma(2.5) = \frac{1}{1+e^{-2.5}} \approx 0.924$$

So, the model predicts a 92.4% chance that the student will pass.

3. Decision Rule:

If we use a threshold of 0.5, since $0.924 > 0.5$, the model predicts that the student will pass.

Logistic Regression

Training the Model

To train the model, we would adjust β_0 and β_1 to minimize the cost function. This involves:

1. **Initialization:** Start with initial guesses for β_0 and β_1 .
2. **Prediction:** Use the logistic function to predict probabilities for each example.
3. **Cost Calculation:** Compute the log loss.
4. **Gradient Calculation:** Compute the gradients of the cost with respect to β_0 and β_1 .
5. **Update Weights:** Adjust the weights in the direction that reduces the cost.

This process iterates until the weights converge to values that minimize the cost function, resulting in a model that can make accurate predictions on new data.

Loss function

- The cost function in logistic regression, also known as the log loss or binary cross-entropy loss, measures the performance of a classification model whose output is a probability value between 0 and 1.
- The cost function quantifies the error between the predicted probabilities and the actual binary labels.
- Minimizing this cost function helps to fit the logistic regression model to the training data.

Detailed Explanation of the Cost Function

For logistic regression, the cost function for a single training example is defined as follows:

$$\text{Cost}(h_{\theta}(x), y) = - [y \log(h_{\theta}(x)) + (1 - y) \log(1 - h_{\theta}(x))]$$

where:

- $h_{\theta}(x)$ is the predicted probability that the output is 1 given input x .
- y is the actual label (0 or 1).

Logistic Regression

Derivation and Intuition

Case 1: $y = 1$

When the actual label y is 1, the cost function becomes:

$$\text{Cost}(h_{\theta}(x), 1) = -\log(h_{\theta}(x))$$

If the model predicts $h_{\theta}(x)$ close to 1, $\log(h_{\theta}(x))$ is close to 0, making the cost near zero, which is desirable. However, if the model predicts $h_{\theta}(x)$ close to 0, $\log(h_{\theta}(x))$ becomes a large negative number, and the cost becomes very high, penalizing the model for the incorrect prediction.

Case 2: $y = 0$

When the actual label y is 0, the cost function becomes:

$$\text{Cost}(h_{\theta}(x), 0) = -\log(1 - h_{\theta}(x))$$

If the model predicts $h_{\theta}(x)$ close to 0, $\log(1 - h_{\theta}(x))$ is close to 0, making the cost near zero. However, if the model predicts $h_{\theta}(x)$ close to 1, $\log(1 - h_{\theta}(x))$ becomes a large negative number, and the cost becomes very high, penalizing the model for the incorrect prediction.

Logistic Regression

Combined Cost Function

For a dataset with m training examples, the overall cost function (log loss) is the average of the individual costs across all examples:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))]$$

Here:

- m is the number of training examples.
- x_i is the feature vector of the i -th training example.
- y_i is the actual label of the i -th training example.
- $h_{\theta}(x_i)$ is the predicted probability for the i -th training example.

Summary

- The cost function in logistic regression measures the error between the predicted probabilities and the actual binary labels.
- It penalizes wrong predictions more heavily, encouraging the model to produce probabilities closer to the true labels.
- By minimizing this cost function using optimization techniques like gradient descent, we can fit the logistic regression model to the training data, resulting in accurate predictions on new data.

Summary

1. Case 1: Actual = 1, Predicted = 0

- **Actual Outcome:** The true label is 1 (e.g., the patient has the disease).
- **Predicted Outcome:** The model predicts 0 (e.g., the patient does not have the disease).
- **Cost:** High, because the prediction is incorrect, and the error is significant.

2. Case 2: Actual = 0, Predicted = 1

- **Actual Outcome:** The true label is 0 (e.g., the customer did not buy the product).
- **Predicted Outcome:** The model predicts 1 (e.g., the customer will buy the product).
- **Cost:** High, because the prediction is incorrect, and the error is significant.

Summary

3. Case 3: Actual = 1, Predicted = 1

- **Actual Outcome:** The true label is 1 (e.g., the email is spam).
- **Predicted Outcome:** The model predicts 1 (e.g., the email is spam).
- **Cost:** Low, because the prediction is correct.

4. Case 4: Actual = 0, Predicted = 0

- **Actual Outcome:** The true label is 0 (e.g., the student did not pass the exam).
- **Predicted Outcome:** The model predicts 0 (e.g., the student will not pass the exam).
- **Cost:** Low, because the prediction is correct.

Summary

Cost Function Calculation

Let's calculate the cost for each case using the cost function:

$$\text{Cost}(h_{\theta}(x), y) = -[y \log(h_{\theta}(x)) + (1 - y) \log(1 - h_{\theta}(x))]$$

1. Case 1: Actual = 1, Predicted = 0

- $y = 1$
- $h_{\theta}(x) = 0$ (predicted probability)
- Cost: $-[1 \cdot \log(0) + (1 - 1) \cdot \log(1 - 0)]$
- Since $\log(0)$ is undefined and approaches negative infinity, the cost is very high.

Summary

2. Case 2: Actual = 0, Predicted = 1

- $y = 0$
- $h_{\theta}(x) = 1$ (predicted probability)
- Cost: $-[0 \cdot \log(1) + (1 - 0) \cdot \log(1 - 1)]$
- Since $\log(0)$ is undefined and approaches negative infinity, the cost is very high.

3. Case 3: Actual = 1, Predicted = 1

- $y = 1$
- $h_{\theta}(x) = 1$ (predicted probability)
- Cost: $-[1 \cdot \log(1) + (1 - 1) \cdot \log(1 - 1)]$
- Since $\log(1) = 0$, the cost is $-[1 \cdot 0 + 0 \cdot \text{undefined}] = 0$.

Summary

4. Case 4: Actual = 0, Predicted = 0

- $y = 0$
- $h_{\theta}(x) = 0$ (predicted probability)
- Cost: $-[0 \cdot \log(0) + (1 - 0) \cdot \log(1 - 0)]$
- Since $\log(1) = 0$, the cost is $-[0 \cdot \text{undefined} + 1 \cdot 0] = 0$.

Summary

- **High Cost:** When the prediction is significantly incorrect (e.g., actual = 1, predicted = 0 or actual = 0, predicted = 1).
- **Low Cost:** When the prediction is correct (e.g., actual = 1, predicted = 1 or actual = 0, predicted = 0).

Polynomial Regression

1. Polynomial Regression

Polynomial regression is a form of linear regression in which the relationship between the independent variable x and the dependent variable y is modeled as an n th degree polynomial. Polynomial regression is used when data points show a non-linear relationship that a simple linear regression (a straight line through the data) cannot capture.

Example: Predicting the speed of a car at different times of the day based on traffic conditions. If the data shows that the speed doesn't decrease linearly as traffic increases, but rather follows a curved path, you might use a quadratic model (second-degree polynomial) like $y = ax^2 + bx + c$ to fit the data.

Linear ————— $ax + b = 0$

Quadratic ————— $ax^2 + bx + c = 0$

Cubic ——— $ax^3 + bx^2 + cx + d = 0$

Polynomial Regression

Linear ————— $ax + b = 0$

Quadratic ————— $ax^2 + bx + c = 0$

Cubic ————— $ax^3 + bx^2 + cx + d = 0$

Simple
Linear
Regression

$$y = b_0 + b_1x_1$$

Multiple
Linear
Regression

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Polynomial
Linear
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

Polynomial Regression

Simple
Linear
Regression

$$y = b_0 + b_1 x_1$$

Multiple
Linear
Regression

$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$$

Polynomial
Linear
Regression

$$y = b_0 + b_1 x_1 + b_2 x_1^2 + \dots + b_n x_1^n$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{bmatrix},$$

Polynomial Regression

Deep Dive into the Loss Function for Polynomial Regression

In polynomial regression, similar to linear regression, we often use the Mean Squared Error (MSE) as the loss function to optimize the model parameters. This is because MSE is simple to compute and differentiable, which makes it suitable for optimization algorithms like gradient descent.

Why MSE in Polynomial Regression?

Polynomial regression, despite its ability to model non-linear relationships, is still considered a form of linear regression because the coefficients (or weights) of the polynomial are linear. The polynomial terms (like x^2 , x^3 , etc.) are treated as independent features.

MSE for Polynomial Regression: The Mean Squared Error measures the average squared difference between the estimated values ($\hat{y}$) predicted by the model and the actual values (y). For a set of n data points and a model predicting outputs as $\hat{y}$ from inputs x , the MSE is defined as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Polynomial Regression

Mathematical Formulation

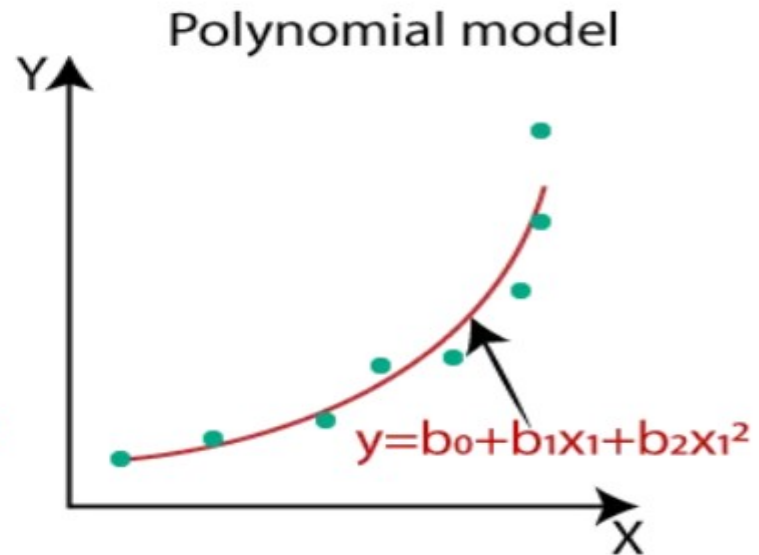
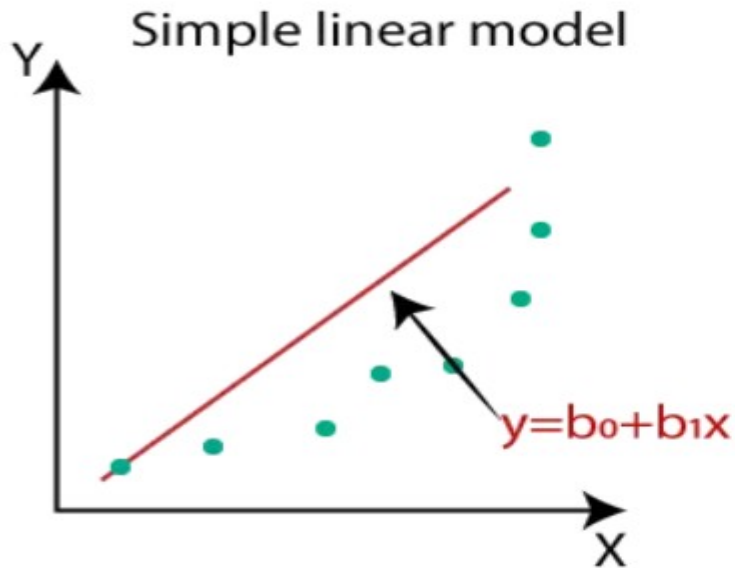
In the context of example where the relationship is modeled as a quadratic polynomial ($y = ax^2 + bx + c$), the MSE would look like this:

- Let's say you have n observations (x_i, y_i) , where x_i is the feature (e.g., traffic conditions) and y_i is the target variable (e.g., car speed).
- Your model predictions based on the polynomial regression would be $\hat{y}_i = ax_i^2 + bx_i + c$ for each observation i .

The MSE can be calculated as:

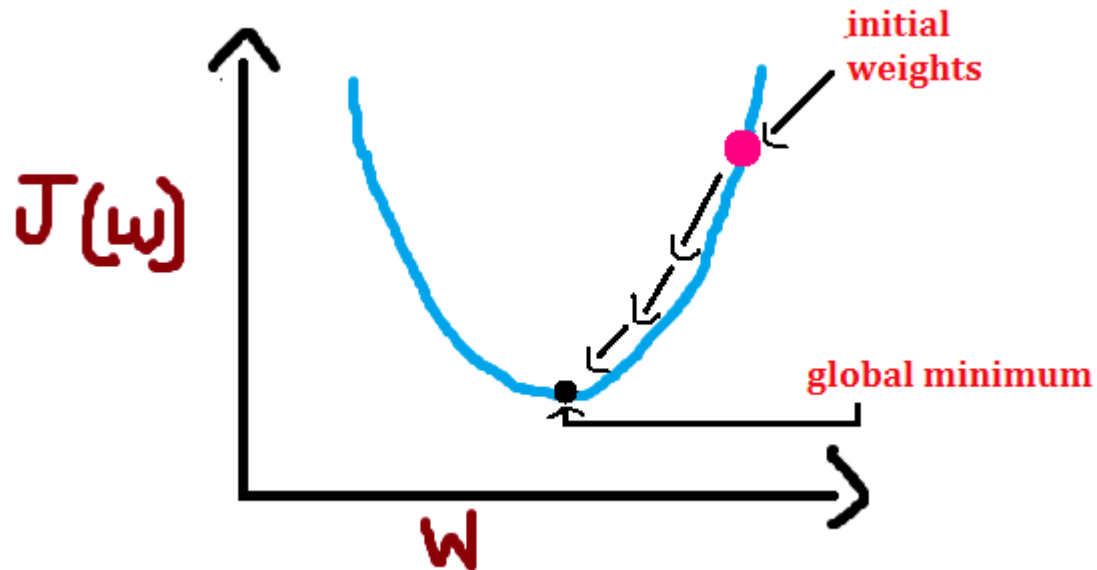
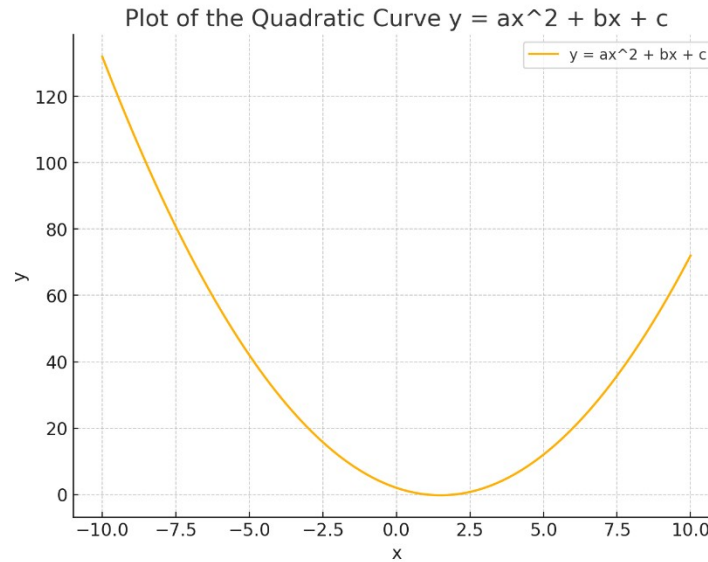
$$MSE = \frac{1}{n} \sum_{i=1}^n (ax_i^2 + bx_i + c - y_i)^2$$

Polynomial Regression



1. Polynomial Regression is a form of regression analysis in which the relationship between the independent variables and dependent variables are modeled in the n^{th} degree polynomial.
2. Polynomial Regression models are usually fit with the method of least squares. The least square method minimizes the variance of the coefficients, under the Gauss Markov Theorem.
3. Polynomial Regression is a special case of Linear Regression where we fit the polynomial equation on the data with a curvilinear relationship between the dependent and independent variables.

Polynomial Regression



Polynomial Regression

Minimizing the Sum of Squared Residuals (SSR)

In polynomial regression and many other types of regression models, the goal is to minimize the sum of squared residuals (SSR). This involves finding the set of coefficients (parameters) that result in the smallest possible difference between the predicted values and the actual values of the dependent variable. Let's delve into how this is accomplished:

Differentiation and the Normal Equations

1. Derivation:

To find the coefficients that minimize the SSR, you first formulate the SSR as a function of the coefficients. For a quadratic model $y = ax^2 + bx + c$, the SSR function would be:

$$SSR(a, b, c) = \sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2$$

2. Differentiation:

You differentiate this function with respect to each coefficient a , b , and c , set these derivatives to zero, and solve the resulting system of equations. These equations are known as the normal equations:

$$\frac{\partial SSR}{\partial a} = 0, \quad \frac{\partial SSR}{\partial b} = 0, \quad \frac{\partial SSR}{\partial c} = 0$$

Solving these gives the values of a , b , and c that minimize the SSR.

Polynomial Regression

1. Partial Derivative with Respect to a :

$$\frac{\partial SSR}{\partial a} = \frac{\partial}{\partial a} \sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2$$

Using the chain rule, this derivative simplifies to:

$$\frac{\partial SSR}{\partial a} = \sum_{i=1}^n -2x_i^2(y_i - (ax_i^2 + bx_i + c)) = 0$$

2. Partial Derivative with Respect to b :

$$\frac{\partial SSR}{\partial b} = \frac{\partial}{\partial b} \sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2$$

Similarly, applying the chain rule:

$$\frac{\partial SSR}{\partial b} = \sum_{i=1}^n -2x_i(y_i - (ax_i^2 + bx_i + c)) = 0$$

Polynomial Regression

Certainly! To clearly present the system of linear equations derived from setting the partial derivatives of the Sum of Squared Residuals (SSR) to zero in the context of polynomial regression, here it is formatted properly:

1. For the coefficient a :

$$\sum_{i=1}^n x_i^2 (y_i - (ax_i^2 + bx_i + c)) = 0$$

2. For the coefficient b :

$$\sum_{i=1}^n x_i (y_i - (ax_i^2 + bx_i + c)) = 0$$

Polynomial Regression

3. For the coefficient c :

$$\sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c)) = 0$$

These equations represent the "normal equations" for a quadratic regression model. Each equation is derived by taking the partial derivative of the SSR with respect to one of the coefficients a , b , and c , and setting it to zero. Solving this system simultaneously will provide the values of a , b , and c that minimize the SSR, thereby fitting the best possible quadratic curve to the given data set.

1. Gradient Descent:

- This is an iterative optimization algorithm used in finding the minimum of a function. Here, you start with initial guesses for a , b , and c , and iteratively update these values in the direction that reduces the SSR the most.
- **Process:** Each iteration updates the coefficients based on the gradient of the SSR at that point:

$$a_{new} = a_{old} - \eta \cdot \frac{\partial SSR}{\partial a}, \quad b_{new} = b_{old} - \eta \cdot \frac{\partial SSR}{\partial b}, \quad \text{etc.}$$

Where η is the learning rate, a small constant that determines the size of each update step.

Lasso and Ridge Regression

In data analysis and machine learning, developing accurate predictive models is crucial for making informed decisions and deriving valuable insights from data. This process typically involves several steps, among which the creation of training and test datasets, as well as the selection and evaluation of statistical models, are key components. Here's a detailed explanation of these processes:

Splitting Data into Training and Test Sets

1. **Training Set:** This subset of the data is used to train or fit the model. It includes both the input features (predictors) and the outputs (response variables) which the model learns to predict. The training set helps the model understand the underlying patterns and relationships in the data.
2. **Test Set:** After the model has been trained, it is important to evaluate its performance on unseen data to estimate how well it will perform in real-world scenarios. This is where the test set comes in. It is a separate portion of the data that the model has not seen during training. By comparing the model's predictions on the test set to the actual outcomes, you can assess the accuracy and generalizability of the model.

Lasso and Ridge Regression

Understanding Linear Regression

Linear regression is a fundamental statistical method used to model the relationship between a dependent variable and one or more independent variables (predictors). It assumes a linear relationship between the input (X) and output (Y), formulated as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

- $\beta_0, \beta_1, \dots, \beta_n$ are the coefficients that the model tries to learn.
- ϵ is the error term, which accounts for the variability in Y that cannot be explained by the predictors.

The Problem of Overfitting in Linear Regression

Overfitting occurs when a statistical model learns not only the underlying pattern but also the noise in the training data. This typically happens when the model is too complex relative to the amount of input data or the number of predictors is too high compared to the number of observations. Here's how overfitting affects the model:

- **High Variance:** Overfit models have high variance, which means they perform very well on the training data but poorly on any unseen test data. This is because the model's complexity allows it to fit the training data too closely, including the random fluctuations that aren't representative of the overall data distribution.
- **Poor Generalization:** An overfitted model is less able to generalize from the training data to new, unseen data. This is problematic for making predictions in practical applications where the model encounters data that differ from the data it was trained on.

Lasso and Ridge Regression

Addressing Overfitting

To prevent overfitting, particularly in the case of linear regression with many predictors, several strategies can be employed:

1. **Regularization:** Techniques such as Ridge (L2) and Lasso (L1) regularization add a penalty term to the loss function (the sum of squared differences between observed and predicted values). This penalty discourages overly complex models by penalizing the magnitude of the coefficients.
 - **Ridge Regression** adds a penalty proportional to the square of the coefficient magnitudes.
 - **Lasso Regression** adds a penalty proportional to the absolute value of the coefficients, which can reduce some coefficients to zero, effectively performing feature selection.

Lasso and Ridge Regression

Importance of Balance Between Observations and Predictors

In statistical modeling and machine learning, having a balanced ratio of observations to predictors is crucial for building effective models:

- **Too Few Observations:** If the number of predictors is high compared to the number of observations, there is a higher risk of overfitting. Overfitting occurs when a model is too complex, capturing noise instead of the underlying pattern in the data. Essentially, the model learns the details, including the noise and errors in the training data, to an extent that it performs poorly on new, unseen data.
- **Adequate Observations:** Having a sufficiently large number of observations relative to predictors helps in generalizing better. The model can learn the noise as well as the signal but is validated or tested on many unseen examples, which helps it learn to ignore random fluctuations in the data.

Lasso and Ridge Regression

The concept of "penalizing the complexity of a model" in statistical modeling and machine learning refers to deliberately adding constraints or penalties to a model to discourage it from becoming overly complex. This approach is aimed at preventing overfitting, where a model captures not just the underlying data trends but also the random noise within the training data.

Why Model Complexity Needs to be Penalized

1. **Overfitting Risks:** Complex models, such as those with many parameters or high-degree polynomials, can fit the training data very closely. They can adapt their predicted values to the noise or fluctuations in the data, which do not generalize to new, unseen data.
2. **Generalization:** Simplifying a model, or limiting its ability to overfit the training data, helps ensure that it performs well not just on the training data but also on new, unseen data.

Lasso and Ridge Regression

Methods to Penalize Complexity

Regularization: This is the most direct method to penalize complexity by modifying the loss function that the model optimizes. There are several approaches:

- **Ridge Regression (L2 Regularization):** Adds a penalty equal to the square of the magnitude of the coefficients. The penalty term is $\lambda \sum_{j=1}^p \beta_j^2$, where β_j are the coefficients, p is the number of predictors, and λ is a tuning parameter. This penalty discourages large values of coefficients.
- **Lasso Regression (L1 Regularization):** Adds a penalty equal to the absolute value of the magnitude of the coefficients, $\lambda \sum_{j=1}^p |\beta_j|$. This can result in setting some coefficients to zero, effectively reducing the number of predictors in the model.
- **Elastic Net:** Combines L1 and L2 regularization penalties. It is useful when there are correlations among features, or when you want to blend the feature elimination properties of Lasso with the regularization strength of Ridge.

Lasso and Ridge Regression

Ridge Regression (L2 Regularization)

Concept: Ridge regression is a method that introduces a penalty equivalent to the square of the magnitude of the coefficients to the loss function of linear regression. This technique is known as L2 regularization.

Mathematical Formulation:

$$\text{Objective} = \text{RSS} + \alpha \sum (\beta_j^2)$$

Where:

- **RSS** (Residual Sum of Squares) represents the loss without regularization.
- β_j are the coefficients of the model.
- α is a tuning parameter that determines the strength of the regularization. A higher α increases the penalty, shrinking the coefficients towards zero, but never exactly zero.

Lasso and Ridge Regression

Why Use Ridge?:

- It helps to reduce model complexity and prevent overfitting, which is crucial when the number of predictors (features) is large.
- It handles multicollinearity (high correlation among features) effectively by distributing coefficients among them.

Usage: Ridge is commonly used when the goal is to keep all variables in the model, making it especially useful in cases where each feature has some contribution to the output, but you want to penalize their weights to avoid overfitting.

in the context of Ridge Regression, the term "penalty" specifically refers to adding the sum of the squared values of the coefficients (β) to the loss function. This added term serves as a regularization mechanism designed to control the magnitude of the coefficients in the model.

Lasso and Ridge Regression

Mathematical Implication of Squaring

When you square a coefficient β_j , the resulting value (β_j^2) increases exponentially with the magnitude of β_j :

- **Small values** (between 0 and 1): Squaring these values makes them even smaller. For instance, $0.5^2 = 0.25$.
- **Large values** (greater than 1): Squaring amplifies them significantly. For example, $2^2 = 4$ and $3^2 = 9$.

This non-linear scaling means that the impact of large coefficients on the loss function is disproportionately larger compared to that of smaller coefficients.

Lasso and Ridge Regression

Effect on the Loss Function

In Ridge Regression, the regularization part of the loss function is given by:

$$\lambda \sum_{j=1}^p \beta_j^2$$

Where:

- β_j are the model coefficients,
- λ is a non-negative regularization parameter controlling the strength of the penalty,
- p is the number of coefficients.

Adding the squared coefficients into the loss function creates a substantial penalty for larger coefficients because:

- The squared term β_j^2 increases the value of the penalty exponentially with the coefficient magnitude.
- A larger β_j^2 contributes significantly to increasing the overall penalty term, thus increasing the total loss that the model optimization process seeks to minimize.

Lasso and Ridge Regression

Consequences for Model Training

During the training process, the objective is to minimize the total loss, which now includes both the residual sum of squares (RSS, measuring model fit) and the penalty term (controlling complexity).

The inclusion of β_j^2 means:

- **Minimizing Loss with Large Coefficients:** To achieve a lower total loss, the model must significantly reduce any large coefficients because they contribute exponentially to the penalty term. Thus, the model is forced to prioritize smaller, more stable coefficients that do not overfit the data.
- **Balance Between Fit and Regularization:** The model must find a balance between fitting the data well (minimizing RSS) and keeping coefficients small (minimizing the regularization term). This balance helps to prevent the model from fitting the noise in the training data, which would hinder its performance on new, unseen data.

Lasso and Ridge Regression

Lasso Regression

Concept: Lasso (Least Absolute Shrinkage and Selection Operator) Regression is similar to ridge regression but it uses an absolute value penalty instead of a square penalty. Lasso is particularly useful when we have a high number of features because it not only helps in reducing overfitting but can also perform feature selection.

Mathematical Formulation: The cost function for Lasso regression is given by:

$$\text{Cost Function} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

where the symbols have similar meanings as in ridge regression.

Lasso and Ridge Regression

How Lasso Encourages Sparse Solutions

1. **Effect of the Absolute Value Penalty:** The L1 penalty ($\lambda \sum |\beta_j|$) influences the optimization process by enforcing a cost proportional to the absolute size of the coefficients. Unlike the square of the coefficients used in Ridge, the absolute value grows linearly. This linear relationship makes it feasible for the optimization algorithm to set coefficients to zero to minimize the overall cost.
2. **Zeroing Out Coefficients:** As λ increases, the penalty for any non-zero coefficient becomes increasingly significant in the overall optimization calculation. There comes a point where setting some coefficients to zero (thus removing their contribution to the penalty entirely) is more beneficial for minimizing the total loss than retaining a small or moderately sized non-zero coefficient. This is particularly the case for coefficients corresponding to variables that provide less predictive power regarding the dependent variable.

Lasso and Ridge Regression

How Optimization Reduces Coefficients to Zero

1. **Cost Reduction at Zero:** The optimization algorithm seeks to minimize the total cost, which is a combination of the RSS and the penalty $\lambda \sum |\beta_j|$. For any coefficient β_j , if reducing β_j towards zero decreases the total cost more than keeping β_j at a small non-zero value, then the algorithm will reduce β_j to zero. This is often the case when:
 - The contribution of β_j to decreasing the RSS is minimal or less significant than the reduction in penalty achieved by setting β_j to zero.

Regression Ends